

Decentralized Multi-Authority Attribute-Based Inner-Product Functional Encryption: Noisy and Evasive Constructions from Lattices[†]

Jiaqi Liu, Yan Wang, Fang-Wei Fu

Abstract

We initiate the study of multi-authority attribute-based functional encryption for *noisy inner-product functionality*, and propose two new primitives: (1) multi-authority attribute-based (noisy) inner-product functional encryption (MA-AB(N)IPFE), and (2) multi-authority attribute-based evasive inner-product functional encryption (MA-ABevIPFE). The MA-AB(N)IPFE primitive generalizes the existing multi-authority attribute-based inner-product functional encryption schemes by Agrawal et al. [AGT21], by enabling *approximate* inner-product computation under decentralized attribute-based control. This newly proposed notion combines the approximate function evaluation of noisy inner-product functional encryption (IPFE) with the decentralized key-distribution structure of multi-authority attribute-based encryption. To better capture noisy functionalities within a flexible security framework, we formulate the MA-ABevIPFE primitive under a generic-model view, inspired by the evasive IPFE framework by Hsieh et al. [HLL24]. It shifts the focus from pairwise ciphertext indistinguishability to a more relaxed pseudorandomness-based game.

To support the above notions, we introduce two variants of lattice-based computational assumptions:

- The *evasive IPFE assumption* (evIPFE): it generalizes the assumption introduced in [HLL24] to the multi-authority setting and admits a reduction from the evasive LWE assumption proposed by Waters et al. [WWW22];
- The *indistinguishability-based evasive IPFE assumption* (IND-evIPFE): it is an indistinguishability-based variant of the evasive IPFE assumption designed to capture the stronger security guarantees required by our MA-AB(N)IPFE scheme.

We present concrete lattice-based constructions for both primitives supporting subset policies, building upon the framework of [WWW22]. Our schemes are proven to be statically secure in the random oracle model under the standard LWE assumption and the newly introduced assumptions. Additionally, we demonstrate that our MA-AB(N)IPFE scheme can be transformed, via standard modulus switching, into a *noiseless* MA-ABIPFE scheme that supports exact inner-product functionality consistent with the MA-ABIPFE syntax in [AGT21, DP23]. This yields the first lattice-based construction of such a primitive. All our schemes support arbitrary polynomial-size attribute policies and are secure in the random oracle model under lattice assumptions with a sub-exponential modulus-to-noise ratio, making them practical candidates for noise-tolerant, fine-grained access control in multi-authority settings.

Jiaqi Liu, Yan Wang and Fang-Wei Fu are with Chern Institute of Mathematics and LPMC, Nankai University, Tianjin 300071, P. R. China, Emails: ljqi@mail.nankai.edu.cn, yan.wang@mail.nankai.edu.cn, fwfu@nankai.edu.cn.

[†]This research is supported by the National Key Research and Development Program of China (Grant No. 2022YFA1005000), the National Natural Science Foundation of China (Grant Nos. 12141108 and 62371259), the Fundamental Research Funds for the Central Universities of China (Nankai University), and the Nankai Zhide Foundation.

manuscript submitted May 16, 2025

CONTENTS

I	Introduction	3
I-A	Related Works	4
I-B	Our Results	5
I-C	Paper Organization	7
II	Technical Overviews	7
II-A	Our Schemes	8
II-B	Static Security for Our Scheme	9
III	Preliminaries	12
III-A	Notations	12
III-B	Lattice Preliminaries	13
III-C	LWE Assumption and Its Variants	15
IV	Evasive IPFE Assumptions	15
IV-A	The Evasive IPFE Assumption	15
IV-B	The Indistinguishability-Based Evasive IPFE Assumption	17
V	Multi-Authority Attribute-Based Inner Product Functional Encryption	19
V-A	MA-AB(N)IPFE	19
V-B	MA-ABevIPFE	23
VI	MA-ABevIPFE Scheme from evIPFE Assumption (in the Random Oracle Model)	25
VI-A	Correctness	26
VI-B	Security	27
VI-C	Parameters	47
VII	MA-ABNIPFE Scheme from IND-evIPFE Assumption (in the Random Oracle Model)	47
VII-A	Correctness	48
VII-B	Security	48
VII-C	Parameters	57
VIII	Noiseless MA-ABIPFE Scheme from IND-evIPFE Assumption (in the Random Oracle Model)	57
VIII-A	Correctness	58
VIII-B	Static Security	59
VIII-C	Parameters	59
Appendix A:	Hardness of Evasive IPFE	62
A-A	Evasive LWE from [WWW22]	62
A-B	Reduction from evLWE ([WWW22]) to evIPFE	62

I. INTRODUCTION

Functional Encryption (FE), introduced by Boneh, Sahai, and Waters [BSW11], is a versatile cryptographic paradigm that extends the capabilities of traditional public-key encryption schemes. It facilitates fine-grained access control over encrypted data, enabling authorized users to compute specific functions of the plaintext without recovering the entire plaintext. More formally, in an FE scheme, each secret key sk_f is associated with a particular function f . Given an encryption $\text{Enc}(\text{mpk}, x)$ of a message x encrypted under the master public key mpk , decrypting with sk_f reveals only the value $f(x)$ without leaking any additional information about the message x . The inherent property of selective disclosure in FE makes it particularly valuable in applications involving sensitive or confidential data. For example, in healthcare settings, FE allows researchers to aggregate statistics from encrypted patient records while maintaining the confidentiality of individual data entries.

The standard security framework for FE is *indistinguishability-based security* (IND). In this model, an adversary attempts to distinguish between the encryptions of two selected messages x_0 and x_1 , while being allowed to query secret keys sk_f for functions f such that $f(x_0) = f(x_1)$. This restriction ensures that the adversary gains no additional information beyond the outputs of permitted functions. The security guarantee requires that the adversary remain unable to distinguish between the ciphertexts, even when given access to multiple such keys.

Inner-Product Functional Encryption. A notable subclass of FE schemes designed for computing linear functions is known as *Inner-Product Functional Encryption* (IPFE), which has become an active area of research over the past decade. The study of IPFE began with the work of Abdalla et al. [ABDP15] and has since been extensively studied in a series of works [ABDP16, ALS16, DDM16, AGRW17, BBL17, CSG⁺18, ACF⁺18, LT19, ABKW19, Tom19, ABM⁺20]. In an IPFE scheme, a ciphertext ct_u encrypts an ℓ -dimensional vector $u \in \mathcal{R}^\ell$ over some ring $\mathcal{R}$, while a secret key sk_v for a vector $v \in \mathcal{R}^\ell$ enables the computation of the inner-product function $f_v(\cdot) = \langle \cdot, v \rangle$. Decrypting ct_u using sk_v yields the value $\langle u, v \rangle$, without leaking any other information about the message u . IPFE provides a powerful cryptographic tool for selective computation over encrypted data, ensuring that sensitive information remains protected. Notable applications include secure data analysis, privacy-preserving machine learning, and privacy-enhanced database queries. In addition, IPFE serves as a foundational building block for more advanced cryptographic primitives, such as FE for quadratic functions [JLS19, Gay20] and *attribute-based encryption* (ABE) [WFL19, HLL24]. These extensions further enhance the versatility and applicability of functional encryption.

Building on IPFE, Agrawal [Agr19, AP20] introduced *Noisy Inner-Product Functional Encryption* (NIPFE), an extension that incorporates noise into the computation process. Unlike the standard IPFE scheme in [ABDP15], which supports the *exact* evaluation of inner products, NIPFE enables the computation of *approximate* inner products with an additive noise term of the form $\langle u, v \rangle + e$, where e is a small-norm noise term. This relaxation allows decryption to return an approximate result, which improves flexibility in noisy settings. The security guarantee of NIPFE ensures that for any two plaintext vectors u and u' , and key vectors $v_1, \dots, v_m$, if $\langle u, v_j \rangle \approx \langle u', v_j \rangle$ holds for each j , the ciphertexts of u and u' remain computationally indistinguishable, even when the adversary is given the secret keys corresponding to v_j . The security property is achieved by leveraging the noise term e , which effectively “smudges” small differences in the inner-product computations, thereby preventing the adversary from extracting meaningful distinguishing information. NIPFE provides a security guarantee in applications in which a certain level of noise tolerance is acceptable, balancing between utility and privacy.

Attribute-Based Inner-Product Functional Encryption. *Attribute-Based Inner-Product Functional Encryption* (AB-IPFE), introduced by Abdalla et al. [ACGU20], is an advanced cryptographic primitive that combines the access control capabilities of ABE with the inner-product computation functionality of IPFE. This hybrid notion enables fine-grained access control over encrypted data while supporting privacy-preserving computations on ciphertexts. In a *ciphertext-policy* AB-IPFE scheme, each ciphertext

is associated with a set of attributes, and each secret key corresponds to an access policy defined over the attribute universe, following a structure similar to that of a ciphertext-policy ABE. In the dual setting, namely the *key-policy* AB-IPFE scheme, ciphertexts are associated with access policies, and secret keys are associated with attribute sets. In either case, decryption is allowed only when the attributes satisfy the access policy. Unlike traditional IPFE, AB-IPFE allows inner-product computations on encrypted data only when decryption is authorized, making it particularly useful for applications such as privacy-preserving machine learning, secure data analytics, and encrypted search. By integrating the strengths of ABE and IPFE, AB-IPFE offers a powerful tool for secure data sharing and processing in multi-user environments, with practical relevance to applications such as cloud computing, the Internet of Things (IoT), and healthcare systems.

Multi-Authority Attribute-Based Inner-Product Functional Encryption. Most prior work on ABE or AB-IPFE focuses on the single-authority setting, where a central trusted authority is the only entity responsible for validating user attributes and issuing the corresponding secret keys. However, in many practical scenarios, it is more natural for multiple separate authorities to independently manage and authorize different subsets of attributes for users. *Multi-Authority* ABE (MA-ABE) [LW11, RW15, DKW21a, DKW21b, WWW22] addresses this limitation by decentralizing the authority structure in an ABE system, allowing multiple independent authorities to operate and manage attribute-based access control. This decentralization improves the scalability and flexibility of the system. Motivated by the need for secure and privacy-preserving computation in decentralized environments, recent efforts have explored the integration of MA-ABE and IPFE into a unified framework known as *Multi-Authority Attribute-Based Inner-Product Functional Encryption* (MA-ABIPFE). Agrawal et al. [AGT21] first highlighted the potential of combining multi-authority systems with functional encryption by extending the attribute-based component of AB-IPFE to a multi-authority setting. MA-ABIPFE is a significant cryptographic framework that unifies the decentralized access control of MA-ABE and the computation functionality of IPFE.

Informally, in an MA-ABIPFE scheme, each authority generates its own master secret key and is responsible for issuing secret keys associated with attributes it governs. Let $\text{ct}_f(\mathbf{u})$ be a ciphertext encrypting a plaintext vector $\mathbf{u} \in \mathcal{R}^\ell$ under an access policy P . A user holding secret keys corresponding to a vector $\mathbf{v} \in \mathcal{R}^\ell$ and a set of attributes A can recover the inner product $\langle \mathbf{u}, \mathbf{v} \rangle$ if and only if A satisfies the policy P (i.e., A is *authorized*); otherwise, the ciphertext reveals no information about $\mathbf{u}$.

A. Related Works

The MA-ABIPFE scheme proposed in [AGT21] lifts the AB-IPFE construction in [ACGU20] to the multi-authority setting. It is known as the first nontrivial multi-authority FE scheme beyond MA-ABE. It supports access policies that can be realized by *Linear Secret Sharing Schemes* (LSSS) and is constructed using pairing-based techniques. Its security relies on variants of the subgroup decision assumptions over composite-order bilinear groups introduced in [BSW13].

Subsequently, Datta and Pal [DP23] proposed two MA-ABIPFE schemes that also support LSSS-based access structures. These schemes are built in the more efficient prime-order bilinear group setting and rely on the well-studied *Decisional Bilinear Diffie-Hellman Assumption* (DBDH) and its variants. Their constructions are proven secure in the Random Oracle Model. By moving to the prime-order group setting, their constructions achieve notable improvements in efficiency over the scheme of [AGT21]. The construction supports computations over vectors of a priori unbounded length and an unbounded number of authorities. Furthermore, the schemes in [DP23] overcome the “one-use” restriction, allowing attributes to appear arbitrarily many times within access policies, thereby providing greater flexibility and versatility in policy design.

Evasive LWE Assumption. *Evasive Learning with Errors* (Evasive LWE) [Wee22] is a non-standard variant of the LWE assumption. Roughly, the assumption states that for any efficient sampling algorithm

Samp that outputs a matrix $\mathbf{Q} \in \mathbb{Z}_q^{n \times t}$,

$$\begin{aligned} \text{if } (\mathbf{A}, \mathbf{Q}, \mathbf{s}^\top \mathbf{A} + \mathbf{e}_1^\top, \mathbf{s}^\top \mathbf{Q} + \mathbf{e}_2^\top, \text{aux}) &\approx (\mathbf{A}, \mathbf{Q}, \$1, \$2, \text{aux}), \\ \text{then } (\mathbf{A}, \mathbf{Q}, \mathbf{s}^\top \mathbf{A} + \mathbf{e}_1^\top, \mathbf{A}^{-1}(\mathbf{Q}), \text{aux}) &\approx (\mathbf{A}, \mathbf{Q}, \$1, \mathbf{A}^{-1}(\mathbf{Q}), \text{aux}), \end{aligned} \quad (1)$$

for uniformly random matrix $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, uniformly random LWE secret $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$, Gaussian noise vectors $\mathbf{e}_1 \in \mathbb{Z}_q^m, \mathbf{e}_2 \in \mathbb{Z}_q^t$, and uniformly random vectors $\$1 \in \mathbb{Z}_q^m, \$2 \in \mathbb{Z}_q^t$. Here, $\mathbf{A}^{-1}(\mathbf{Q})$ denotes a low-norm (typically Gaussian) preimage of $\mathbf{Q}$ with respect to $\mathbf{A}$, i.e., a matrix with low-norm entries such that $\mathbf{A} \cdot (\mathbf{A}^{-1}(\mathbf{Q})) = \mathbf{Q}$. In [Wee22], the assumption is formulated for *public-coin* sampling algorithms Samp, meaning that the auxiliary string aux contains all the tossed coins (randomness) used by Samp.

Intuitively, the evasive LWE assumption essentially asserts that the only meaningful way to exploit the preimage $\mathbf{A}^{-1}(\mathbf{Q})$ is to multiply it by the LWE sample $\mathbf{s}^\top \mathbf{A} + \mathbf{e}_1^\top$, obtaining

$$(\mathbf{s}^\top \mathbf{A} + \mathbf{e}_1^\top) \cdot (\mathbf{A}^{-1}(\mathbf{Q})) \approx \mathbf{s}^\top \mathbf{Q},$$

and then attempting to distinguish this value from uniform samples. However, the precondition (1) in the assumption guarantees that this advantage remains negligible, thereby preventing zeroizing attacks such as those discussed in [CHL⁺15, CVW18, HJL21, JLLS23].

In follow-up work, [WWW22] proposed a variant of public-coin evasive LWE assumption involving multiple matrix pairs $(\mathbf{A}_i, \mathbf{Q}_i)_i$, along with their respective LWE samples $(\mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top, \mathbf{s}_i^\top \mathbf{Q}_i + \mathbf{e}_{2,i}^\top)_i$. Roughly, the evasive LWE assumption in [WWW22] states that if

$$\begin{aligned} \text{if } (\{\mathbf{A}_i, \mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top\}_i, \{\mathbf{s}_i^\top \mathbf{Q}_i + \mathbf{e}_{2,i}^\top\}_i, \text{aux}) &\approx (\{\mathbf{A}_i, \$1,i\}_i, \{\$2,i\}_i, \text{aux}), \\ \text{then } (\{\mathbf{A}_i, \mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top\}_i, \{\mathbf{A}_i^{-1}(\mathbf{Q}_i)\}_i, \text{aux}) &\approx (\{\mathbf{A}_i, \$1,i\}_i, \{\mathbf{A}_i^{-1}(\mathbf{Q}_i)\}_i, \text{aux}), \end{aligned}$$

where $\mathbf{s}^\top = [\mathbf{s}_1^\top \mid \cdots \mid \mathbf{s}_\ell^\top] \xleftarrow{\$} \mathbb{Z}_q^{n\ell}$ is an LWE secret.

Evasive IPFE. *Evasive Inner-Product Functional Encryption* (Evasive IPFE), introduced by Yao-Ching Hsieh, Huijia Lin, and Ji Luo in [HLL24], extends the standard IPFE framework [ABDP15]. Similar to the NIPFE scheme [Agr19], evasive IPFE encrypts an ℓ -dimensional vector $\mathbf{u} \in \mathcal{R}^\ell$ over some ring $\mathcal{R}$ into a ciphertext $\text{ct}_\mathbf{u}$, and allows the generation of secret keys $\{\text{sk}_{\mathbf{v}_j}\}_j$ for vectors $\{\mathbf{v}_j\}_j$. Decryption using $\text{sk}_{\mathbf{v}_j}$ yields noisy inner products of the form $\langle \mathbf{u}, \mathbf{v}_j \rangle + e_j$, where each e_j is a small-norm noise term, while revealing no additional information about $\mathbf{u}$. The key distinction lies in the underlying security model, which reflects a *generic-model view* analogous to that of the evasive LWE assumption. Specifically,

$$\begin{aligned} \text{if } \{\mathbf{v}_j, \langle \mathbf{u}, \mathbf{v}_j \rangle + e_j\}_j &\approx \{\mathbf{v}_j, \$j\}_j, \\ \text{then } (\text{ct}_\mathbf{u}, \{\mathbf{v}_j, \text{sk}_{\mathbf{v}_j}\}_j) &\approx (\text{ct}_\$, \{\mathbf{v}_j, \text{sk}_{\mathbf{v}_j}\}_j), \end{aligned}$$

where each e_j is small-norm noise term, and each $\$j$ and $\$$ is a uniformly random elements over the appropriate range. In other words, if the noisy inner products with $\mathbf{u}$ are pseudorandom, then the ciphertext encrypting $\mathbf{u}$ is also pseudorandom, even given the key-vector pairs $\{\mathbf{v}_j, \text{sk}_{\mathbf{v}_j}\}_j$.

B. Our Results

In this work, we make the following conceptual and technical contributions: We begin by proposing two new notions for attribute-based functional encryption in the multi-authority setting (Section V).

- **Multi-authority attribute-based (noisy) inner-product functional encryption** (MA-AB(N)IPFE): We propose the notion of MA-ABNIPFE, a natural generalization of the MA-ABIPFE scheme [AGT21, DP23] by allowing the *approximate* computation of inner products rather than *exact* ones when decrypting using authorized secret keys. The MA-AB(N)IPFE primitive integrates two core features: (i) the approximate inner-product functionality of NIPFE, and (ii) the decentralized access control mechanism of MA-ABE, where secret keys are issued by multiple authorities based on their attribute

sets. This combination enables fine-grained, decentralized access control over approximate inner-product computations in a multi-authority setting.

We then formalize the security of the MA-AB(N)IPFE scheme in the static setting, following the framework of [RW15, WWW22]. In this model (also referred to as the *selective* model) the adversary must commit to its challenge plaintexts, secret-key queries, and authority corruption choices immediately after the global parameters are initialized. Unlike the standard MA-ABE setting, where secret-key queries must be unauthorized to decrypt the challenge ciphertext, our static security model permits the adversary to request even *authorized* secret keys. However, such secret keys yield sufficiently *close* inner products when evaluated over the two challenge plaintexts. This relaxed notion is tailored to the noisy functional nature of our scheme. It ensures that no efficient adversary can distinguish between the encryptions of two plaintexts unless it can produce keys that yield meaningfully different decryption results.

- **Multi-authority attribute-based evasive inner-product functional encryption (MA-ABevIPFE):** We further introduce the notion of MA-ABevIPFE, which serves as a relaxed variant of MA-AB(N)IPFE in terms of its underlying security definition. While both primitives enable approximate inner-product computation in a multi-authority setting, the MA-ABevIPFE scheme adopts a “generic-model view” inspired by recent works on evasive IPFE in [HLL24]. Instead of requiring the adversary to distinguish between two challenge ciphertexts, the security model is formulated via a pseudorandomness-based game: the goal of the adversary is to distinguish the ciphertext of a structured plaintext generated by a public-coin sampler from that of a uniformly random plaintext.

The main difference between the adversarial capabilities in the MA-AB(N)IPFE and MA-ABevIPFE games lies in the secret-key queries. For unauthorized queries, both models handle them identically. However, for authorized queries, the adversary in MA-ABevIPFE is more restricted. Specifically, it does not have full control over both the attribute set and the key vector $\mathbf{v}$ that together define the secret-key queries. Instead, the adversary specifies only the attribute set, while the key vector $\mathbf{v}$ is sampled by a public-coin sampler $\text{Samp}_{\mathbf{v}}$. This restriction mirrors the structure of evasive IPFE games. In addition, the challenge plaintext $\mathbf{u}$ is sampled by another public sampling algorithm $\text{Samp}_{\mathbf{u}}$ with private randomness (i.e., the adversary does not have access to $\mathbf{u}$). The security definition guarantees that if the sampler pair $(\text{Samp}_{\mathbf{v}}, \text{Samp}_{\mathbf{u}})$ produces noisy pseudorandom inner products, i.e.,

$$(\mathbf{r}_{\text{pub}}, \langle \mathbf{u}, \mathbf{v} \rangle + e) \approx (\mathbf{r}_{\text{pub}}, \$),$$

then no efficient adversary can distinguish the ciphertext of $\mathbf{u}$ and that of a uniformly random plaintext with non-negligible advantage. Here $\mathbf{r}_{\text{pub}}$ denotes the randomness used by $\text{Samp}_{\mathbf{v}}$, e is a noise term, and $\$$ is a uniformly random element.

To instantiate the above notions, we provide three concrete lattice-based constructions, supporting approximate inner-product computation under subset policies with decentralized access control, offering different trade-offs between noise management and security guarantees.

Construction of MA-ABevIPFE. We construct an MA-ABevIPFE scheme for subset policies, following the framework of [WWW22]. Our construction is proven to be statically secure in the random oracle model under the standard LWE assumption and a new evasive IPFE assumption (evIPFE) introduced in this work, which generalizes the evasive IPFE assumption proposed in [HLL24]. The idea of evasive IPFE assumption is conceptually motivated by the evasive LWE assumption and admits a reduction from the evasive LWE assumption from [WWW22] (cf. Appendix A). Full details of the construction, security proof, and parameter selection are provided in Section VI.

Theorem 1 (Informal). *Suppose that the LWE assumption and the evLWE assumption (cf. Section IV) hold with sub-exponential modulus-to-noise ratio, then there exists a statically secure MA-ABevIPFE scheme for subset policies of arbitrary polynomial size in the random oracle model.*

Construction of MA-ABNIPFE. We then construct an MA-ABNIPFE scheme for subset policies. This construction follows the same syntax as the MA-ABevIPFE scheme but achieves a stronger notion of security under different lattice-based assumptions. Specifically, it is proven to be statically secure in the random oracle model under the LWE assumption and the new IND-evIPFE assumption (cf. Section IV) introduced in this work. The IND-evIPFE assumption is an indistinguishability-based variant of the evIPFE discussed earlier. Full details of the construction, security analysis, and parameter selection are provided in Section VII.

Theorem 2 (Informal). *Suppose that the LWE assumption and the IND-evIPFE assumption (cf. Section IV) hold with sub-exponential modulus-to-noise ratio, then there exists a statically secure MA-ABNIPFE scheme for subset policies of arbitrary polynomial size in the random oracle model.*

Construction of Noiseless MA-ABIPFE. Finally, we modify our MA-ABNIPFE scheme into a noiseless MA-ABIPFE scheme, which enables *exact* computation of the inner products upon successful decryption. This construction aligns with the definition of MA-ABIPFE from [AGT21, DP23]. The construction proposed in this work is the first such scheme based on lattice-related assumptions. Our modification relies on a standard modulus switching technique that removes the noise introduced during decryption. The resulting scheme retains a similar overall structure but achieves exact correctness. The scheme is proven to be statically secure in the random oracle model, under the LWE assumption and the IND-evIPFE assumption. Full details of the construction, security analysis, and parameter selection are provided in Section VIII.

Theorem 3 (Informal). *Suppose that the LWE assumption and the IND-evIPFE assumption (cf. Section IV) hold with sub-exponential modulus-to-noise ratio, then there exists a statically secure noiseless MA-ABIPFE scheme for subset policies of arbitrary polynomial size in the random oracle model.*

C. Paper Organization

The paper is organized as follows. Section II provides a technical overview of our main constructions and ideas. Section III introduces necessary notations, basic concepts of lattices, the LWE assumption, and its variants. Section IV, presents two variants of the Evasive IPFE assumptions, building on the framework of [HLL24]. In Section V, we formalize monotone access structures and give the definitions of multi-authority attribute-based evasive inner-product functional encryption (MA-ABevIPFE) and multi-authority attribute-based (noisy) inner-product functional encryption (MA-AB(N)IPFE). Section VI presents our construction of an MA-ABevIPFE scheme. Section VII introduces a construction of an MA-ABNIPFE scheme. Section VIII describes a construction of a noiseless MA-ABIPFE scheme using the modulus-switching technique. Sections VI through VIII include detailed analyses of correctness, security, and parameter selection.

II. TECHNICAL OVERVIEWS

In this section, we demonstrate a high-level overview of our construction of multi-authority attribute-based (noisy/evasive) inner-product functional encryption with subset policies in the random oracle model. Our construction is inspired by the frameworks of [WWW22] and [HLL24].

Preliminaries. We introduce some notations used throughout this section. To simplify expressions involving noise, we adopt the wavy underline to indicate that a term is perturbed by some noise term. For example, we use the notation $\underline{s^\top \mathbf{A}}$ to denote the term $s^\top \mathbf{A} + \mathbf{e}$ for an unspecified noise vector $\mathbf{e}$.

For a discrete set S , we denote $x \stackrel{\$}{\leftarrow} S$ as $\mathbf{x}$ is uniformly sampled from the set S . The term $\mathbf{A}^{-1}(\mathbf{y})$ means sampling a short preimage $\mathbf{x}$ such that $\mathbf{A}\mathbf{x} = \mathbf{y}$. For a positive integer $n \in \mathbb{N}$, denote $[n]$ as the set $\{k \in \mathbb{N} : 1 \leq k \leq n\}$. Let $\$$ denote a uniformly random vector of appropriate dimension.

Let $\mathbf{G}_n = \mathbf{I}_n \otimes \mathbf{g}^\top \in \mathbb{Z}_q^{n \times n^{\lceil \log q \rceil}}$ denote the gadget matrix, where $\mathbf{g}^\top = (1, 2, \dots, 2^{\lceil \log q \rceil - 1})$. The inverse function $\mathbf{G}_n^{-1}$ maps the vectors in $\mathbb{Z}_q^n$ to their binary expansions in $\{0, 1\}^{n^{\lceil \log q \rceil}}$. We will omit the subscript when the dimension n is clear. We use $\approx^c$ as the abbreviation for computationally indistinguishable.

A. Our Schemes

Let $[L]$ denote the index set of all authorities. In the subset policy setting, each ciphertext is associated with a subset $X \subseteq [L]$, while each secret key is associated with a subset $Y \subseteq [L]$ and a key vector $\mathbf{v} \in \mathbb{Z}_q^n$. Decryption of a ciphertext encrypting $\mathbf{u} \in \mathbb{Z}_q^n$ associated with X using the secret key corresponding to Y and $\mathbf{v} \in \mathbb{Z}_q^n$ yields an approximate value of the Euclidean inner product $\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v}$ whenever $X \subseteq Y$.

We next provide an informal description of our construction. The scheme consists of the following components:

- The master public key for the authority indexed by i is given by $(\mathbf{A}_i, \mathbf{B}_i, \mathbf{P}_i) \xleftarrow{\$} \mathbb{Z}_q^{n \times m} \times \mathbb{Z}_q^{n \times m'} \times \mathbb{Z}_q^{n \times m}$.
- The master secret key for the authority indexed by i is the trapdoor $\text{td}_{\mathbf{A}_i}$ for the matrix $\mathbf{A}_i$. This trapdoor allows efficient sampling of short preimages for $\mathbf{A}_i$.
- Given a subset $Y \subseteq [L]$, a key vector $\mathbf{v} \in \mathbb{Z}_q^n$, and a global identifier gid , each authority $i \in Y$ uses its trapdoor to generate the secret key component corresponding to i . The corresponding secret key is generated as

$$\text{sk} \leftarrow \{\mathbf{A}_i^{-1}(\mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{B}_i \text{H}(\text{gid}, \mathbf{v}))\}_{i \in Y}.$$

Here H is a hash function modeled as a random oracle that outputs vectors in $\mathbb{Z}_q^{m'}$ with small norm.

- To encrypt a message $\mathbf{u} \in \mathbb{Z}_q^n$ under an attribute set $X \subseteq [L]$, the ciphertext is generated as

$$\text{ct} \leftarrow \left(\underbrace{\{\mathbf{s}_i^\top \mathbf{A}_i\}}_{i \in X}, \underbrace{\sum_{i \in X} \mathbf{s}_i^\top \mathbf{B}_i}, \underbrace{\sum_{i \in X} \mathbf{s}_i^\top \mathbf{P}_i + \mathbf{u}^\top \mathbf{G}} \right),$$

where $\mathbf{s}_i \xleftarrow{\$} \mathbb{Z}_q^n$ for each $i \in X$.

- The decryption operation using sk is performed as follows. It first computes $\mathbf{r} \leftarrow \text{H}(\text{gid}, \mathbf{v})$, and then computes

$$-\sum_{i \in X} \underbrace{\mathbf{s}_i^\top \mathbf{A}_i} \cdot \mathbf{A}_i^{-1}(\mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{B}_i \mathbf{r}) + \left(\underbrace{\sum_{i \in X} \mathbf{s}_i^\top \mathbf{B}_i} \right) \cdot \mathbf{r} + \left(\underbrace{\sum_{i \in X} \mathbf{s}_i^\top \mathbf{P}_i + \mathbf{u}^\top \mathbf{G}} \right) \cdot \mathbf{G}^{-1}(\mathbf{v}) \approx \mathbf{u}^\top \mathbf{v}$$

whenever $Y \supseteq X$.

The randomization of gid and $\mathbf{v}$. To prevent collusion among users in the system who might attempt to combine their individual secret keys to decrypt ciphertexts, which contradicts the intended security requirements of our setting, we adopt the *global identifier model* (gid), which is a standard technique widely used in multi-authority cryptographic schemes.

In the gid model, each user is assigned a unique and efficiently verifiable global identifier during setup, which remains fixed once global parameters are established. Upon verifying a user's global identifier, the authority validates his access rights and issues secret keys only for attributes authorized for that user. When instantiated in the random oracle model, the hash value of the global identifier $\text{H}(\text{gid})$ provides consistent randomness across secret keys issued by different authorities, thereby preventing unauthorized key combination. In our MA-ABIPFE setting, we further extend this model by additionally incorporating the key vector $\mathbf{v}$ as part of the input to the hash function. This ensures that secret keys are randomized with respect to both the user's identity and their functional target, further mitigating potential collusion attacks. We elaborate on this approach and propose our constructions in Sections VI~VIII.

B. Static Security for Our Scheme

In the following, we provide a high-level overview demonstrating that the proposed scheme is statically secure under certain lattice-based assumptions. The static security model of MA-AB(N)IPFE follows a structure similar to that of MA-ABE. Specifically, the security game involves a set of corrupt authorities $\mathcal{C} \subseteq [L]$. The adversary is allowed to arbitrarily assign the public keys and master secret keys for these corrupt authorities. Let $X \subseteq [L]$ denote the subset of authorities associated with the challenge ciphertexts. The adversary's goal is to distinguish between the encryptions of two selected vectors $\mathbf{u}_0$ and $\mathbf{u}_1$ under the authority set X .

Within this setting, the adversary is also allowed to submit secret-key queries of the form $\{(\text{gid}, Y, \mathbf{v})\}$, where Y is a subset of non-corrupt (honest) authorities, gid is a global user identifier, and $\mathbf{v}$ is a key vector. Each secret-key query must satisfy at least one of the two following conditions:

- (a) $(Y \cup \mathcal{C}) \cap X \not\subseteq X$: This condition reflects the multi-authority aspect of the setting. The security requirement states that even if the adversary has access to secret keys with authorities that do not satisfy the policy (unauthorized keys), the ciphertexts remain indistinguishable, aligning with the static security notion in the MA-ABE scheme.
- (b) $(\mathbf{u}_0 - \mathbf{u}_1)^\top \mathbf{v} \approx 0$: This condition enforces the inner-product functional encryption scheme requirement. It guarantees that secret keys satisfying the policy reveal only an approximate value of the inner-product $\mathbf{u}_i^\top \mathbf{v}$ ($i = 0, 1$), and no additional information beyond that.

Weakened Static Security: MA-ABevIPFE Scheme. Inspired by the evasive IPFE framework introduced in [HLL24], we begin our analysis with a weaker notion of static security, under which we define the MA-ABevIPFE scheme. While sharing the same syntax as the MA-AB(N)IPFE, this primitive imposes weaker adversarial control. In particular, the queries mentioned above are no longer entirely determined by the adversary. More precisely, the secret-key queries satisfying condition (b) are jointly generated by both the adversary and the challenger, while the challenge plaintext $\mathbf{u}_0$ is generated by a public-coin sampler and $\mathbf{u}_1$ is uniformly sampled. This setting reflects a “generic-model view” of the scheme, analogous to evasive LWE [Wee22] and evasive IPFE [HLL24]. The challenge plaintext $\mathbf{u}_0$ and the secret key queries $\{\text{sk}_{\mathbf{v}_j}\}_j$ (associated with key vector $\mathbf{v}_j$) satisfying condition (b) are generated such that

$$\{\mathbf{v}_j, \{\mathbf{u}_0^\top \mathbf{v}_j\}_j\} \stackrel{c}{\approx} \{\mathbf{v}_j, \{\$j\}_j\}.$$

The security requirement in this model is that the adversary cannot distinguish between the ciphertext of $\mathbf{u}_0$ and that of uniformly generated $\mathbf{u}_1$, even when it has access to the corresponding secret keys of its queries.

Our Assumption: Generalized Evasive IPFE. We prove the static security of our construction as an MA-ABevIPFE scheme under the standard LWE assumption and the evIPFE assumption, which we formally introduce in Section IV. This assumption can be viewed as an extension of the evasive IPFE assumption proposed by [HLL24]. We begin by informally recalling the assumption from [HLL24]. Let $\mathbf{v}_1, \dots, \mathbf{v}_k$ be vectors generated by a public-coin sampler with public randomness $\mathbf{r}_{\text{pub}}$, and let $\mathbf{u}$ be generated by a private-coin sampler. Let $\mathbf{A}, \mathbf{P}$ be uniformly random matrices, and define

$$\mathbf{Q} := \mathbf{P}[\mathbf{G}^{-1}(\mathbf{v}_1) \mid \dots \mid \mathbf{G}^{-1}(\mathbf{v}_k)].$$

The assumption states that if the following *precondition* holds:

$$(\mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{s}^\top \mathbf{A}, \mathbf{P}, \mathbf{s}^\top \mathbf{P} + \mathbf{u}^\top \mathbf{G}, \mathbf{s}^\top \mathbf{Q}) \stackrel{c}{\approx} (\mathbf{r}_{\text{pub}}, \mathbf{A}, \$1, \mathbf{P}, \$2, \$3).$$

then the following *postcondition* also holds:

$$(\mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{s}^\top \mathbf{A}, \mathbf{P}, \mathbf{s}^\top \mathbf{P} + \mathbf{u}^\top \mathbf{G}, \mathbf{K}) \stackrel{c}{\approx} (\mathbf{r}_{\text{pub}}, \mathbf{A}, \$1, \mathbf{P}, \$2, \mathbf{K}),$$

where $\mathbf{K} \leftarrow \mathbf{A}^{-1}(\mathbf{Q})$ and $\mathbf{r}_{\text{pub}}$ consists of all public randomness used by the samplers. The intuition behind this assumption is that the only way to use the matrix $\mathbf{K}$ is via computing

$$\mathbf{s}^\top \mathbf{A} \mathbf{K} \approx \mathbf{s}^\top \mathbf{Q},$$

which is pseudorandom by the precondition, and hence rules out potential zeroizing attacks.

We now generalize this assumption in two stages:

- 1) **Introducing an auxiliary matrix $\mathbf{B}$.** Let $\mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{P}, \mathbf{u}, \mathbf{v}_1, \dots, \mathbf{v}_k$ be the same as in the original assumption. Let $\mathbf{B}$ be an additional uniformly random matrix, and let $\mathbf{r}_1, \dots, \mathbf{r}_k$ be generated by a public-coin algorithm. Let $\mathbf{Q}$ be modified as

$$\mathbf{Q} = [\mathbf{B} \mid \mathbf{P}] \left[\begin{array}{c|c} \mathbf{r}_1 & \cdots \\ \mathbf{G}^{-1}(\mathbf{v}_1) & \mathbf{G}^{-1}(\mathbf{v}_k) \end{array} \right].$$

The corresponding modified *precondition* becomes

$$(\mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{s}^\top \mathbf{A}, \mathbf{B}, \mathbf{s}^\top \mathbf{B}, \mathbf{P}, \mathbf{s}^\top \mathbf{P} + \mathbf{u}^\top \mathbf{G}, \mathbf{s}^\top \mathbf{Q}) \stackrel{c}{\approx} (\mathbf{r}_{\text{pub}}, \mathbf{A}, \$1, \mathbf{B}, \$2, \mathbf{P}, \$2', \$3).$$

and the *postcondition* is given as

$$(\mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{s}^\top \mathbf{A}, \mathbf{B}, \mathbf{s}^\top \mathbf{B}, \mathbf{P}, \mathbf{s}^\top \mathbf{P} + \mathbf{u}^\top \mathbf{G}, \mathbf{K}) \stackrel{c}{\approx} (\mathbf{r}_{\text{pub}}, \mathbf{A}, \$1, \mathbf{B}, \$2, \mathbf{P}, \$2', \mathbf{K}),$$

where $\mathbf{K}$ is also sampled as $\mathbf{K} \leftarrow \mathbf{A}^{-1}(\mathbf{Q})$. The intuition behind this modified assumption remains similar to the original one.

- 2) **Generalized to the multi-instance setting.** In analogy to [WWW22], which modified the evasive LWE assumption of [Wee22], we present a tailored variant of the evasive lPFE assumption [HLL24] for our multi-authority setting. We can also extend the variant from Step 1) into our final evlPFE assumption. Specifically, for each index $i \in [\ell]$, let $\mathbf{A}_i, \mathbf{B}_i, \mathbf{P}_i$ be uniformly and independently random matrices. Let $\{\mathbf{v}_{i,j}\}_{i \in [\ell], j \in [k_i]}$, $\{\mathbf{r}_{i,j}\}_{i \in [\ell], j \in [k_i]}$ be vectors generated by a public-coin sampler with public randomness $\mathbf{r}_{\text{pub}}$, and let $\mathbf{u}$ be generated by a private-coin sampler. Define the matrices $\mathbf{Q}_i$ for each $i \in [\ell]$ as:

$$\mathbf{Q}_i \leftarrow [\mathbf{B}_i \mid \mathbf{P}_i] \left[\begin{array}{c|c} \mathbf{r}_{i,1} & \cdots \\ \mathbf{G}^{-1}(\mathbf{v}_{i,1}) & \mathbf{G}^{-1}(\mathbf{v}_{i,k_i}) \end{array} \right].$$

The evlPFE assumption states that if the following *precondition* holds:

$$\begin{aligned} & (\mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{s}_i^\top \mathbf{A}_i\}, \{\mathbf{B}_i\}, \sum \mathbf{s}_i^\top \mathbf{B}_i, \{\mathbf{P}_i\}, \sum \mathbf{s}_i^\top \mathbf{P}_i + \mathbf{u}^\top \mathbf{G}, \{\mathbf{s}_i^\top \mathbf{Q}_i\}) \\ & \stackrel{c}{\approx} (\mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \$1,i\}, \{\mathbf{B}_i\}, \$2, \{\mathbf{P}_i\}, \$2', \{\$3,i\}), \end{aligned} \quad (2)$$

then the *postcondition*

$$\begin{aligned} & (\mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{s}_i^\top \mathbf{A}_i\}, \{\mathbf{B}_i\}, \sum \mathbf{s}_i^\top \mathbf{B}_i, \{\mathbf{P}_i\}, \sum \mathbf{s}_i^\top \mathbf{P}_i + \mathbf{u}^\top \mathbf{G}, \{\mathbf{K}_i\}) \\ & \stackrel{c}{\approx} (\mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \$1,i\}, \{\mathbf{B}_i\}, \$2, \{\mathbf{P}_i\}, \$2', \{\mathbf{K}_i\}) \end{aligned} \quad (3)$$

also holds, where $\mathbf{K}_i \leftarrow \mathbf{A}_i^{-1}(\mathbf{Q}_i)$. Informally, the evlPFE assumption is no stronger than the evasive LWE assumption of [WWW22], as it admits a reduction from the latter. Details of the reduction are provided in Appendix A.

Based on this evlPFE assumption, we can establish the (weakened) static security as an MA-ABevlPFE scheme. The matrices $\mathbf{A}_i, \mathbf{B}_i, \mathbf{P}_i$ in the assumption correspond to the public keys for the authorities. Each column of $\mathbf{Q}_i$ is of the form $\mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}_{i,j}) + \mathbf{B}_i \mathbf{r}_{i,j}$. Here, $\mathbf{v}_{i,j}$ corresponds to the key vector in a secret-key query in the game, and $\mathbf{r}_{i,j}$ is interpreted as the hash output of some tuple $(\text{gid}, \mathbf{v})$ in the

scheme. By carefully designing the matrix, the columns of $\mathbf{K}_i$ in the postcondition instance serve as the secret-key responses to the secret-key queries. Noting that each secret-key query satisfies either condition (a) or condition (b), we can show the constructed matrices satisfy the precondition (2). By the evIPFE assumption, we obtain that postcondition (3) holds, implying that the resulting ciphertext is pseudorandom. A complete formal proof is provided in Section VI.

Desired Static Security: MA-ABNIPFE Scheme. In the static security model, the adversary is required to commit to its full set of secret-key queries and challenge plaintexts in advance. A statically secure MA-ABNIPFE scheme requires that no efficient adversary can distinguish the ciphertexts encrypting the challenge plaintexts with non-negligible advantage, even in the presence of corrupted authorities and access to the secret keys corresponding to its queries.

Our Assumption: Indistinguishability-Based Evasive IPFE. We base the security of our construction on the *Indistinguishability-based Evasive IPFE* (IND-evIPFE) assumption, which is a variant of the evIPFE assumption discussed earlier.

The IND-evIPFE assumption follows a structure similar to the evIPFE assumption introduced in this work. Specifically, for each index $i \in [\ell]$, let $\mathbf{A}_i, \mathbf{B}_i, \mathbf{P}_i$ be independent and uniformly random matrices. Let $\{\mathbf{v}_{i,j}\}_{i \in [\ell], j \in [k_i]}$, $\{\mathbf{r}_{i,j}\}_{i \in [\ell], j \in [k_i]}$ and $\mathbf{u}_0, \mathbf{u}_1$ be vectors generated by a public-coin sampler with public randomness $\mathbf{r}_{\text{pub}}$. Define the matrices $\mathbf{Q}_i$ for each $i \in [\ell]$ as:

$$\mathbf{Q}_i \leftarrow [\mathbf{B}_i \mid \mathbf{P}_i] \left[\begin{array}{c|c} \mathbf{r}_{i,1} & \cdots & \mathbf{r}_{i,k_i} \\ \mathbf{G}^{-1}(\mathbf{v}_{i,1}) & \cdots & \mathbf{G}^{-1}(\mathbf{v}_{i,k_i}) \end{array} \right].$$

The IND-evIPFE assumption states that if the following *precondition* holds:

$$\begin{aligned} & (\mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \underbrace{\mathbf{s}_i^\top \mathbf{A}_i}_{\text{secret-key query}}\}, \{\mathbf{B}_i\}, \underbrace{\sum \mathbf{s}_i^\top \mathbf{B}_i}_{\text{secret-key query}}, \{\mathbf{P}_i\}, \underbrace{\sum \mathbf{s}_i^\top \mathbf{P}_i + \mathbf{u}_0^\top \mathbf{G}}_{\text{secret-key query}}, \{\underbrace{\mathbf{s}_i^\top \mathbf{Q}_i}_{\text{secret-key query}}\}) \\ & \stackrel{c}{\approx} (\mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \underbrace{\mathbf{s}_i^\top \mathbf{A}_i}_{\text{secret-key query}}\}, \{\mathbf{B}_i\}, \underbrace{\sum \mathbf{s}_i^\top \mathbf{B}_i}_{\text{secret-key query}}, \{\mathbf{P}_i\}, \underbrace{\sum \mathbf{s}_i^\top \mathbf{P}_i + \mathbf{u}_1^\top \mathbf{G}}_{\text{secret-key query}}, \{\underbrace{\mathbf{s}_i^\top \mathbf{Q}_i}_{\text{secret-key query}}\}), \end{aligned} \quad (4)$$

then the following *postcondition*

$$\begin{aligned} & (\mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \underbrace{\mathbf{s}_i^\top \mathbf{A}_i}_{\text{secret-key query}}\}, \{\mathbf{B}_i\}, \underbrace{\sum \mathbf{s}_i^\top \mathbf{B}_i}_{\text{secret-key query}}, \{\mathbf{P}_i\}, \underbrace{\sum \mathbf{s}_i^\top \mathbf{P}_i + \mathbf{u}_0^\top \mathbf{G}}_{\text{secret-key query}}, \{\mathbf{K}_i\}) \\ & \stackrel{c}{\approx} (\mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \underbrace{\mathbf{s}_i^\top \mathbf{A}_i}_{\text{secret-key query}}\}, \{\mathbf{B}_i\}, \underbrace{\sum \mathbf{s}_i^\top \mathbf{B}_i}_{\text{secret-key query}}, \{\mathbf{P}_i\}, \underbrace{\sum \mathbf{s}_i^\top \mathbf{P}_i + \mathbf{u}_1^\top \mathbf{G}}_{\text{secret-key query}}, \{\mathbf{K}_i\}), \end{aligned} \quad (5)$$

where $\mathbf{K}_i \leftarrow \mathbf{A}_i^{-1}(\mathbf{Q}_i)$.

However, we need to make some additional assumptions about the sampler for vectors $\mathbf{r}_{i,j}$ and $\mathbf{v}_{i,j}$. In this assumption, each pair $(\mathbf{r}_{i,j}, \mathbf{v}_{i,j})$ is required to be nonzero to avoid the obvious attack against the assumption. For the justification of this setting, we refer to Section 4 for details.

Following a strategy similar to the MA-ABevIPFE case, we can prove that our construction satisfies the desired MA-ABNIPFE static security notion. In this setting, all secret-key queries are fully determined by the adversary and the challenger plaintexts $\mathbf{u}_0, \mathbf{u}_1$ are chosen by the adversary. Using the same method as for the MA-ABevIPFE scheme, we construct each matrix $\mathbf{Q}_i$ based on the submitted secret-key queries. By carefully designing $\mathbf{Q}_i$, the resulting columns of each $\mathbf{K}_i$ serve as valid responses to the corresponding secret-key queries. The restriction on secret-key queries to satisfy either condition (a) or (b) ensures that the parameters $\mathbf{A}_i, \mathbf{B}_i, \mathbf{P}_i, \mathbf{Q}_i$ meet the precondition (4) of the assumption. Then by the postcondition (5) of the IND-evIPFE assumption, we obtain that the ciphertexts of $\mathbf{u}_0$ and $\mathbf{u}_1$ are indistinguishable, thereby establishing the desired security.

Removing the Noise: Noiseless MA-ABIPFE. Finally, we construct a noiseless MA-ABIPFE scheme enabling computation of *exact* inner products rather than approximate values, by slightly modifying the

scheme proposed in Section II-A. Specifically, we eliminate the additive error term in the approximate inner products during decryption. Unlike the scheme in Section II-A that operates over plaintext vectors in $\mathbb{Z}_q^n$, this modified construction operates over input vectors in the smaller domain $\mathbb{Z}_p^n$ with $p \ll q$.

The global setup, authority setup, and key generation algorithms remain unchanged. To support computation over $\mathbb{Z}_p^n$ within a $\mathbb{Z}_q$ -based lattice setting, we adopt the *modulus-switching function* $\lceil \cdot \rceil_{p \rightarrow q}$ that maps elements from $\mathbb{Z}_p$ to $\mathbb{Z}_q$. For a vector $\mathbf{u} \in \mathbb{Z}_p^n$, we define the encoding function:

$$\lceil \mathbf{u} \rceil_{p \rightarrow q} : \mathbf{u} \mapsto \left\lceil \frac{q}{p} \cdot \mathbf{u} \right\rceil \in \mathbb{Z}_q^n.$$

Similarly, we can define the decoding function $\lfloor \cdot \rfloor_{q \rightarrow p}$ as the inverse mapping of $\lceil \cdot \rceil_{p \rightarrow q}$. With $\lceil \cdot \rceil_{p \rightarrow q}$, we can embed $\mathbf{u} \in \mathbb{Z}_p^n$ into $\lceil \mathbf{u} \rceil_{p \rightarrow q} \in \mathbb{Z}_q^n$. The encryption algorithm proceeds as

$$\text{ct} \leftarrow \left(\underbrace{\{\mathbf{s}_i^\top \mathbf{A}_i\}}_{i \in X}, \underbrace{\sum_{i \in X} \mathbf{s}_i^\top \mathbf{B}_i}, \underbrace{\sum_{i \in X} \mathbf{s}_i^\top \mathbf{P}_i + \lceil \mathbf{u}^\top \rceil_{p \rightarrow q} \mathbf{G}} \right),$$

where $\mathbf{s}_i \xleftarrow{\$} \mathbb{Z}_q^n$ for each $i \in X$. On the decryption side, we can use the same decryption process over $\mathbb{Z}_q$ as in Section II-A, followed by a final application of $\lfloor \cdot \rfloor_{q \rightarrow p}$ to recover the inner-product result over $\mathbb{Z}_p$.

This modification to the encryption and decryption procedures removes the approximation gap in the decryption procedure while still enabling fine-grained multi-authority access control. The resulting scheme, formally given in Construction 2, is particularly well-suited for applications requiring deterministic, noise-free computation of inner products, and its security is established under the standard LWE assumption and the IND-evIPFE assumption tailored for noiseless settings. A detailed description of this noiseless scheme and its security analysis can be found in Section VIII.

III. PRELIMINARIES

A. Notations

Let $\lambda \in \mathbb{N}$ denote the security parameter used throughout this paper. For a positive integer $n \in \mathbb{N}$, denote $[n]$ as the set $\{k \in \mathbb{N} : 1 \leq k \leq n\}$. For a positive integer $q \in \mathbb{N}$, let $\mathbb{Z}_q$ denote the ring of integers modulo q . Throughout this paper, vectors are assumed to be column vectors by default. We use bold lower-case letters (e.g., $\mathbf{u}, \mathbf{v}$) for vectors and bold upper-case letters (e.g., $\mathbf{A}, \mathbf{B}$) for matrices. Let $\mathbf{v}[i]$ denote the i -th entry of the vector $\mathbf{v}$, and let $\mathbf{U}[i, j]$ denote the (i, j) -entry of the matrix $\mathbf{U}$. We write $\mathbf{0}_n$ for the all-zero vector of length n , and $\mathbf{0}_{n \times m}$ for the all-zero matrix of dimension $n \times m$. The *infinity norm* of a vector $\mathbf{v}$ and the corresponding operator norm of a matrix $\mathbf{U}$ are defined as:

$$\|\mathbf{v}\| = \max_i |\mathbf{v}[i]|, \quad \|\mathbf{U}\| = \max_{i,j} |\mathbf{U}[i, j]|.$$

For two matrices $\mathbf{A}, \mathbf{B}$ of dimensions $n_1 \times m_1$ and $n_2 \times m_2$, respectively, their *Kronecker product* is an $n_1 n_2 \times m_1 m_2$ matrix given by

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} \mathbf{A}[1, 1]\mathbf{B} & \cdots & \mathbf{A}[1, m_1]\mathbf{B} \\ \vdots & \ddots & \vdots \\ \mathbf{A}[n_1, 1]\mathbf{B} & \cdots & \mathbf{A}[n_1, m_1]\mathbf{B} \end{bmatrix}.$$

A function $f(\lambda)$ is called *negligible* if $f(\lambda) = O(\lambda^{-c})$ for every constant $c > 0$. We denote a negligible function of λ by $\text{negl}(\lambda)$. A function $f(\lambda)$ is called *polynomial* if $f(\lambda) = O(\lambda^c)$ for some constant $c > 0$. We denote a polynomial function of λ by $\text{poly}(\lambda)$. A function $f(\lambda)$ is called *super-polynomial*, if $f(\lambda) = \omega(\lambda^c)$ for every constant $c > 0$. We denote a super-polynomial function of λ by $\text{superpoly}(\lambda)$. We say an event occurs with *overwhelming probability* if its probability is $1 - \text{negl}(\lambda)$. An algorithm is

called *efficient* if it runs in probabilistic polynomial time in its input length, typically parameterized by the security parameter λ in this work.

We denote the sampling of an element x from a distribution D by $x \leftarrow D$, and the uniform sampling of an element x from a set S by $x \xleftarrow{\$} S$. For a distribution D and a positive integer m , we write $\mathbf{x} \leftarrow D^m$ to denote an m -dimensional vector whose entries are independently sampled from D . More generally, for integers $m, n \in \mathbb{N}$, we write $\mathbf{X} \leftarrow D^{m \times n}$ to denote an $m \times n$ matrix with independently sampled entries from D . For two distributions D_1 and D_2 , We use the notation $D_1 \equiv D_2$ to denote that the two distributions are identical.

We also define the *modulus-switching function*. For an element $x \in \mathbb{Z}_q$ and some positive integer p , we define a function mapping from $\mathbb{Z}_q$ to $\mathbb{Z}_p$ as

$$\lceil \cdot \rceil_{q \rightarrow p} : x \mapsto \left\lceil \frac{p}{q} \cdot x \right\rceil \in \mathbb{Z}_p.$$

This map is applied coefficient-wise when extended to vectors or matrices. Such functions are commonly used in cryptographic schemes for modulus compression and message encoding.

Indistinguishability. For two distributions D_1 and D_2 over a discrete domain Ω , the *statistical distance* between them is defined as

$$\text{SD}(D_1, D_2) = (1/2) \cdot \sum_{\omega \in \Omega} |D_1(\omega) - D_2(\omega)|.$$

Let $D_1 = \{D_{1,\lambda}\}_{\lambda \in \mathbb{N}}$ and $D_2 = \{D_{2,\lambda}\}_{\lambda \in \mathbb{N}}$ be two ensembles of distributions parameterized by the security parameter λ . We say D_1 and D_2 are *statistically indistinguishable* if there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\text{SD}(D_{1,\lambda}, D_{2,\lambda}) \leq \text{negl}(\lambda).$$

We say D_1 and D_2 are *computationally indistinguishable* if for all $\lambda \in \mathbb{N}$, for all *efficient* algorithms $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\Pr[\mathcal{A}(1^\lambda, x_\lambda) = 1 : x_\lambda \leftarrow D_{1,\lambda}] - \Pr[\mathcal{A}(1^\lambda, y_\lambda) = 1 : y_\lambda \leftarrow D_{2,\lambda}] \leq \text{negl}(\lambda).$$

We use the notations $\overset{s}{\approx}$ and $\overset{c}{\approx}$ to denote statistical and computational indistinguishability, respectively.

B. Lattice Preliminaries

We recall some basic concepts related to lattices.

Discrete Gaussians. Let $D_{\mathbb{Z},\sigma}$ represent the centered *discrete Gaussian distribution* over $\mathbb{Z}$ with standard deviation $\sigma \in \mathbb{R}^+$ (e.g., [Ban93]). For a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a vector $\mathbf{v} \in \mathbb{Z}_q^n$, let $\mathbf{A}_\sigma^{-1}(\mathbf{v})$ denote the distribution of a random variable $\mathbf{u} \leftarrow D_{\mathbb{Z},\sigma}^m$ conditioned on $\mathbf{A}\mathbf{u} = \mathbf{v} \pmod q$. When $\mathbf{A}_\sigma^{-1}$ is applied to a matrix, it is understood as being applied independently to each column of the matrix.

The following lemma (e.g., [MR07]) shows that for a given Gaussian width parameter $\sigma = \sigma(\lambda)$, the probability of a vector drawn from the discrete Gaussian distribution having norm greater than $\sqrt{\lambda}\sigma$ is negligible.

Lemma 4. *Let λ be a security parameter and $\sigma = \sigma(\lambda)$ be a Gaussian width parameter. Then for all polynomials $n = n(\lambda)$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$,*

$$\Pr \left[\|\mathbf{v}\| > \sqrt{\lambda}\sigma : \mathbf{v} \leftarrow D_{\mathbb{Z},\sigma}^n \right] = \text{negl}(\lambda).$$

Smudging Lemma. In the following, we present the standard smudging lemma, which formalizes the intuition that sufficiently large standard deviation can “smudge out” small perturbations, making the resulting distributions statistically indistinguishable.

Lemma 5 ([BDE⁺18]). *Let λ be a security parameter, and let $e \in \mathbb{Z}$ satisfy $|e| \leq B$. Suppose that $\sigma \geq B \cdot \lambda^{\omega(1)}$. Then, the following two distributions are statistically indistinguishable:*

$$\{z : z \leftarrow D_{\mathbb{Z}, \sigma}\} \text{ and } \{z + e : z \leftarrow D_{\mathbb{Z}, \sigma}\}.$$

Lattice Trapdoors. Lattices with trapdoors are structured lattices that are computationally indistinguishable from randomly chosen lattices (without auxiliary information). However, they possess specific “trapdoors”—short bases or auxiliary structures—that enable efficient solutions to otherwise hard lattice problems such as preimage sampling for the *Short Integer Solution* (SIS) function. In this work, we adopt the trapdoor frameworks outlined in [GPV08, MP12], following the formalization in [BTWV17].

Lemma 6 ([GPV08, MP12]). *Let n, m, q be lattice parameters. Then there exist two efficient algorithms (TrapGen, SamplePre) with the following syntax:*

- $\text{TrapGen}(1^n, 1^m, q) \rightarrow (\mathbf{A}, \text{td}_{\mathbf{A}})$: *On input the lattice dimension n , number of samples m , modulus q , this randomized algorithm outputs a matrix $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$ together with a trapdoor $\text{td}_{\mathbf{A}}$.*
- $\text{SamplePre}(\mathbf{A}, \text{td}_{\mathbf{A}}, \mathbf{y}, \sigma) \rightarrow \mathbf{x}$: *On input $(\mathbf{A}, \text{td}_{\mathbf{A}})$ from TrapGen, a target vector $\mathbf{y} \in \mathbb{Z}_q^n$ and a Gaussian width parameter σ , this randomized algorithm outputs a vector $\mathbf{x} \in \mathbb{Z}^m$.*

Moreover, there exists a polynomial $m_0 = m_0(n, q) = O(\sqrt{n \log q})$ such that for all $m \geq m_0$, the above algorithms satisfy the following properties:

- **Trapdoor distribution:** *The matrix $\mathbf{A}$ output by $\text{TrapGen}(1^n, 1^m, q)$ is statistically close to a uniformly random matrix. Specifically, if $(\mathbf{A}, \text{td}_{\mathbf{A}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$ and $\mathbf{A}' \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, then the statistical distance between the distributions of $\mathbf{A}$ and $\mathbf{A}'$ is at most 2^{-n} .*
- **Preimage sampling:** *Suppose $\tau = O(\sqrt{n \log q})$. Then for all $\sigma \geq \tau \cdot \omega(\sqrt{\log n})$ and all target vectors $\mathbf{y} \in \mathbb{Z}_q^n$, the statistical distance between the following two distributions is at most 2^{-n} :*

$$\{\mathbf{x} \leftarrow \text{SamplePre}(\mathbf{A}, \text{td}_{\mathbf{A}}, \mathbf{y}, \sigma)\} \text{ and } \{\mathbf{x} \leftarrow \mathbf{A}_{\sigma}^{-1}(\mathbf{y})\}.$$

The Gadget Matrix. Here we recall the definition of the *gadget matrix* introduced in [MP12]. For positive integers $n, q \in \mathbb{N}$, let $\mathbf{g}^{\top} = (1, 2, \dots, 2^{\lceil \log q \rceil - 1})$ denote the *gadget vector*. Define the *gadget matrix* $\mathbf{G}_n = \mathbf{I}_n \otimes \mathbf{g}^{\top} \in \mathbb{Z}_q^{n \times m}$, where $m = n \lceil \log q \rceil$. The inverse function $\mathbf{G}_n^{-1}$ maps vectors in $\mathbb{Z}_q^n$ to their binary expansions in $\{0, 1\}^m$. More precisely, for a vector $\mathbf{x} \in \mathbb{Z}_q^n$, the output $\mathbf{y} = \mathbf{G}_n^{-1}(\mathbf{x})$ is given by

$$\mathbf{y} = (y_{1,0}, \dots, y_{1,\lceil \log q \rceil - 1}, \dots, y_{n,0}, \dots, y_{n,\lceil \log q \rceil - 1}),$$

where the sequence $(y_{i,0}, \dots, y_{i,\lceil \log q \rceil})$ corresponds to the $\lceil \log q \rceil$ bits of the binary representation of $\mathbf{x}[i]$. The function $\mathbf{G}_n^{-1}(\cdot)$ extends naturally to matrices by applying it column-wise. It is straightforward to verify that $\mathbf{G}_n \cdot \mathbf{G}_n^{-1}(\mathbf{A}) = \mathbf{A} \pmod{q}$ for any matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times t}$. For simplicity, we will omit the subscript when the parameter n is clear from context.

Preimage Sampling. The following lemma describes a useful property of discrete Gaussian distributions. Specifically, it basically states that, given a uniformly random matrix $\mathbf{A}$ with sufficiently many columns, two methods of generating an input-output pair $(\mathbf{x}, \mathbf{y} = \mathbf{A}\mathbf{x})$ produce statistically indistinguishable distributions.

Lemma 7 (Preimage Sampling [GPV08]). *Let $n, m, q \in \mathbb{N}$, where $q > 2$ is a prime. Then there exists polynomials $m_0(n, q) = O(\sqrt{n \log q})$ and $\chi_0(n, q) = \sqrt{n \log q} \cdot \omega(\sqrt{\log n})$ such that for all $m \geq m_0(n, q)$ and $\chi \geq \chi_0(n, q)$, the following two distributions are statistically indistinguishable:*

$$\{(\mathbf{A}, \mathbf{x}, \mathbf{A}\mathbf{x}) : \mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{x} \leftarrow D_{\mathbb{Z}, \chi}^m\} \text{ and } \{(\mathbf{A}, \mathbf{x}, \mathbf{y}) : \mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{y} \xleftarrow{\$} \mathbb{Z}_q^n, \mathbf{x} \leftarrow \mathbf{A}_{\chi}^{-1}(\mathbf{y})\}.$$

C. LWE Assumption and Its Variants

Learning with Errors Assumption. We review the Learning With Errors (LWE) assumption and its variants used throughout this work.

Assumption 1 (Learning with Errors (LWE) [Reg09]). *Let λ be a security parameter and let $n = n(\lambda), m = m(\lambda), q = q(\lambda), \sigma = \sigma(\lambda)$. The decisional learning with errors assumption, denoted $\text{LWE}_{n,m,q,\sigma}$, states that for $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n, \mathbf{e} \leftarrow D_{\mathbb{Z},\sigma}^m$, and $\delta \xleftarrow{\$} \mathbb{Z}_q^m$, the following two distributions are computationally indistinguishable:*

$$(\mathbf{A}, \mathbf{s}^\top \mathbf{A} + \mathbf{e}^\top) \text{ and } (\mathbf{A}, \delta).$$

Assumption 2 (Flipped LWE [BLMR13]). *Let λ be a security parameter and let $k = k(\lambda), m = m(\lambda), q = q(\lambda), \chi = \chi(\lambda), \sigma = \sigma(\lambda)$. The flipped LWE assumption, denoted $\text{FlipLWE}_{k,m,q,\chi,\sigma}$, states that for $\mathbf{A} \xleftarrow{\$} D_{\mathbb{Z},\chi}^{k \times m}, \mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k, \mathbf{e} \leftarrow D_{\mathbb{Z},\sigma}^m$, and $\delta \xleftarrow{\$} \mathbb{Z}_q^m$, the following two distributions are computationally indistinguishable:*

$$(\mathbf{A}, \mathbf{s}^\top \mathbf{A} + \mathbf{e}^\top) \text{ and } (\mathbf{A}, \delta^\top).$$

The following theorem establishes the hardness of the flipped LWE assumption based on the standard LWE assumption.

Theorem 8 ([BLMR13]). *Let q be an integer, and suppose $k \geq 6n \log q$ and $\sigma = \Omega(\sqrt{n \log q})$. Then, if the standard $\text{LWE}_{n,m,q,\chi}$ assumption holds, we have that the assumption $\text{FlipLWE}_{k,m,q,\sigma,\chi}$ holds.*

IV. EVASIVE IPFE ASSUMPTIONS

A. The Evasive IPFE Assumption

We first present a variant of evasive IPFE assumption introduced by [HLL24], with slight modifications tailored to our setting.

Assumption 3 (Evasive IPFE (evIPFE) Assumption, adapted, [WWW22, HLL24]). *Let $\lambda \in \mathbb{N}$ be a security parameter, and let $n, q, m = n \lceil \log q \rceil, \chi, \chi'$ be lattice parameters specified by λ . Denote $\mathbf{gp} = (1^\lambda, q, 1^n, 1^m, 1^{m'}, 1^\chi, 1^{\chi'})$. Let $\mathcal{S} = (\mathcal{S}_v, \mathcal{S}_u)$ be a pair of sampling algorithms with the following syntax:*

- $\mathcal{S}_v(\mathbf{gp}; \mathbf{r}_{\text{pub}})$: *Given the global parameter $\mathbf{gp}$ and randomness $\mathbf{r}_{\text{pub}} \in \{0, 1\}^*$, the algorithm outputs parameters*

$$1^\ell, \{1^{k_i}\}_{i \in [\ell]},$$

as well as $k_1 + k_2 + \dots + k_\ell$ vector tuples

$$(\mathbf{r}_{1,1}, \mathbf{v}_{1,1}), (\mathbf{r}_{1,2}, \mathbf{v}_{1,2}), \dots, (\mathbf{r}_{1,k_1}, \mathbf{v}_{1,k_1}); \dots; (\mathbf{r}_{\ell,1}, \mathbf{v}_{\ell,1}), \dots, (\mathbf{r}_{\ell,k_\ell}, \mathbf{v}_{\ell,k_\ell}),$$

where $\mathbf{r}_{i,j} \in \mathbb{Z}_q^{m'}, \mathbf{v}_{i,j} \in \mathbb{Z}_q^n$ for all $i \in [\ell], j \in [k_i]$.

- $\mathcal{S}_u(\mathbf{gp}, \mathbf{r}_{\text{pub}}; \mathbf{r}_{\text{pri}})$: *Given the global parameter $\mathbf{gp}$, public randomness $\mathbf{r}_{\text{pub}}$ used in $\mathcal{S}_v$, and private randomness $\mathbf{r}_{\text{pri}} \in \{0, 1\}^*$. It outputs a vector $\mathbf{u} \in \mathbb{Z}_q^n$.*

For two adversaries $\mathcal{A}_0$ and $\mathcal{A}_1$, we define their advantage functions as follows:

$$\begin{aligned} \text{Adv}_{\mathcal{S}, \mathcal{A}_0}^{\text{Pre}}(\lambda) &:= \left| \Pr \left[\mathcal{A}_0(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{z}_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top, \{\mathbf{z}_{3,i}^\top\}_{i \in [\ell]}) = 1 \right] \right. \\ &\quad \left. - \Pr \left[\mathcal{A}_0(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \delta_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \delta_2^\top, \{\delta_{3,i}^\top\}_{i \in [\ell]}) = 1 \right] \right|; \\ \text{Adv}_{\mathcal{S}, \mathcal{A}_1}^{\text{Post}}(\lambda) &:= \left| \Pr \left[\mathcal{A}_1(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{z}_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top, \{\mathbf{K}_i\}_{i \in [\ell]}) = 1 \right] \right. \\ &\quad \left. - \Pr \left[\mathcal{A}_1(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \delta_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \delta_2^\top, \{\mathbf{K}_i\}_{i \in [\ell]}) = 1 \right] \right|; \end{aligned}$$

where the parameters are sampled as follows:

- $\begin{pmatrix} 1^\ell, \{1^{k_i}\}_{i \in [\ell]}; \\ (\mathbf{r}_{1,1}, \mathbf{v}_{1,1}), \dots, (\mathbf{r}_{\ell,k_\ell}, \mathbf{v}_{\ell,k_\ell}); \\ \dots \\ (\mathbf{r}_{\ell,1}, \mathbf{v}_{\ell,1}), \dots, (\mathbf{r}_{\ell,k_\ell}, \mathbf{v}_{\ell,k_\ell}) \end{pmatrix} \leftarrow \mathcal{S}_v(\mathbf{gp}; \mathbf{r}_{\text{pub}}), \quad \mathbf{u} \leftarrow \mathcal{S}_u(\mathbf{gp}, \mathbf{r}_{\text{pub}}; \mathbf{r}_{\text{pri}}),$
- $\mathbf{B}_1, \dots, \mathbf{B}_\ell \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}, \mathbf{P}_1, \dots, \mathbf{P}_\ell \xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{B}^\top \leftarrow [\mathbf{B}_1^\top \mid \dots \mid \mathbf{B}_\ell^\top], \mathbf{P}^\top \leftarrow [\mathbf{P}_1^\top \mid \dots \mid \mathbf{P}_\ell^\top],$
- $\mathbf{Q}_i \leftarrow [\mathbf{B}_i \mid \mathbf{P}_i] \begin{bmatrix} \mathbf{r}_{i,1} \\ \mathbf{G}^{-1}(\mathbf{v}_{i,1}) \end{bmatrix} \mid \dots \mid \begin{bmatrix} \mathbf{r}_{i,k_i} \\ \mathbf{G}^{-1}(\mathbf{v}_{i,k_i}) \end{bmatrix} \in \mathbb{Z}_q^{n \times k_i},$
- $(\mathbf{A}_1, \text{td}_1), \dots, (\mathbf{A}_\ell, \text{td}_\ell) \leftarrow \text{TrapGen}(1^n, 1^m, q),$
- $\mathbf{s}_1, \dots, \mathbf{s}_\ell \xleftarrow{\$} \mathbb{Z}_q^n, \mathbf{s}^\top \leftarrow [\mathbf{s}_1^\top \mid \dots \mid \mathbf{s}_\ell^\top] \in \mathbb{Z}_q^{n\ell},$
- $\mathbf{e}_{1,i} \leftarrow D_{\mathbb{Z}, \chi}^m, \mathbf{e}_{3,i} \leftarrow D_{\mathbb{Z}, \chi}^{k_i} \text{ for each } i \in [\ell], \mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^{m'+m},$
- $\delta_{1,i} \xleftarrow{\$} \mathbb{Z}_q^m, \delta_{3,i} \xleftarrow{\$} \mathbb{Z}_q^{k_i} \text{ for each } i \in [\ell], \delta_2 \xleftarrow{\$} \mathbb{Z}_q^{m'+m},$
- $\mathbf{z}_{1,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top, \mathbf{z}_{3,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{Q}_i + \mathbf{e}_{3,i}^\top \text{ for each } i \in [\ell], \quad \mathbf{z}_2^\top \leftarrow \mathbf{s}^\top [\mathbf{B} \mid \mathbf{P}] + \mathbf{e}_2^\top + [\mathbf{0} \mid \mathbf{u}^\top \mathbf{G}],$
- $\mathbf{K}_i \leftarrow \text{SamplePre}(\mathbf{A}_i, \text{td}_i, \mathbf{Q}_i, \chi') \text{ for each } i \in [\ell].$

We say that the $\text{evIPFE}_{n,m,m',q,\chi,\chi'}$ assumption holds, if for all efficient samplers $\mathcal{S}$, the following implication holds: If there exists an efficient adversary $\mathcal{A}_1$ with a non-negligible advantage function $\text{Adv}_{\mathcal{S}, \mathcal{A}_1}^{\text{Post}}(\lambda)$, then there exists another efficient adversary $\mathcal{A}_0$ with a non-negligible advantage function $\text{Adv}_{\mathcal{S}, \mathcal{A}_0}^{\text{Pre}}(\lambda)$.

Remark 9 (Relation to the Evasive IPFE Assumption in [HLL24]). Assumption 3 defined above originates from the evasive IPFE assumption introduced by [HLL24]. Following the approach of [WWW22], which defines a variant of public-coin evasive LWE based on the formulation in [Wee22] by introducing multiple (i.e., $\ell \geq 1$) independently and uniformly sampled matrices $\mathbf{A}_i \in \mathbb{Z}_q^{n \times m}$, we apply an analogous modification to the original evasive IPFE assumption proposed in [HLL24].

In the following, we focus on the connection between Assumption 3 and the evasive IPFE assumption from [HLL24]. Informally, the latter basically states if

$$(1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{z}_1^\top, \mathbf{P}, \mathbf{z}_2^\top + \mathbf{u}^\top \mathbf{G}, \mathbf{z}_3^\top) \stackrel{c}{\approx} (1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \delta_1^\top, \mathbf{P}, \delta_2^\top, \delta_3^\top), \quad (6)$$

where $\mathbf{u}, \mathbf{v}_1, \dots, \mathbf{v}_k \in \mathbb{Z}_q^d$ are generated by some sampler $\mathcal{S}' = (\mathcal{S}'_v, \mathcal{S}'_u)$, and the other parameters are sampled as follows:

$$\begin{aligned} \mathbf{A} &\xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{P} \xleftarrow{\$} \mathbb{Z}_q^{n \times D}, \mathbf{Q} \leftarrow \mathbf{P}[\mathbf{G}^{-1}(\mathbf{v}_1) \mid \dots \mid \mathbf{G}^{-1}(\mathbf{v}_k)], \\ \mathbf{s} &\xleftarrow{\$} \mathbb{Z}_q^n, \mathbf{e}_1 \leftarrow D_{\mathbb{Z}, \chi}^m, \mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^D, \mathbf{e}_3 \leftarrow D_{\mathbb{Z}, \chi}^k, \\ \mathbf{z}_1^\top &\leftarrow \mathbf{s}^\top \mathbf{A} + \mathbf{e}_1^\top, \mathbf{z}_2^\top \leftarrow \mathbf{s}^\top \mathbf{P} + \mathbf{e}_2^\top, \mathbf{z}_3^\top \leftarrow \mathbf{s}^\top \mathbf{Q} + \mathbf{e}_3^\top, \\ \delta_1 &\xleftarrow{\$} \mathbb{Z}_q^m, \delta_2 \xleftarrow{\$} \mathbb{Z}_q^D, \delta_3 \xleftarrow{\$} \mathbb{Z}_q^k, \end{aligned}$$

then the following postcondition holds

$$(1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{z}_1^\top, \mathbf{P}, \mathbf{z}_2^\top + \mathbf{u}^\top \mathbf{G}, \mathbf{K}) \stackrel{c}{\approx} (1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \delta_1^\top, \mathbf{P}, \delta_2^\top, \mathbf{K}),$$

where $\mathbf{K} \leftarrow \mathbf{A}_{\chi'}^{-1}(\mathbf{Q})$.

We now compare Assumption 3 with the version in [HLL24]. On the one hand, when $\ell = 1$, Assumption 3 generalizes original setting in [HLL24] by incorporating an auxiliary matrix $\mathbf{B} \in \mathbb{Z}_q^{n \times m'}$ and additional vectors $\mathbf{r}_{1,j}$. Specifically, if we set $m' = 0, d = n$ and $D = m$, then Assumption 3 collapses to the formulation used in [HLL24].

On the other hand, when $\ell = 1$, Assumption 3 can also be roughly viewed as a special case of [HLL24], instantiated with a restricted sampler $\mathcal{S}'$. Concretely, suppose $D = m' + m$ and we write the matrix $\mathbf{P}$ as $[\mathbf{B} \mid \mathbf{P}']$ with $\mathbf{B} \in \mathbb{Z}_q^{n \times m'}, \mathbf{P}' \in \mathbb{Z}_q^{n \times m}$. If the sampler $\mathcal{S}'_u$ ensures that $\mathbf{u}$ is always of the form $[\mathbf{0}_{m'}^\top \mid \mathbf{u}'^\top]$ for some $\mathbf{u}' \in \mathbb{Z}_q^m$, then

$$\mathbf{z}_2^\top + \mathbf{u}^\top \mathbf{G} = \mathbf{s}^\top [\mathbf{B} \mid \mathbf{P}] + \mathbf{e}_2^\top + [\mathbf{0}_{m'}^\top \mid \mathbf{u}'^\top \mathbf{G}],$$

which aligns with the corresponding component in Assumption 3. Finally, we explain the role of the $\mathbf{r}_{1,j}$ components (note $\ell = 1$) in Assumption 3. Roughly, we consider the term

$$\begin{aligned} \mathbf{z}_3^\top &= \mathbf{s}^\top \mathbf{P}[\mathbf{G}^{-1}(\mathbf{v}_1) \mid \cdots \mid \mathbf{G}^{-1}(\mathbf{v}_k)] + \mathbf{e}_3^\top \\ &= (\mathbf{s}^\top \mathbf{P} + \mathbf{u}^\top \mathbf{G})[\mathbf{G}^{-1}(\mathbf{v}_1) \mid \cdots \mid \mathbf{G}^{-1}(\mathbf{v}_k)] - \mathbf{u}^\top [\mathbf{v}_1 \mid \cdots \mid \mathbf{v}_k] + \mathbf{e}_3^\top \\ &\stackrel{c}{\approx} \delta_2^\top [\mathbf{G}^{-1}(\mathbf{v}_1) \mid \cdots \mid \mathbf{G}^{-1}(\mathbf{v}_k)] - \mathbf{u}^\top [\mathbf{v}_1 \mid \cdots \mid \mathbf{v}_k] + \mathbf{e}_3^\top. \end{aligned}$$

Since $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ are public, and $\mathbf{z}_3^\top$ includes the term $\mathbf{u}^\top [\mathbf{v}_1 \mid \cdots \mid \mathbf{v}_k]$ masked with the noise $\mathbf{e}_3$, its distribution—conditioned on all other public values—is effectively determined by the inner products $\mathbf{u}^\top \mathbf{v}_j$ and the added noise. In particular, if $\mathbf{u}$ is constrained to have zero entries in its first m' coordinates, then the first m' entries of $\mathbf{v}_j$ (corresponding to the $\mathbf{r}_{1,j}$ components in Assumption 3) have no influence on the distribution of $\mathbf{z}_3$. This explains why such components can be included in the assumption without affecting its security implications.

Remark 10 (Public-coin and private-coin sampler). As justified by [HLL24], it is necessary for the sampling procedure of $\mathbf{u}$ to be private-coin, in order to prevent trivial attacks where the adversary directly computes the inner-products $\mathbf{u}^\top \mathbf{v}_j$. In our setting, all other components—including the global parameters and the tuples $(\mathbf{r}_{i,j}, \mathbf{v}_{i,j})$ —are sampled using a public-coin process, thereby avoiding the obfuscation-based counterexamples (cf. [BÜW24]). Moreover, the public-coin nature of $(\mathbf{r}_{i,j}, \mathbf{v}_{i,j})$ ensures that these vectors can be reused across multiple experiments.

Remark 11 (Reduction from Evasive LWE in [WWW22]). In Appendix A, we provide a reduction from the evasive LWE assumption in [WWW22] to our evIPFE assumption. Consequently, the security of our MA-ABevIPFE construction (cf. Section VI) can be based on the standard LWE assumption and the evasive LWE assumption from [WWW22].

B. The Indistinguishability-Based Evasive IPFE Assumption

In this section, we present an indistinguishability-based variant of the evIPFE assumption introduced in the previous subsection. This version refines the original assumption by replacing distributional indistinguishability with a two-challenge indistinguishability game, where an adversary is challenged to distinguish between two inner-product encodings corresponding to $\mathbf{u}_0$ and $\mathbf{u}_1$.

Assumption 4 (Indistinguishability-Based Evasive IPFE (IND-evIPFE) Assumption). Let $\lambda \in \mathbb{N}$ be a security parameter, and let $q, n, m = n \lceil \log q \rceil, m', \chi, \chi'$ be lattice parameters specified by λ . Denote $\text{gp} = (1^\lambda, q, 1^n, 1^m, 1^{m'}, 1^\chi, 1^{\chi'})$. Let $\mathcal{S}$ be an algorithm defined as follows:

- $\mathcal{S}(\text{gp}; \mathbf{r}_{\text{pub}})$: Given the security parameter λ and public randomness $\mathbf{r}_{\text{pub}} \in \{0, 1\}^*$, the algorithm outputs parameters

$$1^\ell, \{1^{k_i}\}_{i \in [\ell]},$$

followed by $k_1 + \cdots + k_\ell$ vector tuples

$$(\mathbf{r}_{1,1}, \mathbf{v}_{1,1}), (\mathbf{r}_{1,2}, \mathbf{v}_{1,2}), \dots, (\mathbf{r}_{1,k_1}, \mathbf{v}_{1,k_1}); \dots; (\mathbf{r}_{\ell,1}, \mathbf{v}_{\ell,1}), \dots, (\mathbf{r}_{\ell,k_\ell}, \mathbf{v}_{\ell,k_\ell}),$$

where $\mathbf{r}_{i,j} \in \mathbb{Z}_q^{m'}$, $\mathbf{v}_{i,j} \in \mathbb{Z}_q^n$ and $(\mathbf{r}_{i,j}, \mathbf{v}_{i,j}) \neq (\mathbf{0}^{m'}, \mathbf{0}^n)$ for all $i \in [\ell], j \in [k_i]$. The output also includes two additional vectors $\mathbf{u}_0, \mathbf{u}_1 \in \mathbb{Z}_q^n$.

For two adversaries $\mathcal{A}_0$ and $\mathcal{A}_1$, we define their respective advantage functions as follows:

$$\begin{aligned} \text{Adv}_{\mathcal{A}_0}^{\text{Pre}}(\lambda) &:= \left| \Pr [\mathcal{A}_0(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{z}_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top + [\mathbf{0} \mid \mathbf{u}_0^\top \mathbf{G}], \mathbf{z}_3^\top) = 1] \right. \\ &\quad \left. - \Pr [\mathcal{A}_0(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{z}_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top + [\mathbf{0} \mid \mathbf{u}_1^\top \mathbf{G}], \mathbf{z}_3^\top) = 1] \right|; \\ \text{Adv}_{\mathcal{A}_1}^{\text{Post}}(\lambda) &:= \left| \Pr [\mathcal{A}_1(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{z}_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top + [\mathbf{0} \mid \mathbf{u}_0^\top \mathbf{G}], \{\mathbf{K}_i\}_{i \in [\ell]}) = 1] \right. \\ &\quad \left. - \Pr [\mathcal{A}_1(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{z}_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top + [\mathbf{0} \mid \mathbf{u}_1^\top \mathbf{G}], \{\mathbf{K}_i\}_{i \in [\ell]}) = 1] \right|; \end{aligned}$$

where the parameters are sampled as follows:

- $\begin{pmatrix} 1^\ell, \{1^{k_i}\}_{i \in [\ell]}; \\ (\mathbf{r}_{1,1}, \mathbf{v}_{1,1}), \dots, (\mathbf{r}_{1,k_1}, \mathbf{v}_{1,k_1}); \\ \dots \\ (\mathbf{r}_{\ell,1}, \mathbf{v}_{\ell,1}), \dots, (\mathbf{r}_{\ell,k_\ell}, \mathbf{v}_{\ell,k_\ell}); \\ \mathbf{u}_0, \mathbf{u}_1 \end{pmatrix} \leftarrow \mathcal{S}(\text{gp}; \mathbf{r}_{\text{pub}}),$
- $\mathbf{B}_1, \dots, \mathbf{B}_\ell \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}, \mathbf{P}_1, \dots, \mathbf{P}_\ell \xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{B}^\top \leftarrow [\mathbf{B}_1^\top \mid \dots \mid \mathbf{B}_\ell^\top], \mathbf{P}^\top \leftarrow [\mathbf{P}_1^\top \mid \dots \mid \mathbf{P}_\ell^\top],$
- $\mathbf{Q}_i \leftarrow [\mathbf{B}_i \mid \mathbf{P}_i] \begin{bmatrix} \mathbf{r}_{i,1} \\ \mathbf{G}^{-1}(\mathbf{v}_{i,1}) \end{bmatrix} \mid \dots \mid \begin{bmatrix} \mathbf{r}_{i,k_i} \\ \mathbf{G}^{-1}(\mathbf{v}_{i,k_i}) \end{bmatrix} \in \mathbb{Z}_q^{n \times k_i},$
- $(\mathbf{A}_1, \text{td}_1), \dots, (\mathbf{A}_\ell, \text{td}_\ell) \leftarrow \text{TrapGen}(1^n, 1^m, q),$
- $\mathbf{s}_1, \dots, \mathbf{s}_\ell \xleftarrow{\$} \mathbb{Z}_q^n, \mathbf{s}^\top \leftarrow [\mathbf{s}_1^\top \mid \dots \mid \mathbf{s}_\ell^\top] \in \mathbb{Z}_q^{n\ell},$
- $\mathbf{e}_{1,i} \leftarrow D_{\mathbb{Z}, \chi}^m, \mathbf{e}_{3,i} \leftarrow D_{\mathbb{Z}, \chi}^{k_i} \text{ for each } i \in [\ell], \mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^{m'+m},$
- $\mathbf{z}_{1,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top, \mathbf{z}_{3,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{Q}_i + \mathbf{e}_{3,i}^\top \text{ for each } i \in [\ell], \quad \mathbf{z}_2^\top \leftarrow \mathbf{s}^\top [\mathbf{B} \mid \mathbf{P}] + \mathbf{e}_2^\top,$
- $\mathbf{K}_i \leftarrow \text{SamplePre}(\mathbf{A}_i, \text{td}_i, \mathbf{Q}_i, \chi') \text{ for each } i \in [\ell].$

We say that the $\text{IND-evIPFE}_{n,m,m',q,\chi,\chi'}$ assumption holds if, for every efficient sampler $\mathcal{S}$, the following implication holds: If there exists an efficient adversary $\mathcal{A}_1$ with non-negligible advantage function $\text{Adv}_{\mathcal{S}, \mathcal{A}_1}^{\text{Post}}(\lambda)$, then there exists another efficient adversary $\mathcal{A}_0$ with non-negligible advantage function $\text{Adv}_{\mathcal{S}, \mathcal{A}_0}^{\text{Pre}}(\lambda)$.

Remark 12 (Public-coin Sampler). In contrast to Assumption 3, Assumption 4 adopts a public-coin sampler. This design is justified by the fact that, in the indistinguishability-based setting, there is no need to keep the challenge vectors $\mathbf{u}_0, \mathbf{u}_1$ private, since computing $\mathbf{u}_0^\top \mathbf{v}_j$ and $\mathbf{u}_1^\top \mathbf{v}_j$ does not help the adversary distinguish between the two challenge instances. Consequently, the two separate algorithms $\mathcal{S}_u$ and $\mathcal{S}_v$ in Assumption 3 can be unified into a single public-coin algorithm in this context. The adoption of the public-coin setting also helps to exclude obfuscation-based counterexamples, as discussed in [BÜW24].

Remark 13. Assumption 4 can be viewed as an indistinguishability-based variant of Assumption 3. However, it is important to emphasize that the evasive IPFE assumption introduced in [HLL24] cannot be directly reformulated as an indistinguishability-style assumption. Specifically, even if the precondition

$$(1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{z}_1^\top, \mathbf{P}, \mathbf{z}_2^\top + \mathbf{u}_0^\top \mathbf{G}, \mathbf{z}_3^\top) \stackrel{c}{\approx} (1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{z}_1^\top, \mathbf{P}, \mathbf{z}_2^\top + \mathbf{u}_1^\top \mathbf{G}, \mathbf{z}_3^\top), \quad (7)$$

holds, where $\mathbf{u}_0, \mathbf{u}_1 \in \mathbb{Z}_q^d, \mathbf{V} = [\mathbf{v}_1 \mid \dots \mid \mathbf{v}_k] \in \mathbb{Z}_q^{d \times k}$ are sampled by some algorithm $\mathcal{S}' = (\mathcal{S}'_v, \mathcal{S}'_u)$, and the remaining parameters are sampled as follows:

$$\begin{aligned} \mathbf{A} &\xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{P} \xleftarrow{\$} \mathbb{Z}_q^{n \times D}, \mathbf{Q} = \mathbf{P} \mathbf{G}^{-1}(\mathbf{V}) = \mathbf{P}[\mathbf{G}^{-1}(\mathbf{v}_1) \mid \dots \mid \mathbf{G}^{-1}(\mathbf{v}_k)], \\ \mathbf{s} &\xleftarrow{\$} \mathbb{Z}_q^n, \mathbf{e}_1 \leftarrow D_{\mathbb{Z}, \chi}^m, \mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^D, \mathbf{e}_3 \leftarrow D_{\mathbb{Z}, \chi}^k, \\ \mathbf{z}_1^\top &\leftarrow \mathbf{s}^\top \mathbf{A} + \mathbf{e}_1^\top, \mathbf{z}_2^\top \leftarrow \mathbf{s}^\top \mathbf{P} + \mathbf{e}_2^\top, \mathbf{z}_3^\top \leftarrow \mathbf{s}^\top \mathbf{Q} + \mathbf{e}_3^\top, \end{aligned}$$

we cannot in general conclude the corresponding postcondition as in the pseudorandomness-version assumption, namely,

$$(1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{z}_1^\top, \mathbf{P}, \mathbf{z}_2^\top + \mathbf{u}_0^\top \mathbf{G}, \mathbf{K}) \stackrel{c}{\approx} (1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{z}_1^\top, \mathbf{P}, \mathbf{z}_2^\top + \mathbf{u}_1^\top \mathbf{G}, \mathbf{K}), \quad (8)$$

where $\mathbf{K} \leftarrow \mathbf{A}_{\chi'}^{-1}(\mathbf{Q})$. To illustrate this, we consider a sampler $\mathcal{S}'$ that always outputs the all-zero matrix

$\mathbf{V} = \mathbf{0}_{d \times k}$. Informally, we then observe the following chain of distributions:

$$\begin{aligned}
& (1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{z}_1^\top, \mathbf{P}, \mathbf{z}_2^\top + \mathbf{u}_0^\top \mathbf{G}, \mathbf{z}_3^\top) \\
& \equiv (1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{z}_1^\top, \mathbf{P}, \mathbf{z}_2^\top + \mathbf{u}_0^\top \mathbf{G}, (\mathbf{z}_2^\top + \mathbf{u}_0^\top \mathbf{G})\mathbf{G}^{-1}(\mathbf{V}) + \mathbf{e}_3^\top - \mathbf{e}_2^\top \mathbf{G}^{-1}(\mathbf{V}) - \mathbf{u}_0^\top \mathbf{V}) \\
& \stackrel{c}{\approx} (1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \delta_1^\top, \mathbf{P}, \delta_2^\top, \delta_2^\top \mathbf{G}^{-1}(\mathbf{V}) + \mathbf{e}_3^\top - \mathbf{e}_2^\top \mathbf{G}^{-1}(\mathbf{V}) - \mathbf{u}_0^\top \mathbf{V}) \\
& \approx (1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \delta_1^\top, \mathbf{P}, \delta_2^\top, \delta_2^\top \mathbf{G}^{-1}(\mathbf{V}) + \mathbf{e}_3^\top - \mathbf{e}_2^\top \mathbf{G}^{-1}(\mathbf{V}) - \mathbf{u}_1^\top \mathbf{V}) \\
& \stackrel{c}{\approx} (1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{z}_1^\top, \mathbf{P}, \mathbf{z}_2^\top + \mathbf{u}_1^\top \mathbf{G}, (\mathbf{z}_2^\top + \mathbf{u}_1^\top \mathbf{G})\mathbf{G}^{-1}(\mathbf{V}) + \mathbf{e}_3^\top - \mathbf{e}_2^\top \mathbf{G}^{-1}(\mathbf{V}) - \mathbf{u}_1^\top \mathbf{V}) \\
& \equiv (1^\lambda, \mathbf{r}_{\text{pub}}, \mathbf{A}, \mathbf{z}_1^\top, \mathbf{P}, \mathbf{z}_2^\top + \mathbf{u}_1^\top \mathbf{G}, \mathbf{z}_3^\top),
\end{aligned}$$

which implies the precondition (7). However, in the postcondition, the presence of the low-norm matrix $\mathbf{K}$ satisfying $\mathbf{AK} = \mathbf{PG}^{-1}(\mathbf{V})$ yields a short basis of the q -ary lattice $\{\mathbf{x} \in \mathbb{Z}_q^m : \mathbf{Ax} = \mathbf{0} \bmod q\}$ with high probability—a trapdoor for the matrix $\mathbf{A}$. This trapdoor enables the adversary to distinguish between $\mathbf{z}_2^\top + \mathbf{u}_0^\top \mathbf{G}$ and $\mathbf{z}_2^\top + \mathbf{u}_1^\top \mathbf{G}$, in the postcondition, thereby breaking indistinguishability.

In contrast, Assumption 4 avoids this issue by defining the matrix $\mathbf{Q}_i$ as

$$\mathbf{Q}_i \leftarrow [\mathbf{B}_i \mid \mathbf{P}_i] \left[\begin{array}{c|c|c} \mathbf{r}_{i,1} & \cdots & \mathbf{r}_{i,k_i} \\ \hline \mathbf{G}^{-1}(\mathbf{v}_{i,1}) & \cdots & \mathbf{G}^{-1}(\mathbf{v}_{i,k_i}) \end{array} \right] \in \mathbb{Z}_q^{n \times k_i},$$

Each column vector $\begin{bmatrix} \mathbf{r}_{i,j} \\ \mathbf{G}^{-1}(\mathbf{v}_{i,j}) \end{bmatrix}$ is selected to be a nonzero vector and independent of the uniformly random matrix $[\mathbf{B}_i \mid \mathbf{P}_i]$. As a result, the marginal distribution of each column of $\mathbf{Q}_i$ is uniform. Thus, the zeroizing attack, for example, by computing

$$(\mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top) \mathbf{K}_i \approx \mathbf{s}_i^\top \mathbf{Q}_i$$

does not yield a valid equation over the integers. This design aligns with the idea of [Wee22] when introducing the evasive LWE assumption.

V. MULTI-AUTHORITY ATTRIBUTE-BASED INNER PRODUCT FUNCTIONAL ENCRYPTION

In this section, we present and formalize the core definitions underlying our work. We begin by introducing the notion of multi-authority attribute-based (noisy) inner-product functional encryption (MA-AB(N)IPFE), which generalizes previous NIPFE [Agr19, AP20] frameworks to support noise-tolerant decryption and multi-authority access control. We then introduce a relaxed “evasive” variant, called multi-authority attribute-based evasive inner-product functional encryption (MA-ABevIPFE), inspired by the evasive IPFE model of [HLL24], which captures a weaker security notion more suitable for certain lattice-based instantiations. Both schemes are formalized in the static security model, with a focus on subset policies for simplicity and clarity.

A. Multi-authority Attribute-based (Noisy) Inner-Product Functional Encryption (MA-AB(N)IPFE)

We first introduce the concept of *monotone access structures* [Bei96], which play a fundamental role in access control mechanisms and secret-sharing schemes. The monotonicity property captures the idea that adding participants to an authorized subset preserves its authorization status. In this context, subset policies can be regarded as a specific instance of monotone access structures.

Definition 14 (Access Structure, [Bei96]). Let S be a set and 2^S be the power set of S , i.e., the collection of all subsets of S . An access structure on S is a set $\mathbb{A} \subseteq 2^S \setminus \{\emptyset\}$, consisting of some non-empty subsets of S . A subset $A \in 2^S$ is called *authorized* if $A \in \mathbb{A}$, and *unauthorized* otherwise.

An access structure is called *monotone* if it satisfies the following condition: for all subsets $B, C \in 2^S$, if $B \in \mathbb{A}$ and $B \subseteq C$, then $C \in \mathbb{A}$. In other words, adding more elements to an authorized subset does not invalidate its authorization.

In the following, we begin by presenting the definition of multi-authority attribute-based (noisy) inner-product functional encryption (MA-AB(N)IPFE). For simplicity, we adopt the *small universe* setting, as described in [DP23], where each authority controls a single attribute, following the convention of [RW15, DKW21a]. In this setting, an authority and its corresponding attribute can be considered equivalent to some extent. This definition naturally extends to a more general setting in which each authority can manage an arbitrary polynomial number of attributes (with respect to the security parameter), as outlined in [RW15]. Our definition extends the standard MA-ABIPFE model (e.g., [DP23]) by allowing the decryption to yield a noisy approximation of the inner product. This reflects more realistic applications where exact reconstruction of the inner product is not required and allows for a broader range of lattice-based constructions. Formally, we define the following.

Definition 15 (MA-AB(N)IPFE, [AGT21, DP23]). *Let λ be the security parameter, and $n = n(\lambda), q = q(\lambda)$ be lattice parameters. Let $\mathcal{AU}$ denote the universe of authority identifiers, and $\mathcal{GID}$ be the universe of global identifiers for users. Let $B_0 = B_0(\lambda), B_1 = B_1(\lambda)$ be bounding values. A (B_0, B_1) -multi-authority attribute-based (noisy) inner-product functional encryption scheme for a class of policies $\mathcal{P}$ (described by a monotone access structure on a subset of $\mathcal{AU}$) over the vector space $\mathbb{Z}_q^n$ is defined as a tuple of efficient algorithms $\Pi_{\text{MA-AB(N)IPFE}} = (\text{GlobalSetup}, \text{AuthSetup}, \text{Keygen}, \text{Enc}, \text{Dec})$. These algorithms proceed as follows:*

- $\text{GlobalSetup}(1^\lambda) \rightarrow \text{gp}$: *The global setup algorithm takes as input the security parameter λ , and outputs the global parameters gp . We assume that gp specifies the description of $\mathcal{AU}$, $\mathcal{GID}$, n , and q .*
- $\text{AuthSetup}(\text{gp}, \text{aid}) \rightarrow (\text{pk}_{\text{aid}}, \text{msk}_{\text{aid}})$: *The authority (local) setup algorithm takes as input the global parameters gp and an authority identifier $\text{aid} \in \mathcal{AU}$. It outputs a public key pk_{aid} and a master secret key msk_{aid} .*
- $\text{Keygen}(\text{gp}, \text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}, \text{gid}, \mathbf{v}) \rightarrow \text{sk}_{\text{aid}, \text{gid}, \mathbf{v}}$: *The key generation algorithm takes as input the global parameters gp , the public key pk_{aid} , the authority's master secret key msk_{aid} , a user identifier $\text{gid} \in \mathcal{GID}$, and a key vector $\mathbf{v} \in \mathbb{Z}_q^n$. It outputs a secret key $\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}}$ associated with attribute aid , user identifier gid , and key vector $\mathbf{v}$.*
- $\text{Enc}(\text{gp}, \mathbb{A}, \{\text{pk}_{\text{aid}}\}_{\text{aid} \in A}, \mathbf{u}) \rightarrow \text{ct}$: *The encryption algorithm takes as input the global parameters gp , an access structure $\mathbb{A} \in \mathcal{P}$ on a set of authorities $A \subseteq \mathcal{AU}$, the set of public keys $\{\text{pk}_{\text{aid}}\}$ associated with authorities $\text{aid} \in A$, and a plaintext vector $\mathbf{u} \in \mathbb{Z}_q^n$. It outputs a ciphertext ct .*
- $\text{Dec}(\text{gp}, \{\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}}\}_{\text{aid} \in A}, \text{gid}, \mathbf{v}, \text{ct}) \rightarrow \Gamma$: *The decryption algorithm takes as input the global parameters gp , a collection of secret keys $\{\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}}\}$ associated with authorities $\text{aid} \in A$, a user identifier $\text{gid} \in \mathcal{GID}$, and a key vector $\mathbf{v} \in \mathbb{Z}_q^n$, and a ciphertext ct . It outputs a value $\Gamma \in \mathbb{Z}_q$, corresponding to the (approximate) inner product of $\mathbf{u}$ and $\mathbf{v}$ or a symbol $\perp$ to indicate the failure of decryption.*

Approximate Correctness. A $(B_0(\lambda), B_1(\lambda))$ -MA-ABNIPFE scheme is said to be *correct* if, for every security parameter $\lambda \in \mathbb{N}$, every plaintext vector $\mathbf{u} \in \mathbb{Z}_q^n$, every key vector $\mathbf{v} \in \mathbb{Z}_q^n$, every global identifier $\text{gid} \in \mathcal{GID}$, every set of authorities $A \subseteq \mathcal{AU}$, every access structure $\mathbb{A} \in \mathcal{P}$ defined over A , and every subset of authorized authorities $A' \in \mathbb{A}$, it holds that

$$\Pr \left[\Gamma = \mathbf{u}^\top \mathbf{v} + e_0 \mid \begin{array}{l} \text{gp} \leftarrow \text{GlobalSetup}(1^\lambda); \\ \forall \text{aid} \in A, (\text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}) \leftarrow \text{AuthSetup}(\text{gp}, \text{aid}); \\ \forall \text{aid} \in A', \text{sk}_{\text{aid}, \text{gid}, \mathbf{v}} \leftarrow \text{Keygen}(\text{gp}, \text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}, \text{gid}, \mathbf{v}); \\ \text{ct} \leftarrow \text{Enc}(\text{gp}, \mathbb{A}, \{\text{pk}_{\text{aid}}\}_{\text{aid} \in A}, \mathbf{u}); \\ \Gamma \leftarrow \text{Dec}(\text{gp}, \{\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}}\}_{\text{aid} \in A'}, \text{gid}, \mathbf{v}, \text{ct}); \\ e_0 \in [-B_0(\lambda), B_0(\lambda)] \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

Intuitively, we adopt an approximate correctness guarantee: if the decryption is authorized, the decryption result Γ should recover the inner product $\mathbf{u}^\top \mathbf{v}$ up to a small additive noise term e_0 , bounded in absolute value by $B_0(\lambda)$.

Static Security. In this paper, we extend the static security model of [AGT21, DP23] by incorporating a *noisy* setting. This model is adapted from the security model for MA-ABE in [RW15] and for noisy IPFE in [AP20]. This model is formalized as a game between a challenger and an adversary. The term *static* refers to the restriction that the adversary must specify all of its queries at the beginning of the game.

For a security parameter $\lambda \in \mathbb{N}$, we define the static security game between a challenger and an adversary $\mathcal{A}$ for an MA-AB(N)IPFE scheme as follows:

Global Setup: The challenger runs $\text{gp} \leftarrow \text{GlobalSetup}(1^\lambda)$ to generate the global parameters and sends gp to the adversary $\mathcal{A}$.

Adversary's Queries: The adversary specifies the following queries:

- A set of corrupt authorities $\mathcal{C} \subseteq \mathcal{AU}$, along with their respective public keys pk_{aid} for each corrupt authority $\text{aid} \in \mathcal{C}$.
- A set of non-corrupt authorities $\mathcal{N} \subseteq \mathcal{AU}$, where $\mathcal{N} \cap \mathcal{C} = \emptyset$.
- A set of secret-key queries $\mathcal{Q} = \{(\text{gid}, A, \mathbf{v})\}$ where each query specifies a global identifier $\text{gid} \in \mathcal{GID}$, a subset of non-corrupt authorities $A \subseteq \mathcal{N}$, and a key vector $\mathbf{v} \in \mathbb{Z}_q^n$. We assume without loss of generality that each pair $(\text{gid}, \mathbf{v})$ appears at most once in the query set $\mathcal{Q}$, even if associated with different authority subsets A . This avoids the adversary combining keys from multiple key responses to enable decryption that would otherwise be unauthorized.
- A pair of plaintext vectors $\mathbf{u}_0, \mathbf{u}_1 \in \mathbb{Z}_q^n$, a set of authorities $A^* \subseteq \mathcal{C} \cup \mathcal{N}$, and an access structure $\mathbb{A} \in \mathcal{P}$ defined over A^* .

Challenger's Replies: The challenger first flips a fair coin $\beta \xleftarrow{\$} \{0, 1\}$ and generates key pairs $(\text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}) \leftarrow \text{AuthSetup}(\text{gp}, \text{aid})$ for each $\text{aid} \in \mathcal{N}$. It then responds to adversary $\mathcal{A}$ as follows:

- The public keys pk_{aid} for each non-corrupt authority $\text{aid} \in \mathcal{N}$.
- The secret keys $\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}} \leftarrow \text{Keygen}(\text{gp}, \text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}, \text{gid}, \mathbf{v})$ for each secret-key query $(\text{gid}, A, \mathbf{v})$ and each $\text{aid} \in A$.
- The challenge ciphertext $\text{ct}_\beta \leftarrow \text{Enc}(\text{gp}, \mathbb{A}, \{\text{pk}_{\text{aid}}\}_{\text{aid} \in A^*}, \mathbf{u}_\beta)$.

Guess Phase: The adversary outputs a guess $\beta' \in \{0, 1\}$ for the value of β .

The *advantage* of the adversary $\mathcal{A}$ in this game is defined by

$$\text{Adv}_{\mathcal{A}}(\lambda) := |\Pr[\beta = \beta'] - 1/2|.$$

Unlike the static security model of MA-ABE (e.g., [DKW21a, WWW22]), where the adversary is restricted to making only *unauthorized* secret-key queries, the static security model for MA-ABNIPFE schemes—as considered in [AGT21, DP23]—allows the adversary to ask secret-key queries that may potentially decrypt the challenge ciphertext and derive an approximate inner product. To account for this broader adversarial capability, we impose the following limitations on the adversary in the game of a $(B_0(\lambda), B_1(\lambda))$ -MA-ABNIPFE scheme:

Definition 16 (Admissible Adversary). *An adversary $\mathcal{A}$ is said to be admissible for the (B_0, B_1) -MA-ABNIPFE security game defined above if, the set $A^* \cap \mathcal{C}$ is not authorized under the access structure $\mathbb{A}$, i.e., $A^* \cap \mathcal{C} \notin \mathbb{A}$, and for every secret-key query $(\text{gid}, A, \mathbf{v}) \in \mathcal{Q}$, at least one of the following conditions holds, ensuring either unauthorized access or functional indistinguishability:*

- **Unauthorized access:** $(A \cup \mathcal{C}) \cap A^* \notin \mathbb{A}$.
- **Functional indistinguishability:** *The vector $\mathbf{v}$ satisfies $\|(\mathbf{u}_0 - \mathbf{u}_1)^\top \mathbf{v}\| \leq B_1$. (This ensures that the adversary cannot gain useful distinguishing information from the inner product of the challenge plaintexts.)*

Definition 17 (Static Security for an MA-ABNIPFE scheme). *An (B_0, B_1) -MA-ABNIPFE scheme is said to be statically secure if, for every efficient admissible adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that, for all $\lambda \in \mathbb{N}$, the advantage of $\mathcal{A}$ in the above security game is at most $\text{negl}(\lambda)$.*

Remark 18 (Noiseless MA-ABIPFE). *For a correct and statically secure (B_0, B_1) -MA-ABNIPFE, it is natural to require that $B_0 \geq B_1$ holds. Otherwise, an admissible adversary could distinguish between the*

two challenge ciphertexts by querying an authorized secret key associated with a key vector $\mathbf{v}$, for which $(\mathbf{u}_0 - \mathbf{u}_1)^\top \mathbf{v} = B_1 > B_0$. In this scenario, the difference between the two inner products would not be sufficiently obscured by the lower noise level guaranteed by the decryption process, thereby compromising security.

In particular, when $B_0 = B_1 = 0$, the (B_0, B_1) -MA-ABNIPFE degenerates into a noiseless MA-ABIPFE scheme, or simply an MA-ABIPFE scheme for brevity. Specifically, in this case, the decryption process using authorized secret keys is expected to return the exact inner product (except with negligible probability). Meanwhile, in the security game, an admissible adversary is restricted to submitting secret-key queries that are either unauthorized or guaranteed to yield the same inner product when decrypting both challenge ciphertexts. In this case, this definition coincides with the standard definition of MA-ABIPFE as formulated in [AGT21, DP23].

Remark 19 (Static Security in the Random Oracle Model). *In this work, we analyze the static security of our MA-AB(N)IPFE construction in the random oracle model [BR93], following the approach of [RW15, DKW21a, DKW21b, WWW22, DP23]. In this setting, the hash function H (modeled as a random oracle programmed by the challenger) is specified as part of the global parameters, and all parties in the scheme have access to this function.*

During the static security game in the random oracle model, the adversary is allowed to submit the random oracle queries as part of its initial queries. It is further allowed to make additional random oracle queries during the phase in which the challenger provides its responses. The challenger must answer all such queries consistently throughout the interaction.

MA-AB(N)IPFE for subset policies. In this paper, we primarily focus on constructing an MA-AB(N)IPFE scheme for the class of subset policies. In this setting, the ciphertext is encrypted under a set of authorities (attributes) $A \subseteq \mathcal{AU}$, which uniquely determines the access structure under the subset policy setting. Consequently, the access structure $\mathbb{A}$ does not need to be explicitly specified in the encryption algorithm, as the policy is implicitly defined by the set A . The decryption algorithm succeeds only if a user possesses secret keys associated with a set of authorities B such that $A \subseteq B$.

The flexibility of the subset policy model provides an efficient framework for expressing access control rules, making it well-suited for a wide range of applications that require dynamic access restrictions based on user roles or permissions. By combining the definition of the MA-ABE scheme for subset policies, as introduced in [WWW22], with the definition of MA-ABIPFE scheme in [AGT21, DP23], we formalize the following definition.

Definition 20 (MA-AB(N)IPFE for subset policies). *Let λ be a security parameter and $n = n(\lambda)$, $q = q(\lambda)$ be lattice parameters. Let $\mathcal{AU}$ be the universe of authority identifiers. Define the class of subset policies $\mathcal{P}$ as*

$$\mathcal{P} = \{\mathbb{A} : \mathbb{A} = \{B : B \supseteq A\} \text{ where } A \subseteq \mathcal{AU}\}.$$

An access structure $\mathbb{A}$ for a subset policy is determined by the set $A \subseteq \mathcal{AU}$. For an $(B_0(\lambda), B_1(\lambda))$ -MA-AB(N)IPFE scheme $\Pi_{\text{MA-AB(N)IPFE}} = \{\text{GlobalSetup}, \text{AuthSetup}, \text{Keygen}, \text{Enc}, \text{Dec}\}$ for subset policies over the vector space $\mathbb{Z}_q^n$, we can omit the explicit specification of the access structure $\mathbb{A}$ in the encryption algorithm, since it is implicitly defined by the set of public keys $\{\text{pk}_{\text{aid}}\}_{\text{aid} \in A}$. Precisely, the encryption algorithm Enc in Definition 15 can be simplified as follows (with other algorithms unchanged):

- $\text{Enc}(\text{gp}, \{\text{pk}_{\text{aid}}\}_{\text{aid} \in A}, \mathbf{u}) \rightarrow \text{ct}$: *The encryption algorithm takes as input the global parameters gp , the set of public keys $\{\text{pk}_{\text{aid}}\}$ corresponding to authorities $\text{aid} \in A$, and a plaintext vector $\mathbf{u} \in \mathbb{Z}_q^n$. It outputs a ciphertext ct .*

In the subset policy setting, admissibility (Definition 16) of an adversary imposes specific structural constraints on secret-key queries. We now make this more explicit by classifying such queries into two types. Let $\mathcal{Q} = \{(\text{gid}, A, \mathbf{v})\}$ denote the set of secret-key queries and let A^* denote the set of attributes

related to the challenging ciphertext. According to Definition 16, an admissible adversary is additionally required to satisfy $A^* \cap \mathcal{C} \subsetneq A^*$, and each query $(\text{gid}, A, \mathbf{v}) \in \mathcal{Q}$ must satisfy at least one of the following conditions:

- **Unauthorized access:** $(A \cup \mathcal{C}) \cap A^* \subsetneq A^*$.
- **Functional indistinguishability:** $\|(\mathbf{u}_1 - \mathbf{u}_0)^\top \mathbf{v}\| \leq B_1$.

To facilitate the analysis in our security proof, we classify secret-key queries into the following types:

- **Type I secret-key query:** A query satisfying $(A \cup \mathcal{C}) \cap A^* \subsetneq A^*$.
- **Type II secret-key query:** A query satisfying $(A \cup \mathcal{C}) \cap A^* = A^*$ and $\|(\mathbf{u}_1 - \mathbf{u}_0)^\top \mathbf{v}\| \leq B_1$.

It follows directly that each secret-key query must belong to one of these two types.

Remark 21. As mentioned in [WWW22], the MA-ABE scheme for subset policies implies an MA-ABE scheme for access structure supporting access structures defined by a polynomial-size conjunction or a DNF formula. Similarly, our construction of the MA-AB(N)IPFE scheme for subset policies can also be extended to an MA-AB(N)IPFE scheme for access structure decided by conjunction or a DNF formula by following an analogous argument.

B. Multi-authority Attribute-based Evasive Inner-product Functional Encryption (MA-ABeIPFE)

Inspired by the evasive IPFE scheme introduced in [HLL24], we define the notion of a *multi-authority attribute-based evasive inner-product functional encryption* (MA-ABeIPFE) scheme by relaxing the security requirement of the MA-AB(N)IPFE scheme described above. In [HLL24], the evasive IPFE captures a “generic-model view” analogous to that of evasive LWE. The authors consider the *static* (also known as *very selective* [HLL24]) security model with respect to samplers producing pseudorandom noisy inner-product outputs.

Following this approach, we consider a weakened static security game where the adversary is no longer required to fully specify the entire Type II secret-key queries or the challenge plaintexts. Instead, these components are jointly determined by a predefined sampler and the adversary. We formalize the definition of an (B_0, χ_s) -MA-ABeIPFE scheme accordingly, adopting most of the notations following the convention of [HLL24]. The syntax of the scheme and its approximate correctness guarantee follow the definition of the (B_0, B_1) -MA-AB(N)IPFE scheme given in Definition 15, where the parameter B_0 retains the same role of bounding the decryption noise. We omit restating this part in the MA-ABeIPFE definition below for clarity. We formalize the static security model for MA-ABeIPFE as follows.

Static security. Let $\text{Samp} = (\text{Samp}_v, \text{Samp}_u)$ be two algorithms with the following syntax.

- $\text{Samp}_v(1^\lambda; \mathbf{r}_{\text{pub}})$: The algorithm takes as input the security parameter λ and public randomness $\mathbf{r}_{\text{pub}}$, and outputs a sequence of vectors $\mathbf{v}'_1, \mathbf{v}'_2, \dots, \mathbf{v}'_{Q'} \in \mathbb{Z}_q^n$.
- $\text{Samp}_u(1^\lambda, \mathbf{r}_{\text{pub}}; \mathbf{r}_{\text{pri}})$: The algorithm takes as input the security parameter λ , public randomness $\mathbf{r}_{\text{pub}}$, and private randomness $\mathbf{r}_{\text{pri}}$, and outputs a single vector $\mathbf{u} \in \mathbb{Z}_q^n$.

For a security parameter $\lambda \in \mathbb{N}$, we define the static security game between a challenger and an (admissible) adversary $\mathcal{A}$ for an MA-ABeIPFE scheme, parameterized by the sampler $\text{Samp} = (\text{Samp}_v, \text{Samp}_u)$, as follows:

Global Setup: The challenger runs $\text{gp} \leftarrow \text{GlobalSetup}(1^\lambda)$ to generate the global parameters and sends gp to the adversary $\mathcal{A}$.

Adversary’s Queries: The adversary specifies the following queries:

- A set $\mathcal{C} \subseteq \mathcal{AU}$ of corrupt authorities, along with their respective public keys pk_{aid} for each corrupt authority $\text{aid} \in \mathcal{C}$.
- A set $\mathcal{N} \subseteq \mathcal{AU}$ of non-corrupt authorities, where $\mathcal{N} \cap \mathcal{C} = \emptyset$.
- A set of authorities $A^* \subseteq \mathcal{C} \cup \mathcal{N}$ such that $A^* \cap \mathcal{C} \subsetneq A^*$.
- A set of Type I secret-key queries $\mathcal{Q} = \{(\text{gid}, A, \mathbf{v})\}$, where $A \subseteq \mathcal{N}$ and $(A \cup \mathcal{C}) \cap A^* \subsetneq A^*$.
- A partial set of Type II secret-key queries $\mathcal{Q}'_{\text{par}} = \{(\text{gid}', A')\}$, where $A' \subseteq \mathcal{N}$ and $(A' \cup \mathcal{C}) \cap A^* = A^*$.

Challenger's Replies: Then the challenger proceeds as follows:

- 1) The challenger first flips a fair coin $\beta \xleftarrow{\$} \{0, 1\}$ and generates key pairs $(\text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}) \leftarrow \text{AuthSetup}(\text{gp}, \text{aid})$ for each $\text{aid} \in \mathcal{N}$.
- 2) The challenger samples $\mathbf{r}_{\text{pub}} \xleftarrow{\$} \{0, 1\}^{\kappa_1}$, $\mathbf{r}_{\text{pri}} \xleftarrow{\$} \{0, 1\}^{\kappa_2}$, where κ_1, κ_2 denotes the upper bounds of the random bits used by Samp_{v} and Samp_{u} , respectively. Then it computes

$$(\mathbf{v}'_1, \mathbf{v}'_2, \dots, \mathbf{v}'_{Q'}) \leftarrow \text{Samp}_{\text{v}}(1^\lambda; \mathbf{r}_{\text{pub}}),$$

$$\mathbf{u}_0 \leftarrow \text{Samp}_{\text{u}}(1^\lambda; \mathbf{r}_{\text{pub}}, \mathbf{r}_{\text{pri}}), \mathbf{u}_1 \xleftarrow{\$} \mathbb{Z}_q^n.$$

Next, the challenger builds the full set of Type II queries

$$\mathcal{Q}' = \{(\text{gid}'_1, A'_1, \mathbf{v}'_1), \dots, (\text{gid}'_{Q'}, A'_{Q'}, \mathbf{v}'_{Q'})\}.$$

Before detailing the responses, we make the following assumptions without loss of generality:

- Each pair $(\text{gid}, \mathbf{v})$ appears at most once in $\mathcal{Q}$, for the same reason as in MA-AB(N)IPFE. Likewise, each pair $(\text{gid}', \mathbf{v}')$ appears at most once in $\mathcal{Q}'$, since two queries $(\text{gid}', A'_1, \mathbf{v}')$ and $(\text{gid}', A'_2, \mathbf{v}')$ can, without loss of generality, be replaced by a single query $(\text{gid}', A'_1 \cup A'_2, \mathbf{v}')$.
 - Each pair $(\text{gid}'_j, \mathbf{v}'_j)$ in $\mathcal{Q}'_{\text{par}}$ does not appear in the Type I secret-key queries set $\mathcal{Q}$. If a pair $(\text{gid}'_j, \mathbf{v}'_j)$ appears in both query sets, it can be safely removed from $\mathcal{Q}$. This is because the challenger will already return the secret keys associated with all authorities in $A^* \cap \mathcal{N}$ —these cover all the authorities in any Type I query involving $(\text{gid}'_j, \mathbf{v}'_j)$. As such, the Type I secret-key would be redundant.
 - The partial set of Type II queries $\mathcal{Q}'_{\text{par}}$ is assumed to contain exactly Q' elements, matching the number of vectors output by the sampler Samp_{v} . If necessary, the set $\mathcal{Q}'_{\text{par}}$ can be padded or truncated to ensure that its size matches Q' , without affecting the adversary's view or the underlying distribution.
- 3) It then responds to the adversary $\mathcal{A}$ as follows:
- The public keys pk_{aid} for each non-corrupt authorities $\text{aid} \in \mathcal{N}$.
 - The secret keys $\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}} \leftarrow \text{Keygen}(\text{gp}, \text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}, \text{gid}, \mathbf{v})$ for each Type I secret-key query $(\text{gid}, A, \mathbf{v}) \in \mathcal{Q}$ and each $\text{aid} \in A$.
 - The secret keys $\text{sk}_{\text{aid}, \text{gid}', \mathbf{v}'} \leftarrow \text{Keygen}(\text{gp}, \text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}, \text{gid}', \mathbf{v}')$ for each Type II secret-key query $(\text{gid}', A', \mathbf{v}') \in \mathcal{Q}'$ and each $\text{aid} \in A'$.
 - The challenge ciphertext $\text{ct}_\beta \leftarrow \text{Enc}(\text{gp}, \{\text{pk}_{\text{aid}}\}_{\text{aid} \in A^*}, \mathbf{u}_\beta)$.
 - The public randomness $\mathbf{r}_{\text{pub}}$.

Guess phase: The adversary outputs a guess $\beta' \in \{0, 1\}$ for the value of β .

The *advantage* of the adversary $\mathcal{A}$ in distinguishing β is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{Samp}}(\lambda) := |\Pr[\beta = \beta'] - 1/2|.$$

Definition 22 (Static Security for an MA-ABeVLPFE scheme). *We say that the sampling algorithms $\text{Samp} = (\text{Samp}_{\text{v}}, \text{Samp}_{\text{u}})$ in the game above produces pseudorandom noisy inner products with noise parameter σ , if*

$$(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{u}_0^\top \mathbf{v}'_i + e_i\}_{i \in [Q']}) \stackrel{c}{\approx} (1^\lambda, \mathbf{r}_{\text{pub}}, \{\delta_i\}_{i \in [Q']}),$$

where $e_i \leftarrow D_{\mathbb{Z}, \sigma}$, $\delta_i \xleftarrow{\$} \mathbb{Z}_q$ for each $i \in [Q']$. An MA-ABeVLPFE scheme is said to be *Samp-secure* if for every efficient admissible adversary $\mathcal{A}$, the advantage of $\text{Adv}_{\mathcal{A}}^{\text{Samp}}(\lambda)$ is negligible in λ . Furthermore, an (B_0, χ_s) -MA-ABeVLPFE scheme is said to be *statically secure* if it is *Samp-secure* for all efficient Samp that produce pseudorandom noisy inner products with noise parameter $\sigma \leq \chi_s$.

Remark 23. As justified in [HLL24], to ensure indistinguishability in the static security game, it is necessary to make the sampling process of $\mathbf{u}_0$ private-coin, meaning the distinguisher cannot trivially use

randomness to obtain $\mathbf{u}_0$. Otherwise, the distinguisher may obtain $\mathbf{u}_0$ and $\mathbf{v}'_i$ for $i \in [Q']$, and compute the corresponding inner products $\mathbf{u}_0^\top \mathbf{v}'_i$. This would allow distinguishing the challenge ciphertext and thus break the security of the scheme.

VI. MA-ABeVLPFE SCHEME FROM evLPFE ASSUMPTION (IN THE RANDOM ORACLE MODEL)

In this section, we present our construction of the MA-ABeVLPFE scheme for subset policies in the random oracle model. The construction follows a structure similar to that in [DKW21a, WWW22, HLL24]. Later, we prove the static security of this scheme based on the LWE assumption and evLPFE assumption, analogous to the approach introduced in [WWW22, HLL24].

Construction 1 (MA-ABeVLPFE for subset policies in the random oracle model). *Let λ be the security parameter. Let n and q be lattice parameters, and define $m = n \lceil \log q \rceil$. Let m' , χ , and χ' be additional lattice parameters. Let $\mathcal{AU} = \{0, 1\}^\lambda$ denote the universe of authority identifiers, and let $\mathcal{GID} = \{0, 1\}^\lambda$ denote the universe of global identifiers for users. Let $H : \mathcal{GID} \times \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q^{m'}$ be a hash function, modeled as a random oracle, similarly to Construction 5 in [WWW22]. More precisely,*

- *The outputs of the random oracle H are modeled as following a discrete Gaussian distribution with parameter χ' . Specifically, on each input query $(\text{gid}, \mathbf{v}) \in \mathcal{GID} \times \mathbb{Z}_q^n$, the output $H(\text{gid}, \mathbf{v})$ is distributed according to discrete Gaussian $D_{\mathbb{Z}, \chi'}^{m'}$.*
- *As noted in [WWW22], such a hash function H can be instantiated using a standard random oracle $H' : \mathcal{GID} \times \mathbb{Z}_q^n \rightarrow \{0, 1\}^{\lambda m'}$, where the outputs of $H'(\text{gid}, \mathbf{v})$ are uniformly distributed over $\{0, 1\}^{\lambda m'}$. According to Lemma 5.7 and Remark 5.9 of [WWW22], the outputs of H' can then be efficiently transformed into the desired discrete Gaussian distribution above using inversion sampling techniques.*

A (B_0, χ_s) -MA-ABeVLPFE scheme over $\mathbb{Z}_q^n$ for subset policies consists of a tuple of efficient algorithms $\Pi_{\text{MA-ABeVLPFE}} = (\text{GlobalSetup}, \text{AuthSetup}, \text{Keygen}, \text{Enc}, \text{Dec})$. These algorithms proceed as follows:

- $\text{GlobalSetup}(1^\lambda) \rightarrow \text{gp}$: The global setup algorithm takes as input the security parameter λ , and outputs the global parameters $\text{gp} = (\lambda, n, m, m', q, \chi, \chi', H)$.
- $\text{AuthSetup}(\text{gp}, \text{aid}) \rightarrow (\text{pk}_{\text{aid}}, \text{msk}_{\text{aid}})$: The authority setup algorithm takes as input the global parameters gp and an authority identifier $\text{aid} \in \mathcal{AU}$. It samples $(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, $\mathbf{B}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}$, $\mathbf{P}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$. It outputs a public key $\text{pk}_{\text{aid}} = (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$ and a master secret key $\text{msk}_{\text{aid}} = \text{td}_{\text{aid}}$.
- $\text{Keygen}(\text{gp}, \text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}, \text{gid}, \mathbf{v}) \rightarrow \text{sk}_{\text{aid}, \text{gid}, \mathbf{v}}$: The key generation algorithm takes as input the global parameters gp , the public key $\text{pk}_{\text{aid}} = (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$, the authority's master secret key $\text{msk}_{\text{aid}} = \text{td}_{\text{aid}}$, the user identifier $\text{gid} \in \mathcal{GID}$, the key vector $\mathbf{v} \in \mathbb{Z}_q^n$, the key generation algorithm computes $\mathbf{r} \leftarrow H(\text{gid}, \mathbf{v})$ and uses the trapdoor td_{aid} for $\mathbf{A}_{\text{aid}}$ to sample $\mathbf{k} \leftarrow \text{SamplePre}(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}, \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{B}_{\text{aid}} \mathbf{r}, \chi)$. It outputs a secret key $\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}} = \mathbf{k}$.
- $\text{Enc}(\text{gp}, \{\text{pk}_{\text{aid}}\}_{\text{aid} \in A}, \mathbf{u}) \rightarrow \text{ct}$: The encryption algorithm takes as input the global parameters gp , a set of public keys $\text{pk}_{\text{aid}} = (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$ associated with authorities $A \subseteq \mathcal{AU}$, a plaintext vector $\mathbf{u} \in \mathbb{Z}_q^n$. The encryption algorithm samples $\mathbf{s}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^n$, $\mathbf{e}_{1, \text{aid}} \leftarrow D_{\mathbb{Z}, \chi}^m$ for each $\text{aid} \in A$, $\mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^{m'}$, and $\mathbf{e}_3 \leftarrow D_{\mathbb{Z}, \chi}^m$. It outputs a ciphertext $\text{ct} \in \mathbb{Z}_q^m \times \mathbb{Z}_q^{m'} \times \mathbb{Z}_q^m$, where

$$\text{ct} = \left(\left\{ \mathbf{s}_{\text{aid}}^\top \mathbf{A}_{\text{aid}} + \mathbf{e}_{1, \text{aid}}^\top \right\}_{\text{aid} \in A}, \sum_{\text{aid} \in A} \mathbf{s}_{\text{aid}}^\top \mathbf{B}_{\text{aid}} + \mathbf{e}_2^\top, \sum_{\text{aid} \in A} \mathbf{s}_{\text{aid}}^\top \mathbf{P}_{\text{aid}} + \mathbf{e}_3^\top + \mathbf{u}^\top \mathbf{G} \right).$$

- $\text{Dec}(\text{gp}, \{\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}}\}_{\text{aid} \in A}, \text{gid}, \mathbf{v}, \text{ct}) \rightarrow \Gamma$: The decryption algorithm takes as input the global parameters gp , a collection of secret keys $\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}} = \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}}$ associated with authorities $\text{aid} \in A$, a user identifier gid , a key vector $\mathbf{v}$, and a ciphertext $\text{ct} = (\{\mathbf{c}_{1, \text{aid}}^\top\}_{\text{aid} \in A}, \mathbf{c}_2^\top, \mathbf{c}_3^\top)$. The decryption algorithm first computes $\mathbf{r} \leftarrow H(\text{gid}, \mathbf{v})$ and outputs

$$\Gamma = \mathbf{c}_3^\top \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{c}_2^\top \mathbf{r} - \sum_{\text{aid} \in A} \mathbf{c}_{1, \text{aid}}^\top \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}}.$$

A. Correctness

Theorem 24 (Correctness). *Let L be an upper bound on the number of attributes associated with a ciphertext. Let $\chi_0 = \sqrt{n \log q} \cdot \omega(\sqrt{\log n})$ be a polynomial such that Lemma 7 holds. Suppose that the lattice parameters n, q, χ are such that $\chi \geq \chi_0(n, q)$. Then the scheme $\Pi_{\text{MA-ABevIPFE}}$ in Construction 1 is correct as a (B_0, χ_s) -MA-ABevIPFE scheme, where the parameter $B_0 = \sqrt{\lambda} \chi m + \lambda \chi \chi' m' + \lambda \chi^2 m L$.*

Proof. Let $\mathbf{u} \in \mathbb{Z}_q^n$ be any plaintext, $\mathbf{v} \in \mathbb{Z}_q^n$ a key vector, $\text{gid} \in \mathcal{GID}$ a user identifier, and A a set of authorities. First, sample the global parameters $\text{gp} \leftarrow \text{GlobalSetup}(1^\lambda)$, the authority key pairs $(\text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}) \leftarrow \text{AuthSetup}(\text{gp}, \text{aid})$ for each $\text{aid} \in A$, the secret keys $\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}} \leftarrow \text{Keygen}(\text{gp}, \text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}, \text{gid}, \mathbf{v})$ for each $\text{aid} \in A$, and the ciphertext $\text{ct} \leftarrow \text{Enc}(\text{gp}, \{\text{pk}_{\text{aid}}\}_{\text{aid} \in A}, \mathbf{u})$.

In the following, we verify the correctness by expanding the computation of the decryption process $\text{Dec}(\text{gp}, \{\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}}\}_{\text{aid} \in A}, \text{gid}, \mathbf{v}, \text{ct})$ in detail:

- The global parameters $\text{gp} = (\lambda, n, m, m', q, \chi, \chi', H)$ consist of the security parameter, the lattice parameters, and the description of a hash function $H: \mathcal{GID} \times \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q^{m'}$.
- For the plaintext vector $\mathbf{u} \in \mathbb{Z}_q^n$, the ciphertext associated with authorities $\{\text{aid}\}_{\text{aid} \in A}$ is constructed as $(\{\mathbf{c}_{1, \text{aid}}^\top\}_{\text{aid} \in A}, \mathbf{c}_2^\top, \mathbf{c}_3^\top)$, where

$$\mathbf{c}_{1, \text{aid}}^\top = \mathbf{s}_{\text{aid}}^\top \mathbf{A}_{\text{aid}} + \mathbf{e}_{1, \text{aid}}^\top, \mathbf{c}_2^\top = \sum_{\text{aid} \in A} \mathbf{s}_{\text{aid}}^\top \mathbf{B}_{\text{aid}} + \mathbf{e}_2^\top, \mathbf{c}_3^\top = \sum_{\text{aid} \in A} \mathbf{s}_{\text{aid}}^\top \mathbf{P}_{\text{aid}} + \mathbf{e}_3^\top + \mathbf{u}^\top \mathbf{G},$$

where $(\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$ is the public key associated with the authority $\text{aid} \in A$.

- Each secret key is generated as $\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}} = \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}} \leftarrow \text{SamplePre}(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}, \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{B}_{\text{aid}} \mathbf{r}, \chi)$, where $\mathbf{r} \leftarrow H(\text{gid}, \mathbf{v})$. By Lemma 6, $\mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}}$ is distributed according to $D_{\mathbb{Z}, \chi}^m$ conditioned on

$$\mathbf{A}_{\text{aid}} \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}} = \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{B}_{\text{aid}} \mathbf{r}.$$

By Lemma 4, it holds with overwhelming probability that $\|\mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}}\| \leq \sqrt{\lambda} \chi$ and $\|\mathbf{r}\| \leq \sqrt{\lambda} \chi'$.

- Using the secret key $\mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}}$, the decryption algorithm computes

$$\mathbf{c}_{1, \text{aid}}^\top \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}} = \mathbf{s}_{\text{aid}}^\top \mathbf{A}_{\text{aid}} \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}} + \mathbf{e}_{1, \text{aid}}^\top \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}} = \mathbf{s}_{\text{aid}}^\top \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{s}_{\text{aid}}^\top \mathbf{B}_{\text{aid}} \mathbf{r} + \mathbf{e}_{1, \text{aid}}^\top \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}}$$

for each $\text{aid} \in A$. Substituting the above expression into the decryption formula, we obtain:

$$\mathbf{c}_3^\top \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{c}_2^\top \mathbf{r} - \sum_{\text{aid} \in A} \mathbf{c}_{1, \text{aid}}^\top \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}} = \mathbf{u}^\top \mathbf{v} + \mathbf{e}_3^\top \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{e}_2^\top \mathbf{r} - \sum_{\text{aid} \in A} \mathbf{e}_{1, \text{aid}}^\top \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}},$$

where the error term is

$$\tilde{e} := \mathbf{e}_3^\top \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{e}_2^\top \mathbf{r} - \sum_{\text{aid} \in A} \mathbf{e}_{1, \text{aid}}^\top \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}}.$$

We conclude that the total error term $\tilde{e}$ satisfies $|\tilde{e}| \leq B_0$, as required.

- To bound $|\tilde{e}|$, we analyze each of its components: By Lemma 4, the following bounds hold with overwhelming probability:

$$\begin{aligned} \|\mathbf{e}_{1, \text{aid}}\| &\leq \sqrt{\lambda} \chi, \quad \text{for each } \text{aid} \in A, \\ \|\mathbf{e}_2\| &\leq \sqrt{\lambda} \chi, \quad \|\mathbf{e}_3\| \leq \sqrt{\lambda} \chi. \end{aligned}$$

Using these bounds, we have

$$\begin{aligned} \|\mathbf{e}_3^\top \mathbf{G}^{-1}(\mathbf{v})\| &\leq m \sqrt{\lambda} \chi, \\ \|\mathbf{e}_2^\top \mathbf{r}\| &\leq m' \lambda \chi \chi', \\ \|\mathbf{e}_{1, \text{aid}}^\top \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}}\| &\leq m \lambda \chi^2. \end{aligned}$$

Combining these results, we obtain the bound for the error term $\tilde{e}$:

$$\|\tilde{e}\| \leq \sqrt{\lambda} \chi m + \lambda \chi \chi' m' + \lambda \chi^2 m L.$$

Thus, the scheme satisfies correctness, as claimed. $\square$

B. Security

Theorem 25. Let $\chi_0(n, q) = \sqrt{n \log q} \cdot \omega(\sqrt{\log n})$ be a polynomial such that Lemma 7 holds. Let Q_0 be the upper bound on the total number of secret-key queries (including those in the partial set) submitted by the adversary. Suppose that the following parameter conditions hold:

- $\chi' = \Omega(\sqrt{n \log q})$.
- Let χ be an error distribution parameter such that $\chi > \chi_0$ and $\chi \geq \lambda^{\omega(1)} \cdot (\sqrt{\lambda}(m + \ell)\chi_s + \lambda m \chi' \chi_s)$ for all $\ell = \text{poly}(\lambda)$, where χ_s is a noise parameter such that $\text{LWE}_{n, m_1, q, \chi_s}$ assumption holds for some $m_1 = \text{poly}(m, m', Q_0)$.
- The assumption $\text{evIPFE}_{n, m, m', q, \chi, \chi}$ holds.

Then Construction 1 is statically secure as a (B_0, χ_s) -evIPFE scheme.

Proof. We prove the static security of Construction 1 by defining a sequence of hybrid experiments. We begin by fixing a public sampling algorithm tuple $\text{Samp} = (\text{Samp}_v, \text{Samp}_u)$ producing pseudorandom noisy inner products with noise parameter χ_s .

Game H_0 . This experiment corresponds to the real static security game, in which the challenger encrypts the plaintext u sampled from Samp_u , exactly as specified in Construction 1. The experiment proceeds as follows.

At the beginning of the experiment, the adversary specifies the following queries:

- A set of corrupt authorities $\mathcal{C} \subseteq \mathcal{AU}$, along with their public keys: $\text{pk}_{\text{aid}} = (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$ for each $\text{aid} \in \mathcal{C}$.
- A set of non-corrupt authorities $\mathcal{N} \subseteq \mathcal{AU}$, satisfying $\mathcal{N} \cap \mathcal{C} = \emptyset$.
- A set of authorities $A^* \subseteq \mathcal{C} \cup \mathcal{N}$, satisfying $(A^* \cap \mathcal{C}) \subsetneq A^*$.
- A set of Type I secret-key queries $\mathcal{Q} = \{(\text{gid}, A, \mathbf{v})\}$, where $A \subseteq \mathcal{N}$ and $(A \cup \mathcal{C}) \cap A^* \subsetneq A^*$.
- A partial set of Type II secret-key queries $\mathcal{Q}'_{\text{par}} = \{(\text{gid}', A')\}$, where $A' \subseteq \mathcal{N}$ and $(A' \cup \mathcal{C}) \cap A^* = A^*$.

To simulate the random oracle, the challenger initializes an empty table $T : \mathcal{GID} \times \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q^{m'}$, which will be used to consistently store and respond to all random oracle queries throughout the experiment. Then the challenger with respect to samplers $\text{Samp} = (\text{Samp}_v, \text{Samp}_u)$ processes the adversary's queries as follows.

- **Public keys for non-corrupt authorities:** For each non-corrupt authority $\text{aid} \in \mathcal{N}$, the challenger samples $(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, $\mathbf{B}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}$, and $\mathbf{P}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$. The public key associated with aid is then set as $\text{pk}_{\text{aid}} \leftarrow (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$.
- **Secret-key queries:** The challenger handles secret-key queries in two types as follows:
 - **Type I:** For each Type I secret-key query $(\text{gid}, A, \mathbf{v}) \in \mathcal{Q}$, the challenger first computes $\mathbf{r}_{\text{gid}, \mathbf{v}} \leftarrow H(\text{gid}, \mathbf{v})$, and samples

$$\mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}} \leftarrow \text{SamplePre}(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}, \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{B}_{\text{aid}} \mathbf{r}_{\text{gid}, \mathbf{v}}, \chi)$$

for each $\text{aid} \in A$. Then it sets the secret key as $\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}} \leftarrow \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}}$.

- **Type II:** The Type II secret-key queries are determined jointly by the partial set $\mathcal{Q}'_{\text{par}}$ and by the vectors $\{\mathbf{v}'_j\}$ output by the sampler Samp_v . The challenger samples public randomness $\mathbf{r}_{\text{pub}} \xleftarrow{\$} \{0, 1\}^{\kappa_1}$, $\mathbf{r}_{\text{pri}} \xleftarrow{\$} \{0, 1\}^{\kappa_2}$, where κ_1, κ_2 are the upper bounds of the random bits used by Samp_v and Samp_u , respectively. Then it samples

$$\mathbf{v}'_1, \dots, \mathbf{v}'_{Q'} \leftarrow \text{Samp}_v(1^\lambda; \mathbf{r}_{\text{pub}}) \text{ and } \mathbf{u} \leftarrow \text{Samp}_u(1^\lambda, \mathbf{r}_{\text{pub}}; \mathbf{r}_{\text{pri}}).$$

The full set of Type II secret-key queries is then given by

$$\mathcal{Q}' = \{(\text{gid}'_1, A'_1, \mathbf{v}'_1), \dots, (\text{gid}'_{Q'}, A'_{Q'}, \mathbf{v}'_{Q'})\}.$$

For each $(\text{gid}', A', \mathbf{v}') \in \mathcal{Q}'$, the challenger computes $\mathbf{r}_{\text{gid}', \mathbf{v}'} \leftarrow H(\text{gid}', \mathbf{v}')$, and samples

$$\mathbf{k}_{\text{aid}, \text{gid}', \mathbf{v}'} \leftarrow \text{SamplePre}(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}, \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}') + \mathbf{B}_{\text{aid}} \mathbf{r}_{\text{gid}', \mathbf{v}'}, \chi)$$

for each $\text{aid} \in A'$. It then sets the secret key as $\text{sk}_{\text{aid}, \text{gid}', \mathbf{v}'} \leftarrow \mathbf{k}_{\text{aid}, \text{gid}', \mathbf{v}'}$.

- The challenger sends the public randomness $\mathbf{r}_{\text{pub}}$ and all the secret keys generated above to the adversary.

- **Challenge ciphertext:** For each $\text{aid} \in A^*$, the challenger samples $s_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^n$ and $\mathbf{e}_{1, \text{aid}} \leftarrow D_{\mathbb{Z}, \chi}^m$. Then, it samples $\mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^{m'}$, $\mathbf{e}_3 \leftarrow D_{\mathbb{Z}, \chi}^m$. Finally, the challenge ciphertext is constructed as $\text{ct} = (\{\mathbf{c}_{1, \text{aid}}^\top\}_{\text{aid} \in A^*}, \mathbf{c}_2^\top, \mathbf{c}_3^\top)$, where

$$\mathbf{c}_{1, \text{aid}}^\top = s_{\text{aid}}^\top \mathbf{A}_{\text{aid}} + \mathbf{e}_{1, \text{aid}}^\top, \mathbf{c}_2^\top = \sum_{\text{aid} \in A^*} s_{\text{aid}}^\top \mathbf{B}_{\text{aid}} + \mathbf{e}_2^\top, \mathbf{c}_3^\top = \sum_{\text{aid} \in A^*} s_{\text{aid}}^\top \mathbf{P}_{\text{aid}} + \mathbf{e}_3^\top + \mathbf{u}^\top \mathbf{G}.$$

- **Random oracle queries:** Upon receiving a query $(\text{gid}, \mathbf{v}) \in \mathcal{GID} \times \mathbb{Z}_q^n$, the challenger checks whether the input $(\text{gid}, \mathbf{v})$ has been queried before—either during the adversary's direct random oracle queries or implicitly through processing the secret-key queries. If so, the challenger retrieves and responds with the stored value $\mathbf{r}_{\text{gid}, \mathbf{v}}$ from the table $\mathbf{T}$. If the input is new, the challenger samples $\mathbf{r}_{\text{gid}, \mathbf{v}} \leftarrow D_{\mathbb{Z}, \chi}^{m'}$, records the mapping $(\text{gid}, \mathbf{v}) \mapsto \mathbf{r}_{\text{gid}, \mathbf{v}}$ in the table $\mathbf{T}$, and then replies with $\mathbf{r}_{\text{gid}, \mathbf{v}}$.

At the end of the experiment, the adversary outputs a bit $b' \in \{0, 1\}$, which is taken as the output of the experiment.

Game H_1 . This experiment is identical to H_0 , except for how the challenger generates the challenge ciphertext.

- For each $\text{aid} \in A^* \cap \mathcal{C}$, the challenger samples $s_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^n$ and $\mathbf{e}_{1, \text{aid}} \leftarrow D_{\mathbb{Z}, \chi}^m$, and computes $\mathbf{c}_{1, \text{aid}} \leftarrow s_{\text{aid}}^\top \mathbf{A}_{\text{aid}} + \mathbf{e}_{1, \text{aid}}^\top$. Then for each $\text{aid}^* \in A^* \cap \mathcal{N}$, the challenger samples $\boxed{\mathbf{c}_{1, \text{aid}^*} \xleftarrow{\$} \mathbb{Z}_q^m}$. It also samples $\boxed{\mathbf{c}_2 \xleftarrow{\$} \mathbb{Z}_q^{m'}, \mathbf{c}_3 \xleftarrow{\$} \mathbb{Z}_q^m}$. Finally, it outputs the challenge ciphertext

$$(\{\mathbf{c}_{1, \text{aid}}^\top\}_{\text{aid} \in A^*}, \mathbf{c}_2^\top, \mathbf{c}_3^\top).$$

Game H_2 . This experiment is identical to H_1 , except for how the challenger generates the third component of the challenge ciphertext. Specifically,

- The challenger samples $\mathbf{u}_\delta \xleftarrow{\$} \mathbb{Z}_q^n$, and computes the challenge ciphertext:

$$(\{\mathbf{c}_{1, \text{aid}}^\top\}_{\text{aid} \in A^*}, \mathbf{c}_2^\top, \boxed{\mathbf{c}_3^\top + \mathbf{u}_\delta^\top \mathbf{G}}).$$

Game H_3 . This experiment corresponds to the real static security game, in which the challenger encrypts the plaintext $\mathbf{u} + \mathbf{u}_\delta \in \mathbb{Z}_q^n$, where $\mathbf{u}$ is sampled from $\text{Samp}_{\mathbf{u}}$ as in H_1 and $\mathbf{u}_\delta \xleftarrow{\$} \mathbb{Z}_q^n$. Specifically, this experiment is identical to H_0 , except that the embedded plaintext is shifted by $\mathbf{u}_\delta$. In particular, the third component of the challenge ciphertext is computed as

$$\mathbf{c}_3^\top = \sum_{\text{aid} \in A^*} s_{\text{aid}}^\top \mathbf{P}_{\text{aid}} + \mathbf{e}_3^\top + (\mathbf{u}^\top + \mathbf{u}_\delta^\top) \mathbf{G}.$$

Game H_4 . This experiment corresponds to the real static security game, in which the challenger encrypts the uniformly random plaintext $\mathbf{u}_\delta$. Specifically, this experiment is identical to H_0 , except that the embedded plaintext is $\mathbf{u}_\delta$. In particular,

$$\mathbf{c}_3^\top = \sum_{\text{aid} \in A^*} s_{\text{aid}}^\top \mathbf{P}_{\text{aid}} + \mathbf{e}_3^\top + \boxed{\mathbf{u}_\delta^\top \mathbf{G}}.$$

Lemma 26. Let Q_0 be the upper bound on the total number of secret-key queries (including those in the partial set Q'_{par}) submitted by the adversary. Suppose that the following parameter conditions hold:

- $m' > 6n \log q$.
- $\chi' = \Omega(\sqrt{n \log q})$.
- Let χ be an error distribution parameter such that $\chi \geq \lambda^{\omega(1)} \cdot (\sqrt{\lambda}(m + \ell)\chi_s + \lambda m \chi' \chi_s)$, where χ_s is a noise parameter such that $\text{LWE}_{n, m_1, q, \chi_s}$ assumption holds for some $m_1 = \text{poly}(m, m', Q_0)$.
- The assumption $\text{evIPFE}_{n, m, m', q, \chi, \chi'}$ holds.

Then we have that $H_0 \stackrel{c}{\approx} H_1$.

Proof. Suppose that there exists an *efficient* adversary $\mathcal{A}$ that distinguishes H_0 from H_1 with non-negligible advantage. Based on adversary $\mathcal{A}$, we define a pair of sampling algorithms $\mathcal{S}_{\mathcal{A}} = (\mathcal{S}_{\mathcal{A}, u}, \mathcal{S}_{\mathcal{A}, v})$ for global parameters $\text{gp} = (1^\lambda, q, 1^n, 1^m, 1^{m'}, 1^\chi, 1^{\chi'})$, with respect to the evIPFE assumption, as follows.

- $\mathcal{S}_{\mathcal{A}, v}(\text{gp}; (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}))$: This algorithm takes as inputs the global parameter gp and public randomness $\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2} \in \{0, 1\}^*$, and proceeds as follows.
 - Run adversary $\mathcal{A}(1^\lambda; \mathbf{r}_{\text{pub}_1})$ with randomness $\mathbf{r}_{\text{pub}_1}$, and extract from its output: a set of Type I secret-key queries $\mathcal{Q} = \{(\text{gid}, A, \mathbf{v})\}$, a partial set of Type II secret-key queries $\mathcal{Q}'_{\text{par}} = \{(\text{gid}'_1, A'_1), \dots, (\text{gid}'_{Q'}, A'_{Q'})\}$, and a set of ciphertext authorities $A^* = \{\text{aid}_1^*, \dots, \text{aid}_\ell^*\}$.
 - Denote $|\mathcal{Q}| = Q$, and index the queries as $(\text{gid}_1, A_1, \mathbf{v}_1), \dots, (\text{gid}_Q, A_Q, \mathbf{v}_Q)$. Then for each $i \in [\ell]$, let $N_i \in [Q]$ denote the number of Type I secret-key queries in which the challenge authority aid_i^* appears. Suppose that authority aid_i^* is contained in the set $A_{j_1}, \dots, A_{j_{N_i}}$ for some indices $j_1, \dots, j_{N_i} \in [Q]$, listed in increasing order. Define the mapping $\rho_i : [N_i] \rightarrow [Q]$ by setting $\rho_i(k) = j_k$. That is, aid_i^* appears exactly in the set $A_{\rho_i(1)}, \dots, A_{\rho_i(N_i)}$ with the indices ordered increasingly.
 - Run the sampler $\text{Samp}_v(1^\lambda; \mathbf{r}_{\text{pub}_2})$ with randomness $\mathbf{r}_{\text{pub}_2}$, which outputs the vectors $\mathbf{v}'_1, \dots, \mathbf{v}'_{Q'}$.
 - For each $i \in [Q]$, sample $\mathbf{r}_i \leftarrow D_{\mathbb{Z}, \chi'}^{m'}$, and for each $j \in [Q']$, sample $\mathbf{r}'_j \leftarrow D_{\mathbb{Z}, \chi'}^{m'}$.
 - Finally, $\mathcal{S}_{\mathcal{A}, v}$ outputs

$$\begin{aligned}
 & 1^\ell, \{1^{N_i+Q'}\}_{i \in [\ell]}; \\
 & (\mathbf{r}_{\rho_1(1)}, \mathbf{v}_{\rho_1(1)}), \dots, (\mathbf{r}_{\rho_1(N_1)}, \mathbf{v}_{\rho_1(N_1)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\
 & (\mathbf{r}_{\rho_2(1)}, \mathbf{v}_{\rho_2(1)}), \dots, (\mathbf{r}_{\rho_2(N_2)}, \mathbf{v}_{\rho_2(N_2)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\
 & \dots; \\
 & (\mathbf{r}_{\rho_\ell(1)}, \mathbf{v}_{\rho_\ell(1)}), \dots, (\mathbf{r}_{\rho_\ell(N_\ell)}, \mathbf{v}_{\rho_\ell(N_\ell)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}).
 \end{aligned}$$

The output is structured as shown above, where each row (excluding the first) corresponds to a challenge authority.

- $\mathcal{S}_{\mathcal{A}, u}(\text{gp}, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}); \mathbf{r}_{\text{pri}})$: takes as input the global parameter gp , public randomness $\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}$ and private randomness $\mathbf{r}_{\text{pri}}$. It computes $\mathbf{u} \leftarrow \text{Samp}_u(1^\lambda, \mathbf{r}_{\text{pub}_2}; \mathbf{r}_{\text{pri}})$ and outputs $\mathbf{u}$.

With respect to the sampling algorithm $\mathcal{S}_{\mathcal{A}}$, We invoke Claim 27 below, and defer its proof to the end of this section.

Claim 27. Let Q_0 be the upper bound on the total number of secret-key queries (including those in the partial set $\mathcal{Q}'_{\text{par}}$) submitted by the adversary. Suppose that the following parameter conditions hold:

- $m' > 6n \log q$.
- $\chi' = \Omega(\sqrt{n \log q})$.
- Let χ be an error distribution parameter such that $\chi \geq \lambda^{\omega(1)} \cdot (\sqrt{\lambda}(m + \ell)\chi_s + \lambda m \chi' \chi_s)$, where χ_s is a noise parameter such that $\text{LWE}_{n, m_1, q, \chi_s}$ assumption holds for some $m_1 = \text{poly}(m, m', Q_0)$.
- The sampling algorithm $\text{Samp} = (\text{Samp}_v, \text{Samp}_u)$ produces pseudorandom noisy inner products with noise parameter χ_s .

Then, for every efficient distinguisher $\mathcal{D}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}^*$, we have $\text{Adv}_{\mathcal{D}, \mathcal{S}_{\mathcal{A}}}^{\text{Pre}}(\lambda) = \text{negl}(\lambda)$, where $\text{Adv}_{\mathcal{D}, \mathcal{S}_{\mathcal{A}}}^{\text{Pre}}$ denotes the advantage of the distinguisher $\mathcal{D}$ (as defined in Assumption IV) in the evIPFE assumption with respect to the sampling algorithm $\mathcal{S}_{\mathcal{A}} = (\mathcal{S}_{\mathcal{A}, v}, \mathcal{S}_{\mathcal{A}, u})$.

Proof of Lemma 26 (Continued). To complete the proof, we prove that if there exists an *efficient* adversary $\mathcal{A}$ that can distinguish between H_0 and H_1 with non-negligible advantage, then we can construct an *efficient* algorithm $\mathcal{B}$ such that $\text{Adv}_{\mathcal{S}_{\mathcal{A},\mathcal{B}}}^{\text{Post}}$ is non-negligible in the evasive IPFE assumption with respect to the sampling algorithm $\mathcal{S}_{\mathcal{A}} = (\mathcal{S}_{\mathcal{A},u}, \mathcal{S}_{\mathcal{A},v})$. The algorithm $\mathcal{B}$ proceeds as follows:

- 1) Algorithm $\mathcal{B}$ begins by receiving an evIPFE challenge

$$(1^\lambda, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}), \{\mathbf{A}_i, \mathbf{y}_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{y}_2^\top, \{\mathbf{K}_i\}_{i \in [\ell]}),$$

where $\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2} \in \{0, 1\}^*$, $\mathbf{A}_i \in \mathbb{Z}_q^{n \times m}$, $\mathbf{y}_{1,i} \in \mathbb{Z}_q^m$, $\mathbf{K}_i \in \mathbb{Z}_q^{m \times (N_i + Q')}$ for each $i \in [\ell]$, $\mathbf{B} \in \mathbb{Z}_q^{n\ell \times m'}$, $\mathbf{P} \in \mathbb{Z}_q^{n\ell \times m}$, $\mathbf{y}_2 \in \mathbb{Z}_q^{m' + m}$.

- 2) For each $i \in [\ell]$, algorithm $\mathcal{B}$ parses the matrix $\mathbf{K}_i$ as

$$\mathbf{K}_i = [\mathbf{K}_i^{(1)} \mid \mathbf{K}_i^{(2)}],$$

where $\mathbf{K}_i^{(1)} \in \mathbb{Z}_q^{m \times N_i}$ and $\mathbf{K}_i^{(2)} \in \mathbb{Z}_q^{m \times Q'}$. Let $\mathbf{k}_{i,j}, \mathbf{k}'_{i,j}$ denote the j -th column vectors of $\mathbf{K}_i^{(1)}$ and $\mathbf{K}_i^{(2)}$, respectively.

- 3) Algorithm $\mathcal{B}$ runs algorithm $\mathcal{A}(1^\lambda; \mathbf{r}_{\text{pub}_1})$, feeding in the randomness $\mathbf{r}_{\text{pub}_1}$. The adversary $\mathcal{A}$ outputs the following queries:
 - A set of corrupt authorities $\mathcal{C} \subseteq \mathcal{AU}$, along with their public keys $\text{pk}_{\text{aid}} = (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$ for all $\text{aid} \in \mathcal{C}$.
 - A set of non-corrupt authorities $\mathcal{N} \subseteq \mathcal{AU}$, satisfying $\mathcal{N} \cap \mathcal{C} = \emptyset$.
 - A ciphertext authority set $A^* \subseteq \mathcal{C} \cup \mathcal{N}$, satisfying $(A^* \cap \mathcal{C}) \subsetneq A^*$.
 - A set of Type I secret-key queries $\mathcal{Q} = \{(\text{gid}, A, \mathbf{v})\}$, where $A \subseteq \mathcal{N}$ and $(A \cup \mathcal{C}) \cap A^* \subsetneq A^*$.
 - A partial set of Type II secret-key queries $\mathcal{Q}'_{\text{par}} = \{(\text{gid}', A')\}$, where $A' \subseteq \mathcal{N}$ and $(A' \cup \mathcal{C}) \cap A^* = A^*$.
- 4) Algorithm $\mathcal{B}$ runs sampling algorithm $\mathcal{S}_{\mathcal{A},v}(\text{gp}; (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}))$, which outputs

$$\begin{aligned} &1^\ell, \{1^{N_i + Q'}\}_{i \in [\ell]}; \\ &(\mathbf{r}_{\rho_1(1)}, \mathbf{v}_{\rho_1(1)}), \dots, (\mathbf{r}_{\rho_1(N_1)}, \mathbf{v}_{\rho_1(N_1)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ &(\mathbf{r}_{\rho_2(1)}, \mathbf{v}_{\rho_2(1)}), \dots, (\mathbf{r}_{\rho_2(N_2)}, \mathbf{v}_{\rho_2(N_2)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ &\dots; \\ &(\mathbf{r}_{\rho_\ell(1)}, \mathbf{v}_{\rho_\ell(1)}), \dots, (\mathbf{r}_{\rho_\ell(N_\ell)}, \mathbf{v}_{\rho_\ell(N_\ell)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}). \end{aligned}$$

The full set of Type II secret-key queries is then given by $\mathcal{Q}' = \{(\text{gid}'_1, A'_1, \mathbf{v}'_1), \dots, (\text{gid}'_{Q'}, A'_{Q'}, \mathbf{v}'_{Q'})\}$. Since the randomness used to simulate $\mathcal{A}$ in Step 3) comes from $\mathbf{r}_{\text{pub}_1}$, the values produced here—namely, the vectors $\mathbf{v}_1, \dots, \mathbf{v}_{Q'}$ —align exactly with those generated by $\mathcal{A}$'s simulation under the public randomness in Step 3). This ensures that algorithm $\mathcal{B}$'s internal simulation of $\mathcal{A}$ is consistent with the actual input instance of the evIPFE challenge.

- 5) From the construction of $\mathcal{S}_{\mathcal{A}}$, we have that $\ell = |A^* \cap \mathcal{N}|$, i.e., the number of non-corrupt authorities appearing in the challenge ciphertext. Let $A^* \cap \mathcal{N} = \{\text{aid}_1^*, \dots, \text{aid}_\ell^*\}$. Algorithm $\mathcal{B}$ first sets $\mathbf{A}_{\text{aid}_i^*} \leftarrow \mathbf{A}_i$ for each $i \in [\ell]$, and parses $[\mathbf{B} \mid \mathbf{P}]$ as

$$[\mathbf{B} \mid \mathbf{P}] = \left[\begin{array}{c|c} \mathbf{B}_{\text{aid}_1^*} & \mathbf{P}_{\text{aid}_1^*} \\ \vdots & \vdots \\ \mathbf{B}_{\text{aid}_\ell^*} & \mathbf{P}_{\text{aid}_\ell^*} \end{array} \right] \in \mathbb{Z}_q^{n\ell \times (m' + m)},$$

where $\mathbf{B}_{\text{aid}_i^*} \in \mathbb{Z}_q^{n \times m'}$ and $\mathbf{P}_{\text{aid}_i^*} \in \mathbb{Z}_q^{n \times m}$ for each $\text{aid}_i^* \in A^* \cap \mathcal{N}$.

- 6) Let $|\mathcal{Q}| = Q$ and $|\mathcal{Q}'| = Q'$. For each $i \in [Q]$, algorithm $\mathcal{B}$ partitions $A_i = A_{i,\text{chal}} \cup \bar{A}_{i,\text{chal}} \subseteq \mathcal{N}$, where $A_{i,\text{chal}}$ consists of the authorities in A_i that appear in the ciphertext, i.e., $A_{i,\text{chal}} = A_i \cap A^*$.
- 7) Algorithm $\mathcal{B}$ initializes an empty table $\mathsf{T} : \mathcal{GID} \times \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q^{m'}$. This table will be used to store and consistently respond to all queries made to the random oracle during the experiment.

8) The algorithm $\mathcal{B}$ responds to the queries as follows:

• **Public keys for non-corrupt authorities:**

- For each $\text{aid}_i^* \in A^* \cap \mathcal{N}$, set $\text{pk}_{\text{aid}_i^*} \leftarrow (\mathbf{A}_{\text{aid}_i^*}, \mathbf{B}_{\text{aid}_i^*}, \mathbf{P}_{\text{aid}_i^*})$.
- For authorities $\text{aid} \in \mathcal{N} \setminus A^*$, algorithm $\mathcal{B}$ samples $(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, $\mathbf{B}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}$, $\mathbf{P}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$ and sets the public key $\text{pk}_{\text{aid}} \leftarrow (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$.

• **Secret keys:** The algorithm $\mathcal{B}$ responds to each secret-key query depending on its type:

- **Type I:** For a Type I secret-key query $(\text{gid}_k, A_k, \mathbf{v}_k)$, recall that A_k is partitioned as $A_k = A_{k,\text{chal}} \cup \bar{A}_{k,\text{chal}}$, where $A_{k,\text{chal}} = A_k \cap A^*$.
 - * For each $\text{aid}_i^* \in A^* \cap \mathcal{N}$, recall that the number of secret-key queries involving aid_i^* is the parameter N_i given in the evIPFE challenge. Let $\rho(\cdot)$ be the index mapping previously defined in the proof of Lemma 26. For each $j \in [N_i]$, set $\text{sk}_{\text{aid}_i^*, \text{gid}_{\rho_i(j)}, \mathbf{v}_{\rho_i(j)}} \leftarrow \mathbf{k}_{i,j}$. Then the algorithm $\mathcal{B}$ checks if the table T has ever recorded the image of $(\text{gid}_{\rho_i(j)}, \mathbf{v}_{\rho_i(j)})$, if not, store the mapping $(\text{gid}_{\rho_i(j)}, \mathbf{v}_{\rho_i(j)}) \mapsto \mathbf{r}_{\rho_i(j)}$ to the table.
 - * For each $k \in [Q]$, if $A_{k,\text{chal}} = \emptyset$, then sample $\mathbf{r}_{\text{gid}_k, \mathbf{v}_k} \leftarrow D_{\mathbb{Z}, \chi}^{m'}$, and add the mapping $(\text{gid}_k, \mathbf{v}_k) \mapsto \mathbf{r}_{\text{gid}_k, \mathbf{v}_k}$ to the table. At this point, the table contains the image of all pairs $(\text{gid}_k, \mathbf{v}_k)$ for each $k \in [Q]$.
 - * For each $k \in [Q]$, for each $\text{aid} \in \bar{A}_{k,\text{chal}}$, compute

$$\text{sk}_{\text{aid}, \text{gid}_k, \mathbf{v}_k} \leftarrow \text{SamplePre}(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}, \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}_k) + \mathbf{B}_{\text{aid}} \mathbf{H}(\text{gid}_k, \mathbf{v}_k)),$$

where the value $\mathbf{H}(\text{gid}_k, \mathbf{v}_k)$ is retrieved from the table T .

- **Type II:** For a Type II query $(\text{gid}'_j, A'_j, \mathbf{v}'_j)$, recall that $A^* \cap \mathcal{N} = \{\text{aid}_1^*, \dots, \text{aid}_\ell^*\} \subseteq A'_j$.
 - * For each $i \in [\ell], j \in [Q']$, set $\text{sk}_{\text{aid}_i^*, \text{gid}'_j, \mathbf{v}'_j} \leftarrow \mathbf{k}'_{i,j} \in \mathbb{Z}_q^m$. Next, algorithm $\mathcal{B}$ adds the mapping $(\text{gid}'_j, \mathbf{v}'_j) \mapsto \mathbf{r}'_j$ to the table T .
 - * For each $\text{aid} \in A'_j \setminus A^*$, algorithm $\mathcal{B}$ computes

$$\text{sk}_{\text{aid}, \text{gid}'_j, \mathbf{v}'_j} \leftarrow \text{SamplePre}(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}, \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}'_j) + \mathbf{B}_{\text{aid}} \mathbf{r}'_j, \chi)$$

efficiently.

- **Challenge ciphertext:** Algorithm $\mathcal{B}$ parses $\mathbf{y}_2$ as $\mathbf{y}_2^\top = [\hat{\mathbf{y}}_2^\top \mid \hat{\mathbf{y}}_3^\top]$ where $\hat{\mathbf{y}}_2 \in \mathbb{Z}_q^{m'}$, $\hat{\mathbf{y}}_3 \in \mathbb{Z}_q^m$. It constructs the ciphertext as follows: For each $\text{aid} \in A^* \cap \mathcal{C}$, sample $\mathbf{s}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^n$ and $\mathbf{e}_{1,\text{aid}} \leftarrow D_{\mathbb{Z}, \chi}^m$, then set $\mathbf{c}_{1,\text{aid}}^\top \leftarrow \mathbf{s}_{\text{aid}}^\top \mathbf{A}_{\text{aid}} + \mathbf{e}_{1,\text{aid}}^\top \in \mathbb{Z}_q^m$. For $\text{aid}_i^* \in A^* \cap \mathcal{N}$, set $\mathbf{c}_{1,\text{aid}_i^*} \leftarrow \mathbf{y}_{1,i}$. Finally, algorithm $\mathcal{B}$ outputs the ciphertext

$$\text{ct} = \left(\{\mathbf{c}_{1,\text{aid}}^\top\}_{\text{aid} \in A^*}, \sum_{\text{aid} \in A^* \cap \mathcal{C}} \mathbf{s}_{\text{aid}}^\top \mathbf{B}_{\text{aid}} + \hat{\mathbf{y}}_2^\top, \sum_{\text{aid} \in A^* \cap \mathcal{C}} \mathbf{s}_{\text{aid}}^\top \mathbf{P}_{\text{aid}} + \hat{\mathbf{y}}_3^\top \right).$$

- **Random oracle queries:** Upon receiving a query $(\text{gid}, \mathbf{v}) \in \mathcal{GID} \times \mathbb{Z}_q^n$, algorithm $\mathcal{B}$ checks whether the input $(\text{gid}, \mathbf{v})$ has been queried before—either during the adversary's direct random oracle queries or implicitly through processing the secret-key queries. If so, algorithm $\mathcal{B}$ retrieves and responds with the stored value $\mathbf{r}_{\text{gid}, \mathbf{v}}$ from the table T . If the input is new, the challenger samples $\mathbf{r}_{\text{gid}, \mathbf{v}} \leftarrow D_{\mathbb{Z}, \chi}^{m'}$, records the mapping $(\text{gid}, \mathbf{v}) \mapsto \mathbf{r}_{\text{gid}, \mathbf{v}}$ in the table T , and then replies with $\mathbf{r}_{\text{gid}, \mathbf{v}}$.

9) Algorithm $\mathcal{B}$ outputs whatever algorithm $\mathcal{A}$ outputs.

The distributions of the public keys for non-corrupt authorities are exactly the same as those in H_0 and H_1 , as they are uniformly generated. We now analyze the responses to the secret-key queries:

- For each $\text{aid}_i^* \in A^* \cap \mathcal{N}$, we have

$$\begin{aligned} \mathbf{k}_{i,j} &\leftarrow (\mathbf{A}_{\text{aid}_i^*})_\chi^{-1} (\mathbf{P}_{\text{aid}_i^*} \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \mathbf{B}_{\text{aid}_i^*} \mathbf{r}_{\rho_i(j)}), \quad \text{for all } j \in [N_i], \\ \mathbf{k}'_{i,j} &\leftarrow (\mathbf{A}_{\text{aid}_i^*})_\chi^{-1} (\mathbf{P}_{\text{aid}_i^*} \mathbf{G}^{-1}(\mathbf{v}'_j) + \mathbf{B}_{\text{aid}_i^*} \mathbf{r}'_j), \quad \text{for all } j \in [Q']. \end{aligned}$$

This exactly matches the distribution of $\text{sk}_{\text{aid}_i^*, \text{gid}_{\rho_i(j)}, \mathbf{v}_{\rho_i(j)}}$ and $\text{sk}_{\text{aid}_i^*, \text{gid}'_j, \mathbf{v}'_j}$ in the actual game, respectively.

- For each $\text{aid} \in A_j \setminus A^*$ for some $j \in [Q]$, the secret key $\text{sk}_{\text{aid}, \text{gid}_j, \mathbf{v}_j}$ is generated using SamplePre , identical to the procedure in H_0 and H_1 . The same argument also applies to $\text{aid} \in A'_j \setminus A^*$ for $j \in [Q']$.

Finally, we analyze the distribution of the challenge ciphertext. We consider the following two cases:

- If for each $i \in [\ell]$, $\mathbf{y}_{1,i}^\top = \mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top$ and $\mathbf{y}_2^\top = \mathbf{s}^\top [\mathbf{B} \mid \mathbf{P}] + \mathbf{e}_2^\top + [\mathbf{0}_{m'}^\top \mid \mathbf{u}^\top \mathbf{G}]$ for some $\mathbf{s}^\top = [\mathbf{s}_1^\top \mid \dots \mid \mathbf{s}_\ell^\top] \xleftarrow{\$} \mathbb{Z}_q^{n\ell}$, $\mathbf{e}_{1,i} \leftarrow D_{\mathbb{Z}, \chi}^m$, $\mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^{m'+m}$. We parse $\mathbf{e}_2^\top$ into two components as $[\hat{\mathbf{e}}_2^\top \mid \hat{\mathbf{e}}_3^\top]$ where $\hat{\mathbf{e}}_2 \in \mathbb{Z}_q^{m'}$, $\hat{\mathbf{e}}_3 \in \mathbb{Z}_q^m$. Then

$$\begin{aligned} \mathbf{c}_{1, \text{aid}_i^*}^\top &= \mathbf{y}_{1,i}^\top = \mathbf{s}_i^\top \mathbf{A}_{\text{aid}_i^*} + \mathbf{e}_{1,i}^\top, & \text{for each } \text{aid}_i^* \in A^* \cap \mathcal{N}, \\ \hat{\mathbf{y}}_2^\top &= \sum_{\text{aid}_i^* \in A^* \cap \mathcal{N}} \mathbf{s}_i^\top \mathbf{B}_{\text{aid}_i^*} + \hat{\mathbf{e}}_2^\top, \\ \hat{\mathbf{y}}_3^\top &= \sum_{\text{aid}_i^* \in A^* \cap \mathcal{N}} \mathbf{s}_i^\top \mathbf{P}_{\text{aid}_i^*} + \hat{\mathbf{e}}_3^\top + \mathbf{u}^\top \mathbf{G}. \end{aligned}$$

Then the ciphertext is constructed as

$$\begin{aligned} \mathbf{c}_{1, \text{aid}_i^*}^\top &= \mathbf{s}_i^\top \mathbf{A}_{\text{aid}_i^*} + \mathbf{e}_{1,i}^\top, & \text{for each } \text{aid}_i^* \in A^* \cap \mathcal{N}, \\ \mathbf{c}_{1, \text{aid}}^\top &= \mathbf{s}_{\text{aid}}^\top \mathbf{A}_{\text{aid}} + \mathbf{e}_{1, \text{aid}}^\top, & \text{for each } \text{aid} \in A^* \cap \mathcal{C}, \\ \mathbf{c}_2^\top &= \sum_{\text{aid} \in A^* \cap \mathcal{C}} \mathbf{s}_{\text{aid}}^\top \mathbf{B}_{\text{aid}} + \sum_{\text{aid}_i^* \in A^* \cap \mathcal{N}} \mathbf{s}_i^\top \mathbf{B}_{\text{aid}_i^*} + \hat{\mathbf{e}}_2^\top, \\ \mathbf{c}_3^\top &= \sum_{\text{aid} \in A^* \cap \mathcal{C}} \mathbf{s}_{\text{aid}}^\top \mathbf{P}_{\text{aid}} + \sum_{\text{aid}_i^* \in A^* \cap \mathcal{N}} \mathbf{s}_i^\top \mathbf{P}_{\text{aid}_i^*} + \hat{\mathbf{e}}_3^\top + \mathbf{u}^\top \mathbf{G}, \end{aligned}$$

where $\mathbf{s}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^n$ and $\mathbf{e}_{1, \text{aid}} \leftarrow D_{\mathbb{Z}, \chi}^m$ for each $\text{aid} \in A^* \cap \mathcal{C}$. Since all randomness is sampled exactly as in H_0 , the resulting ciphertext generated by $\mathcal{B}$ is identically distributed to that in the experiment H_0 .

- If for each $i \in [\ell]$, $\mathbf{y}_{1,i} \xleftarrow{\$} \mathbb{Z}_q^m$, and $\mathbf{y}_2 \xleftarrow{\$} \mathbb{Z}_q^{m'+m}$. Then from the construction, the ciphertext is constructed as

$$\begin{aligned} \mathbf{c}_{1, \text{aid}_i^*}^\top &= \mathbf{y}_{1,i}^\top, & \text{for each } \text{aid}_i^* \in A^* \cap \mathcal{N}, \\ \mathbf{c}_{1, \text{aid}}^\top &= \mathbf{s}_{\text{aid}}^\top \mathbf{A}_{\text{aid}} + \mathbf{e}_{1, \text{aid}}^\top, & \text{for each } \text{aid} \in A^* \cap \mathcal{C}, \\ \mathbf{c}_2^\top &= \sum_{\text{aid} \in A^* \cap \mathcal{C}} \mathbf{s}_{\text{aid}}^\top \mathbf{B}_{\text{aid}} + \hat{\mathbf{y}}_2^\top, \\ \mathbf{c}_3^\top &= \sum_{\text{aid} \in A^* \cap \mathcal{C}} \mathbf{s}_{\text{aid}}^\top \mathbf{P}_{\text{aid}} + \hat{\mathbf{y}}_3^\top, \end{aligned}$$

where $\mathbf{s}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^n$ and $\mathbf{e}_{1, \text{aid}} \leftarrow D_{\mathbb{Z}, \chi}^m$ for each $\text{aid} \in A^* \cap \mathcal{C}$. Since $\{\mathbf{y}_{1,i}\}_{i \in [\ell]}, \hat{\mathbf{y}}_2, \hat{\mathbf{y}}_3$ are all uniform and independent of each other, the overall distribution of $\{\mathbf{c}_{1, \text{aid}}\}_{\text{aid} \in A^* \cap \mathcal{N}}, \mathbf{c}_2, \mathbf{c}_3$ is also uniform, which matches the distribution in experiment H_1 .

In either case, the algorithm $\mathcal{B}$ constructed perfectly simulates experiments H_0 and H_1 . The advantage $\text{Adv}_{\mathcal{S}_{\mathcal{A}}, \mathcal{B}}^{\text{Post}}$ in the evIPFE assumption with respect to the sampling algorithm $\mathcal{S}_{\mathcal{A}}$ is the same as the non-negligible advantage of $\mathcal{A}$ distinguishing between H_0 and H_1 . By the definition of the evIPFE assumption, the existence of such an efficient post-challenge adversary $\mathcal{B}$ with non-negligible advantage would imply the existence of another efficient adversary $\mathcal{B}'$ breaking the pre-challenge security, i.e., $\text{Adv}_{\mathcal{S}_{\mathcal{A}}, \mathcal{B}'}^{\text{Pre}}$ is non-negligible, contradicting Claim 27. Hence, under the evIPFE assumption, no efficient adversary can distinguish between H_0 and H_1 with non-negligible advantage. This completes the proof. $\square$

Lemma 28. *We have that $H_1 \equiv H_2$.*

Proof. The proof follows directly from the fact that in H_2 , the third component of the ciphertext is generated as $\mathbf{c}_3^\top + \mathbf{u}_\delta^\top \mathbf{G}$, where $\mathbf{u}_\delta$ is sampled uniformly from $\mathbb{Z}_q^n$ and is independent of all other components. The resulting component remains uniform and thus identical to that in H_1 . $\square$

Lemma 29. *Under the same assumptions as in Lemma 26, we have that $H_2 \stackrel{c}{\approx} H_3$.*

Proof. The proof follows essentially the same argument as in Lemma 26. $\square$

Lemma 30. *We have that $H_3 \equiv H_4$.*

Proof. The result follows directly from the fact that both $\mathbf{u} + \mathbf{u}_\delta$ in H_3 and $\mathbf{u}_\delta$ in H_4 are uniformly distributed over $\mathbb{Z}_q^n$, as $\mathbf{u}_\delta$ is sampled uniformly and independently of $\mathbf{u}$. Hence, $\mathbf{c}_3^\top$ is identically distributed in both experiments. $\square$

Proof of Theorem 25 (Continued). By Lemmas 26, 28, 29, and 30, and a standard hybrid argument, we conclude that under the given parameter constraints, $H_0 \stackrel{c}{\approx} H_4$. This implies that ciphertexts encrypting the vector $\mathbf{u}$ generated by Samp_v are computationally indistinguishable from those encrypting a uniformly random vector $\mathbf{u}_\delta$. Therefore, the theorem follows. $\square$

To complete the proof of Theorem 25, it remains to establish the validity of Claim 27.

Proof of Claim 27. We prove the claim by defining a sequence of hybrid experiments.

Game H_0^{Pre} . On input the security parameter λ , the challenger proceeds as follows:

- 1) Let $\kappa = \kappa(\lambda)$ be an upper bound on the number of random bits used by the adversary $\mathcal{A}$, and the sampling algorithms $\text{Samp}_u, \text{Samp}_v$. The challenger samples $\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}, \mathbf{r}_{\text{pri}} \xleftarrow{\$} \{0, 1\}^\kappa$ and run the sampling algorithms $\mathcal{S}_A = (\mathcal{S}_{A,v}, \mathcal{S}_{A,u})$ as follows.

- Run $\mathcal{S}_{A,v}(\text{gp}; (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}))$, which outputs

$$\begin{aligned} & 1^\ell, \{1^{N_i+Q'}\}_{i \in [\ell]}; \\ & (\mathbf{r}_{\rho_1(1)}, \mathbf{v}_{\rho_1(1)}), \dots, (\mathbf{r}_{\rho_1(N_1)}, \mathbf{v}_{\rho_1(N_1)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ & (\mathbf{r}_{\rho_2(1)}, \mathbf{v}_{\rho_2(1)}), \dots, (\mathbf{r}_{\rho_2(N_2)}, \mathbf{v}_{\rho_2(N_2)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ & \dots; \\ & (\mathbf{r}_{\rho_\ell(1)}, \mathbf{v}_{\rho_\ell(1)}), \dots, (\mathbf{r}_{\rho_\ell(N_\ell)}, \mathbf{v}_{\rho_\ell(N_\ell)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}). \end{aligned}$$

- Run $\mathbf{u} \leftarrow \mathcal{S}_{A,u}(\text{gp}, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}); \mathbf{r}_{\text{pri}})$.

- 2) Sample $\mathbf{B} \xleftarrow{\$} \mathbb{Z}_q^{n\ell \times m'}$, $\mathbf{P} \xleftarrow{\$} \mathbb{Z}_q^{n\ell \times m}$, and parse the matrices

$$\mathbf{B} = \begin{bmatrix} \mathbf{B}_1 \\ \vdots \\ \mathbf{B}_\ell \end{bmatrix} \in \mathbb{Z}_q^{n\ell \times m'}, \mathbf{P} = \begin{bmatrix} \mathbf{P}_1 \\ \vdots \\ \mathbf{P}_\ell \end{bmatrix} \in \mathbb{Z}_q^{n\ell \times m},$$

with $\mathbf{B}_i \in \mathbb{Z}_q^{n \times m'}$, $\mathbf{P}_i \in \mathbb{Z}_q^{n \times m}$ for all $i \in [\ell]$.

- 3) For each $i \in [\ell]$, define the matrix:

$$\mathbf{Q}_i^{(1)} \leftarrow [\mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(1)}) + \mathbf{B}_i \mathbf{r}_{\rho_i(1)} \mid \dots \mid \mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(N_i)}) + \mathbf{B}_i \mathbf{r}_{\rho_i(N_i)}] \in \mathbb{Z}_q^{n \times N_i},$$

and similarly define:

$$\mathbf{Q}_i^{(2)} = [\mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}'_1) + \mathbf{B}_i \mathbf{r}'_1 \mid \dots \mid \mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}'_{Q'}) + \mathbf{B}_i \mathbf{r}'_{Q'}] \in \mathbb{Z}_q^{n \times Q'}.$$

Finally, set $\mathbf{Q}_i = [\mathbf{Q}_i^{(1)} \mid \mathbf{Q}_i^{(2)}] \in \mathbb{Z}_q^{n \times (N_i + Q')}$. The construction of each matrix $\mathbf{Q}_i$ follows exactly the format specified in Assumption 3, ensuring consistency with the evLPFE input distribution.

- 4) Then the challenger samples $\mathbf{s}_1, \dots, \mathbf{s}_\ell \xleftarrow{\$} \mathbb{Z}_q^n$ and sets $\mathbf{s}^\top = [\mathbf{s}_1^\top \mid \dots \mid \mathbf{s}_\ell^\top] \in \mathbb{Z}_q^{n\ell}$. For each $i \in [\ell]$, it samples $\mathbf{e}_{1,i} \leftarrow D_{\mathbb{Z}, \chi}^m$, $\mathbf{e}_{3,i} \leftarrow D_{\mathbb{Z}, \chi}^{N_i+Q'}$. Then it samples $\mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^{m'+m}$.
- 5) The challenger samples $(\mathbf{A}_1, \text{td}_1), \dots, (\mathbf{A}_\ell, \text{td}_\ell) \leftarrow \text{TrapGen}(1^n, 1^m, q)$. For each $i \in [\ell]$, it computes the following values :
 - $\mathbf{z}_{1,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top \in \mathbb{Z}_q^m$ for each $i \in [\ell]$.
 - $\mathbf{z}_2^\top = \mathbf{s}^\top [\mathbf{B} \mid \mathbf{P}] + \mathbf{e}_2^\top + [\mathbf{0}_{m'}^\top \mid \mathbf{u}^\top \mathbf{G}] = \left[\sum_{i \in [\ell]} \mathbf{s}_i^\top \mathbf{B}_i \mid \sum_{i \in [\ell]} \mathbf{s}_i^\top \mathbf{P}_i + \mathbf{u}^\top \mathbf{G} \right] + \mathbf{e}_2^\top \in \mathbb{Z}_q^{m'+m}$.
 - $\mathbf{z}_{3,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{Q}_i + \mathbf{e}_{3,i}^\top \in \mathbb{Z}_q^{N_i+Q'}$ for each $i \in [\ell]$.

The component $\mathbf{z}_{3,i}$ and $\mathbf{e}_{3,i}$ are parsed as

$$\mathbf{z}_{3,i}^\top = [\mathbf{z}_{3,i}^{(1)\top} \mid \mathbf{z}_{3,i}^{(2)\top}] \in \mathbb{Z}_q^{N_i+Q'}, \quad \mathbf{e}_{3,i}^\top = [\mathbf{e}_{3,i}^{(1)\top} \mid \mathbf{e}_{3,i}^{(2)\top}] \in \mathbb{Z}_q^{N_i+Q'},$$

where $\mathbf{z}_{3,i}^{(1)}, \mathbf{e}_{3,i}^{(1)} \in \mathbb{Z}_q^{N_i}$ and $\mathbf{z}_{3,i}^{(2)}, \mathbf{e}_{3,i}^{(2)} \in \mathbb{Z}_q^{Q'}$. For clarity, define $\mathbf{t}_{i,j} = \mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \mathbf{B}_i \mathbf{r}_{\rho_i(j)} \in \mathbb{Z}_q^n$ and $y_{i,j} = \mathbf{s}_i^\top \mathbf{t}_{i,j} \in \mathbb{Z}_q$ for each $i \in [\ell]$ and $j \in [N_i]$. Similarly define $\mathbf{t}'_{i,j} = \mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}'_j) + \mathbf{B}_i \mathbf{r}'_j \in \mathbb{Z}_q^n$ and $y'_{i,j} = \mathbf{s}_i^\top \mathbf{t}'_{i,j} \in \mathbb{Z}_q$ for each $i \in [\ell]$ and $j \in [Q']$.

In summary, we obtain

$$\begin{aligned} \mathbf{z}_{3,i}^{(1)\top} &= \mathbf{s}_i^\top \mathbf{Q}_i^{(1)} + \mathbf{e}_{3,i}^{(1)\top} = [\mathbf{s}_i^\top \mathbf{t}_{i,1} \mid \dots \mid \mathbf{s}_i^\top \mathbf{t}_{i,N_i}] + \mathbf{e}_{3,i}^{(1)\top} = [y_{i,1} \mid \dots \mid y_{i,N_i}] + \mathbf{e}_{3,i}^{(1)\top} \in \mathbb{Z}_q^{N_i}, \\ \mathbf{z}_{3,i}^{(2)\top} &= \mathbf{s}_i^\top \mathbf{Q}_i^{(2)} + \mathbf{e}_{3,i}^{(2)\top} = [\mathbf{s}_i^\top \mathbf{t}'_{i,1} \mid \dots \mid \mathbf{s}_i^\top \mathbf{t}'_{i,Q'}] + \mathbf{e}_{3,i}^{(2)\top} = [y'_{i,1} \mid \dots \mid y'_{i,Q'}] + \mathbf{e}_{3,i}^{(2)\top} \in \mathbb{Z}_q^{Q'}. \end{aligned}$$

- 6) The challenger outputs the tuple $(1^\lambda, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}), \{(\mathbf{A}_i, \mathbf{z}_{1,i}^\top)\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top, \{\mathbf{z}_{3,i}^\top\}_{i \in [\ell]})$ and sends it to the distinguisher $\mathcal{D}$.
- 7) The distinguisher $\mathcal{D}$ outputs a bit $\hat{b} \in \{0, 1\}$, which is taken as the output of the experiment.

Game H_1^{Pre} . The experiment is identical to H_0^{Pre} , except for the procedure how the challenger samples $\mathbf{z}_{1,i}, \mathbf{z}_2, \mathbf{z}_{3,i}$, which are now smudged with additional noise.

- $\mathbf{z}_{1,i}$: For each $i \in [\ell]$, sample $\tilde{\mathbf{e}}_{1,i} \leftarrow D_{\mathbb{Z}, \chi_s}^m$, and set $\mathbf{z}_{1,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{A}_i + \tilde{\mathbf{e}}_{1,i}^\top + \mathbf{e}_{1,i}^\top$.
- $\mathbf{z}_2$: For each $i \in [\ell]$, sample $\tilde{\mathbf{e}}_{2,i} \leftarrow D_{\mathbb{Z}, \chi_s}^{m'}$ and $\tilde{\mathbf{e}}'_{2,i} \leftarrow D_{\mathbb{Z}, \chi_s}^m$, and compute $\tilde{\mathbf{z}}_{2,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{B}_i + \tilde{\mathbf{e}}_{2,i}^\top$ and $\tilde{\mathbf{z}}'_{2,i} \leftarrow \mathbf{s}_i^\top \mathbf{P}_i + \tilde{\mathbf{e}}'_{2,i}^\top$. Then set

$$\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{\mathbf{z}}'_{2,i}^\top + \mathbf{u}^\top \mathbf{G} \right] + \mathbf{e}_2^\top = \left[\sum_{i \in [\ell]} (\mathbf{s}_i^\top \mathbf{B}_i + \tilde{\mathbf{e}}_{2,i}^\top) \mid \sum_{i \in [\ell]} (\mathbf{s}_i^\top \mathbf{P}_i + \tilde{\mathbf{e}}'_{2,i}^\top) + \mathbf{u}^\top \mathbf{G} \right] + \mathbf{e}_2^\top.$$

- $\mathbf{z}_{3,i}^{(1)}$: For each $i \in [\ell]$ and $j \in [N_i]$, compute

$$\begin{aligned} y_{i,j} &\leftarrow (\mathbf{s}_i^\top \mathbf{P}_i + \tilde{\mathbf{e}}'_{2,i}^\top) \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + (\mathbf{s}_i^\top \mathbf{B}_i + \tilde{\mathbf{e}}_{2,i}^\top) \mathbf{r}_{\rho_i(j)} \\ &= \tilde{\mathbf{z}}'_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}_{\rho_i(j)}, \end{aligned}$$

and set

$$\mathbf{z}_{3,i}^{(1)\top} \leftarrow [y_{i,1} \mid \dots \mid y_{i,N_i}] + \mathbf{e}_{3,i}^{(1)\top} \in \mathbb{Z}_q^{N_i}.$$

- $\mathbf{z}_{3,i}^{(2)}$: For each $i \in [\ell]$ and $j \in [Q']$, compute

$$\begin{aligned} y'_{i,j} &\leftarrow (\mathbf{s}_i^\top \mathbf{P}_i + \tilde{\mathbf{e}}'_{2,i}^\top) \mathbf{G}^{-1}(\mathbf{v}'_j) + (\mathbf{s}_i^\top \mathbf{B}_i + \tilde{\mathbf{e}}_{2,i}^\top) \mathbf{r}'_j \\ &= \tilde{\mathbf{z}}'_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}'_j, \end{aligned}$$

and set

$$\mathbf{z}_{3,i}^{(2)\top} \leftarrow [y'_{i,1} \mid \cdots \mid y'_{i,Q'}] + \mathbf{e}_{3,i}^{(2)\top} \in \mathbb{Z}_q^{Q'}.$$

Game H_2^{Pre} . The experiment is identical to H_1^{Pre} , except for the procedure how the values $\mathbf{z}_{1,i}, \mathbf{z}_2, \mathbf{z}_{3,i}$ are sampled.

- $\mathbf{z}_{1,i}$: For each $i \in [\ell]$, sample $\boxed{\mathbf{z}_{1,i} \xleftarrow{\$} \mathbb{Z}_q^m}$.
- $\mathbf{z}_2$: For each $i \in [\ell]$, sample $\boxed{\tilde{\mathbf{z}}_{2,i} \xleftarrow{\$} \mathbb{Z}_q^{m'}}$ and $\boxed{\tilde{\mathbf{z}}'_{2,i} \xleftarrow{\$} \mathbb{Z}_q^m}$. Then set

$$\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{\mathbf{z}}'_{2,i}^\top + \mathbf{u}^\top \mathbf{G} \right] + \mathbf{e}_2^\top.$$

- $\mathbf{z}_{3,i}^{(1)}$: For each $i \in [\ell]$ and $j \in [N_i]$, compute

$$y_{i,j} \leftarrow \tilde{\mathbf{z}}_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}_{\rho_i(j)},$$

then set

$$\mathbf{z}_{3,i}^{(1)\top} \leftarrow [y_{i,1} \mid \cdots \mid y_{i,N_i}] + \mathbf{e}_{3,i}^{(1)\top} \in \mathbb{Z}_q^{N_i}.$$

- $\mathbf{z}_{3,i}^{(2)}$: For each $i \in [\ell]$ and $j \in [Q']$, compute

$$y'_{i,j} \leftarrow \tilde{\mathbf{z}}_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}'_j,$$

then set

$$\mathbf{z}_{3,i}^{(2)\top} \leftarrow [y'_{i,1} \mid \cdots \mid y'_{i,Q'}] + \mathbf{e}_{3,i}^{(2)\top} \in \mathbb{Z}_q^{Q'}.$$

Game H_3^{Pre} . The experiment is identical to H_2^{Pre} , except it samples $\boxed{\mathbf{z}_{3,1}^{(1)} \xleftarrow{\$} \mathbb{Z}_q^{N_1}}$.

Game H_4^{Pre} . The experiment is identical to H_3^{Pre} , except for how the components $\mathbf{z}_2$ and $\mathbf{z}_{3,1}^{(2)}$ are sampled.

- $\mathbf{z}_2$: For each $i \in [\ell]$, sample $\tilde{\mathbf{z}}_{2,i} \xleftarrow{\$} \mathbb{Z}_q^{m'}$, $\tilde{\mathbf{z}}'_{2,i} \xleftarrow{\$} \mathbb{Z}_q^m$. Then set

$$\boxed{\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{\mathbf{z}}'_{2,i}^\top \right] + \mathbf{e}_2^\top},$$

which omits the additive term $[\mathbf{0}_{m'} \mid \mathbf{u}^\top \mathbf{G}]$ compared to the construction in H_3^{Pre} .

- $\mathbf{z}_{3,i}^{(2)}$: For $i = 1$, sample $\boxed{\mathbf{z}_{3,1}^{(2)} \xleftarrow{\$} \mathbb{Z}_q^{Q'}}$. For $i \neq 1$, sample $\mathbf{z}_{3,i}^{(2)}$ in the same manner as in H_3^{Pre} .

Game H_5^{Pre} . The experiment is identical to H_4^{Pre} , except it samples $\mathbf{z}_2 \xleftarrow{\$} \mathbb{Z}_q^{m'+m}$, $\mathbf{z}_{3,i}^{(1)} \xleftarrow{\$} \mathbb{Z}_q^{N_i}$, $\mathbf{z}_{3,i}^{(2)} \xleftarrow{\$} \mathbb{Z}_q^{Q'}$ for each $i \in [\ell]$.

Lemma 31. Suppose that $\chi \geq \lambda^{\omega(1)} \cdot (\sqrt{\lambda}(m + \ell)\chi_s + \lambda m' \chi' \chi_s)$. Then we have $H_0^{\text{Pre}} \stackrel{s}{\approx} H_1^{\text{Pre}}$.

Proof. The only difference between H_0^{Pre} and H_1^{Pre} lies in the way the error terms in $\mathbf{z}_{1,i}, \mathbf{z}_2, \mathbf{z}_{3,i}$ are generated. We analyze each of these components individually below:

- **Error term in $\mathbf{z}_{1,i}$:** In experiment H_1^{Pre} , the error term in $\mathbf{z}_{1,i}$ is given by $\mathbf{e}_{1,i} + \tilde{\mathbf{e}}_{1,i}$, where $\mathbf{e}_{1,i} \leftarrow D_{\mathbb{Z}, \chi}^m$ and $\tilde{\mathbf{e}}_{1,i} \leftarrow D_{\mathbb{Z}, \chi_s}^m$. By Lemma 4, we have $\|\tilde{\mathbf{e}}_{1,i}\| \leq \sqrt{\lambda} \chi_s$ with overwhelming probability. Given that $\chi > \lambda^{\omega(1)} \cdot \sqrt{\lambda} \chi_s$, Lemma 5 implies that $\mathbf{e}_{1,i} + \tilde{\mathbf{e}}_{1,i} \stackrel{s}{\approx} \mathbf{e}_{1,i}$.

- **Error term in $\mathbf{z}_2$:** In H_1^{Pre} , the error term in $\mathbf{z}_2$ is given by

$$\mathbf{e}_2^\top + \tilde{\mathbf{e}}_2^\top := \mathbf{e}_2^\top + \left[\sum_{i \in [\ell]} \tilde{\mathbf{e}}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{\mathbf{e}}_{2,i}'^\top \right],$$

where $\tilde{\mathbf{e}}_{2,i} \leftarrow D_{\mathbb{Z}, \chi_s}^{m'}$ and $\tilde{\mathbf{e}}_{2,i}' \leftarrow D_{\mathbb{Z}, \chi_s}^m$. Lemma 4 implies that $\|\tilde{\mathbf{e}}_2\| \leq \ell \cdot \sqrt{\lambda} \chi_s$ with overwhelming probability. Given that $\chi > \lambda^{\omega(1)} \cdot \ell \sqrt{\lambda} \chi_s$, Lemma 5 ensures that $\mathbf{e}_2 + \tilde{\mathbf{e}}_2 \stackrel{s}{\approx} \mathbf{e}_2$. Hence, the distributions of $\mathbf{z}_2$ in H_0^{Pre} and H_1^{Pre} are statistically indistinguishable.

- **Error term in $\mathbf{z}_{3,i}$:** In experiment H_1^{Pre} , the error term of $y_{i,j}$ is given by

$$\tilde{\mathbf{e}}_{2,i}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \tilde{\mathbf{e}}_{2,i}^\top \mathbf{r}_{\rho_i(j)}.$$

By Lemma 4, the infinity norm of this term is at most $\sqrt{\lambda} m \chi_s + \lambda m' \chi' \chi_s$ with overwhelming probability. Since $\chi > \lambda^{\omega(1)} \cdot (\sqrt{\lambda} m \chi_s + \lambda m' \chi' \chi_s)$, Lemma 5 implies that the distributions of $\mathbf{z}_{3,i}^{(1)}$ in the two experiments are statistically indistinguishable. The same argument applies to the component $\mathbf{z}_{3,i}^{(2)}$ as well.

In conclusion, under the constraints of the stated parameters, all error terms in $\mathbf{z}_{1,i}, \mathbf{z}_2, \mathbf{z}_{3,i}$ are statistically close in the two experiments. Hence, we have $H_0^{\text{Pre}} \stackrel{s}{\approx} H_1^{\text{Pre}}$, as claimed. $\square$

Lemma 32. *Suppose that the assumption $\text{LWE}_{n,m_1,q,\chi_s}$ holds for $m_1 = \text{poly}(m, m')$. Then we have $H_1^{\text{Pre}} \stackrel{c}{\approx} H_2^{\text{Pre}}$.*

Proof. We prove this lemma by defining a sequence of intermediate hybrid experiments $H_{1,d}^{\text{Pre}}$ for each $0 \leq d \leq \ell$.

Game $H_{1,d}^{\text{Pre}}$. This experiment is identical to H_1^{Pre} , except for how the value $\mathbf{z}_{1,i}, \mathbf{z}_2, \mathbf{z}_{3,i}$ are generated:

- $\mathbf{z}_{1,i}$:
 - For each $i \leq d$, sample $\mathbf{z}_{1,i} \xleftarrow{\$} \mathbb{Z}_q^m$.
 - For each $i > d$, sample $\mathbf{e}_{1,i} \leftarrow D_{\mathbb{Z}, \chi}^m$, $\tilde{\mathbf{e}}_{1,i} \leftarrow D_{\mathbb{Z}, \chi_s}^m$, and set $\mathbf{z}_{1,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{A}_i + \tilde{\mathbf{e}}_{1,i}^\top + \mathbf{e}_{1,i}^\top$.
- $\mathbf{z}_2$:
 - For each $i \leq d$, sample $\tilde{\mathbf{z}}_{2,i} \xleftarrow{\$} \mathbb{Z}_q^{m'}$ and $\tilde{\mathbf{z}}_{2,i}' \xleftarrow{\$} \mathbb{Z}_q^m$.
 - For each $i > d$, sample $\tilde{\mathbf{e}}_{2,i} \leftarrow D_{\mathbb{Z}, \chi_s}^{m'}$, $\tilde{\mathbf{e}}_{2,i}' \leftarrow D_{\mathbb{Z}, \chi_s}^m$, and set $\tilde{\mathbf{z}}_{2,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{B}_i + \tilde{\mathbf{e}}_{2,i}^\top$ and $\tilde{\mathbf{z}}_{2,i}'^\top \leftarrow \mathbf{s}_i^\top \mathbf{P}_i + \tilde{\mathbf{e}}_{2,i}'^\top$.
 - Then sample $\mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^{m'+m}$, and set

$$\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}'^\top + \mathbf{u}^\top \mathbf{G} \right] + \mathbf{e}_2^\top.$$

- $\mathbf{z}_{3,i}^{(1)}$: For each $i \in [\ell]$, sample $\mathbf{e}_{3,i}^{(1)} \leftarrow D_{\mathbb{Z}, \chi}^{N_i}$. Then for each $j \in [N_i]$, compute

$$y_{i,j} \leftarrow \tilde{\mathbf{z}}_{2,i}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}_{\rho_i(j)},$$

and let

$$\mathbf{z}_{3,i}^{(1)\top} \leftarrow [y_{i,1} \mid \cdots \mid y_{i,N_i}] + \mathbf{e}_{3,i}^{(1)\top} \in \mathbb{Z}_q^{N_i}.$$

- $\mathbf{z}_{3,i}^{(2)}$: Similarly, for each $i \in [\ell]$, sample $\mathbf{e}_{3,i}^{(2)} \leftarrow D_{\mathbb{Z}, \chi}^{Q'}$. Then for $j \in [Q']$, compute

$$y'_{i,j} \leftarrow \tilde{\mathbf{z}}_{2,i}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}'_j,$$

and let

$$\mathbf{z}_{3,i}^{(2)\top} \leftarrow [y'_{i,1} \mid \cdots \mid y'_{i,Q'}] + \mathbf{e}_{3,i}^{(2)\top} \in \mathbb{Z}_q^{Q'}.$$

It follows naturally by the construction that $H_{1,0}^{\text{Pre}} \equiv H_1^{\text{Pre}}$, and $H_{1,\ell}^{\text{Pre}} \equiv H_2^{\text{Pre}}$. We now demonstrate that, for each $d \in [\ell]$, the hybrids $H_{1,d-1}^{\text{Pre}}$ and $H_{1,d}^{\text{Pre}}$ are computationally indistinguishable under the $\text{LWE}_{n,m_1,q,\chi_s}$ assumption for $m_1 = \text{poly}(m, m')$.

Suppose, for contradiction, that there exists an efficient distinguisher $\mathcal{D}$ that can distinguish between $H_{1,d-1}^{\text{Pre}}$ and $H_{1,d}^{\text{Pre}}$ with non-negligible advantage. We use $\mathcal{D}$ to construct an adversary $\mathcal{D}'$ that breaks the LWE assumption. The adversary $\mathcal{D}'$ proceeds as follows:

- 1) Algorithm $\mathcal{D}'$ begins by receiving an LWE challenge $(\mathbf{D}, \mathbf{y})$, where $\mathbf{D} \in \mathbb{Z}_q^{n \times (2m+m')}$, $\mathbf{y} \in \mathbb{Z}_q^{2m+m'}$. Then it parses $\mathbf{D}$ and $\mathbf{y}$ as

$$\mathbf{D} = [\mathbf{A}_d \mid \mathbf{B}_d \mid \mathbf{P}_d], \quad \mathbf{y}^\top = [\mathbf{y}_1^\top \mid \mathbf{y}_2^\top \mid \mathbf{y}_2'^\top],$$

where $\mathbf{A}_d, \mathbf{P}_d \in \mathbb{Z}_q^{n \times m}$, $\mathbf{B}_d \in \mathbb{Z}_q^{n \times m'}$, and $\mathbf{y}_1, \mathbf{y}_2' \in \mathbb{Z}_q^m$, $\mathbf{y}_2 \in \mathbb{Z}_q^{m'}$.

- 2) Algorithm $\mathcal{D}'$ simulates $\mathcal{S}_{\mathcal{A}} = (\mathcal{S}_{\mathcal{A},v}, \mathcal{S}_{\mathcal{A},u})$ as follows:

- It samples $\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}, \mathbf{r}_{\text{pri}} \xleftarrow{\$} \{0, 1\}^\kappa$ as the randomness for algorithm $\mathcal{S}_{\mathcal{A}}$.
- It computes

$$\left(\begin{array}{l} 1^\ell, 1^{N_i+Q'}; \\ (\mathbf{r}_{\rho_1(1)}, \mathbf{v}_{\rho_1(1)}), \dots, (\mathbf{r}_{\rho_1(N_1)}, \mathbf{v}_{\rho_1(N_1)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ (\mathbf{r}_{\rho_2(1)}, \mathbf{v}_{\rho_2(1)}), \dots, (\mathbf{r}_{\rho_2(N_2)}, \mathbf{v}_{\rho_2(N_2)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ \dots; \\ (\mathbf{r}_{\rho_\ell(1)}, \mathbf{v}_{\rho_\ell(1)}), \dots, (\mathbf{r}_{\rho_\ell(N_\ell)}, \mathbf{v}_{\rho_\ell(N_\ell)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}) \end{array} \right) \leftarrow \mathcal{S}_{\mathcal{A},v}(\text{gp}; (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2})).$$

- It computes $\mathbf{u} \leftarrow \mathcal{S}_{\mathcal{A},u}(\text{gp}, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}); \mathbf{r}_{\text{pri}})$.

- 3) Algorithm $\mathcal{D}'$ samples $\mathbf{A}_i, \mathbf{P}_i \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$ and $\mathbf{B}_i \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}$ for each $i \neq d$. It also sets $\mathbf{A}_d, \mathbf{B}_d$, and $\mathbf{P}_d$ to be the respective components provided in the LWE challenge instance. Note that the distribution of the matrix $\{\mathbf{A}_i\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}]$ constructed here exactly matches the setup process in the evLPFE assumption.
- 4) Algorithm $\mathcal{D}'$ constructs the remaining components $\mathbf{Q}_1, \dots, \mathbf{Q}_\ell$ as described in the evLPFE assumption.
- 5) For each $i \in [\ell]$ such that $i > d$, algorithm $\mathcal{D}'$ samples $\mathbf{s}_i \xleftarrow{\$} \mathbb{Z}_q^n$. It then proceeds as follows:

- $\mathbf{z}_{1,i}$:
 - For each $i < d$, sample $\mathbf{z}_{1,i} \xleftarrow{\$} \mathbb{Z}_q^m$.
 - For $i = d$, sample $\mathbf{e}_{1,d} \leftarrow D_{\mathbb{Z},\chi}^m$ and compute $\mathbf{z}_{1,d}^\top \leftarrow \mathbf{y}_1^\top + \mathbf{e}_{1,d}^\top$.
 - For each $i > d$, sample $\mathbf{e}_{1,i} \leftarrow D_{\mathbb{Z},\chi}^m$, $\tilde{\mathbf{e}}_{1,i} \leftarrow D_{\mathbb{Z},\chi_s}^m$, and compute $\mathbf{z}_{1,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{A}_i + \tilde{\mathbf{e}}_{1,i}^\top + \mathbf{e}_{1,i}^\top$.
- $\mathbf{z}_{2,i}$:
 - For each $i < d$, sample $\tilde{\mathbf{z}}_{2,i} \xleftarrow{\$} \mathbb{Z}_q^{m'}$, $\tilde{\mathbf{z}}_{2,i}' \xleftarrow{\$} \mathbb{Z}_q^m$.
 - For $i = d$, set $\tilde{\mathbf{z}}_{2,d} \leftarrow \mathbf{y}_2$ and $\tilde{\mathbf{z}}_{2,d}' \leftarrow \mathbf{y}_2'$.
 - For each $i > d$, sample $\tilde{\mathbf{e}}_{2,i} \leftarrow D_{\mathbb{Z},\chi_s}^{m'}$ and $\tilde{\mathbf{e}}_{2,i}' \leftarrow D_{\mathbb{Z},\chi_s}^m$, and compute $\tilde{\mathbf{z}}_{2,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{B}_i + \tilde{\mathbf{e}}_{2,i}^\top$ and $\tilde{\mathbf{z}}_{2,i}'^\top \leftarrow \mathbf{s}_i^\top \mathbf{P}_i + \tilde{\mathbf{e}}_{2,i}'^\top$. Then sample $\mathbf{e}_2 \leftarrow D_{\mathbb{Z},\chi}^{m'+m}$ and set

$$\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}'^\top + \mathbf{u}^\top \mathbf{G} \right] + \mathbf{e}_2^\top.$$

- $\mathbf{z}_{3,i}^{(1)}, \mathbf{z}_{3,i}^{(2)}$: Sample $\mathbf{z}_{3,i}^{(1)}, \mathbf{z}_{3,i}^{(2)}$ via the same procedure as in $H_{1,d}^{\text{Pre}}$.

- 6) Finally, algorithm $\mathcal{D}'$ sends the challenge $\{1^\lambda, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}), \{\mathbf{A}_i, \mathbf{z}_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top, \{\mathbf{z}_{3,i}^\top\}_{i \in [\ell]}\}$ to distinguisher $\mathcal{D}$ and outputs whatever $\mathcal{D}$ outputs.

We now analyze the distribution of the challenge given to $\mathcal{D}$. Note that the distributions of matrices $\mathbf{A}_d, \mathbf{B}_d, \mathbf{P}_d$ match exactly those in the original game. Therefore, it suffices to analyze the distribution of

vectors $\mathbf{z}_{1,i}, \mathbf{z}_2, \mathbf{z}_{3,i}$. Observe that for all $i \neq d$, the components $\mathbf{z}_{1,i}, \mathbf{z}_2, \mathbf{z}_{3,i}$ are constructed identically in both $H_{1,d-1}^{\text{Pre}}$ and $H_{1,d}^{\text{Pre}}$. Hence, we only need to analyze the case of $i = d$. We now distinguish between the two possible forms of the LWE challenge, depending on whether the challenge is structured or uniformly random.

- If $\mathbf{y}_1 = \mathbf{s}^\top \mathbf{A}_d + \mathbf{e}_1^\top, \mathbf{y}_2 = \mathbf{s}^\top \mathbf{B}_d + \mathbf{e}_2^\top, \mathbf{y}'_2 = \mathbf{s}^\top \mathbf{P}_d + \mathbf{e}'_2{}^\top$ for some $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n, \mathbf{e}_1, \mathbf{e}'_2 \leftarrow D_{\mathbb{Z}, \chi_s}^m, \mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi_s}^{m'}$. In this case, the values $\mathbf{z}_{1,d}, \tilde{\mathbf{z}}_{2,d}, \tilde{\mathbf{z}}'_{2,d}$ are distributed identically to those in $H_{1,d-1}^{\text{Pre}}$.
- If $\mathbf{y}_1, \mathbf{y}'_2 \xleftarrow{\$} \mathbb{Z}_q^m, \mathbf{y}_2 \xleftarrow{\$} \mathbb{Z}_q^{m'}$, then $\mathbf{z}_{1,d}, \tilde{\mathbf{z}}_{2,d}, \tilde{\mathbf{z}}'_{2,d}$ are sampled according to the same distributions as in $H_{1,d}^{\text{Pre}}$.

Consequently, $\mathcal{D}'$ provides a perfect simulation of the distribution of the corresponding hybrid. As a result, algorithm $\mathcal{D}'$ can distinguish the LWE instance from the uniform instance with the same advantage as $\mathcal{D}$ distinguishes between $H_{1,d-1}^{\text{Pre}}$ and $H_{1,d}^{\text{Pre}}$. This leads to a contradiction with the LWE assumption. We can conclude $H_{1,d-1}^{\text{Pre}} \stackrel{c}{\approx} H_{1,d}^{\text{Pre}}$ for each $d \in [\ell]$. By a standard hybrid argument, it follows that $H_1^{\text{Pre}} \stackrel{c}{\approx} H_2^{\text{Pre}}$, as desired. $\square$

Lemma 33. Suppose $m' > 6n \log q$ and $\chi' = \Omega(\sqrt{n \log q})$. Let χ_s be an error parameter such that $\chi \geq \lambda^{\omega(1)} \cdot \sqrt{\lambda \chi_s}$ and the $\text{LWE}_{n, m_1, q, \chi_s}$ assumption holds for $m_1 = \text{poly}(Q_0)$, where Q_0 is the upper bound on the total number of secret-key queries (including those in the partial set $\mathcal{Q}'_{\text{par}}$) submitted by the adversary. Then we have that $H_2^{\text{Pre}} \stackrel{c}{\approx} H_3^{\text{Pre}}$.

Proof. We prove this claim by defining a sequence of intermediate hybrid games between H_2^{Pre} and H_3^{Pre} . For each $d \in [N_1]$, we define the following games:

Game $H_{2,d,0}^{\text{Pre}}$. This experiment is identical to H_2^{Pre} , except for how the components $y_{1,j}$ are sampled.

- For each $j < d$, the challenger samples $y_{1,j} \xleftarrow{\$} \mathbb{Z}_q$.
- For $j \geq d$, the challenger computes $y_{1,j}$ exactly as in H_2^{Pre} , i.e., $y_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_1(j)}) + \tilde{\mathbf{z}}_{2,1}^\top \mathbf{r}_{\rho_1(j)}$.

Game $H_{2,d,1}^{\text{Pre}}$. This experiment is identical to $H_{2,d,0}^{\text{Pre}}$, except for how it samples the values $\tilde{\mathbf{z}}_{2,i}$. Most of the notation follows the convention used in [WW22].

- Denote $\gamma = \rho_1(d)$. Precisely, γ is the index of the d -th Type I secret-key query involving authority aid_1^* .
- We consider the γ -th Type I secret-key query $(\text{gid}_\gamma, A_\gamma, \mathbf{v}_\gamma)$ submitted by the adversary $\mathcal{A}$. Since the restriction of the Type I secret-key query guarantees that $A_\gamma \subsetneq A^* \cap \mathcal{N}$, there exists some index $1 < i^* \leq \ell$ such that $\text{aid}_{i^*}^* \notin A_\gamma$. Let i^* be the smallest such index. In particular, $\text{aid}_{i^*}^* \notin A_\gamma$.
- The challenger samples $\tilde{\mathbf{z}}_{2,1} \xleftarrow{\$} \mathbb{Z}_q^{m'}, \tilde{\mathbf{z}}_{2,i^*} \xleftarrow{\$} \mathbb{Z}_q^{m'}$. Then sample $\mathbf{s}_0 \xleftarrow{\$} \mathbb{Z}_q^{m'}$, and compute $\tilde{\mathbf{z}}_{2,1} \leftarrow \tilde{\mathbf{z}}_{2,1} + \mathbf{s}_0$, $\tilde{\mathbf{z}}_{2,i^*} \leftarrow \tilde{\mathbf{z}}_{2,i^*} - \mathbf{s}_0$. For $i \notin \{1, i^*\}$, the challenger samples $\tilde{\mathbf{z}}_{2,i} \xleftarrow{\$} \mathbb{Z}_q^{m'}$, as in $H_{2,d,0}^{\text{Pre}}$. The components are then constructed as follows:

– $\mathbf{z}_2$: Set

$$\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top + \mathbf{u}^\top \mathbf{G} \right] + \mathbf{e}_2^\top.$$

– $y_{i,j}$: For $i = 1$ and $j < d$, sample $y_{1,j} \xleftarrow{\$} \mathbb{Z}_q$. Otherwise (i.e., for $i \neq 1$ or $j \geq d$), compute $y_{i,j} \leftarrow \tilde{\mathbf{z}}_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}_{\rho_i(j)}$.

– $y'_{i,j}$: For each $i \in [\ell]$ and $j \in [Q']$, compute $y'_{i,j}$ as in $H_{2,d,0}^{\text{Pre}}$, i.e., $y'_{i,j} \leftarrow \tilde{\mathbf{z}}_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}'_j$.

- In particular, we have the affected components in the following:

– For $i = 1$ and $d \leq j \leq N_1$, we have

$$y_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_1(j)}) + \tilde{\mathbf{z}}_{2,1}^\top \mathbf{r}_{\rho_1(j)} = \tilde{\mathbf{z}}_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_1(j)}) + \tilde{\mathbf{z}}_{2,1}^\top \mathbf{r}_{\rho_1(j)} + \mathbf{s}_0^\top \mathbf{r}_{\rho_1(j)}.$$

- For $i = 1$ and $j \in [Q']$, we have

$$y'_{1,j} = \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{r}'_j = \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \boxed{\tilde{\mathbf{z}}_{2,1}'^\top \mathbf{r}'_j + \mathbf{s}_0^\top \mathbf{r}'_j}.$$

- For $i = i^*$ and $j \in [N_{i^*}]$, we have

$$y_{i^*,j} = \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_{i^*}(j)}) + \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}_{\rho_{i^*}(j)} = \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_{i^*}(j)}) + \boxed{\tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}_{\rho_{i^*}(j)} - \mathbf{s}_0^\top \mathbf{r}_{\rho_{i^*}(j)}}.$$

- For $i = i^*$ and $j \in [Q']$, we have that

$$y'_{i^*,j} = \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}'_j = \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \boxed{\tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}'_j - \mathbf{s}_0^\top \mathbf{r}'_j}.$$

Game $H_{2,d,2}^{\text{Pre}}$. The experiment is identical to $H_{2,d,1}^{\text{Pre}}$, except for how $y_{i,j}, y'_{i,j}$ are sampled. Specifically,

- The challenger samples $\tilde{e}_i \leftarrow D_{\mathbb{Z}, \chi_s}$ for each $i \in [Q]$, and $\tilde{e}'_j \leftarrow D_{\mathbb{Z}, \chi_s}$ for each $j \in [Q']$.
- For $i = 1$ and $j \geq d$, set

$$y_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_1(j)}) + \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{r}_{\rho_1(j)} + (\mathbf{s}_0^\top \mathbf{r}_{\rho_1(j)} + \boxed{\tilde{e}_{\rho_1(j)}}).$$

- For $i = 1$ and each $j \in [Q']$, set

$$y'_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{r}'_j + (\mathbf{s}_0^\top \mathbf{r}'_j + \boxed{\tilde{e}'_j}).$$

- For $i = i^*$, set

$$y_{i^*,j} \leftarrow \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_{i^*}(j)}) + \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}_{\rho_{i^*}(j)} - (\mathbf{s}_0^\top \mathbf{r}_{\rho_{i^*}(j)} + \boxed{\tilde{e}_{\rho_{i^*}(j)}}), \quad \text{for all } j \in [N_{i^*}]$$

$$y'_{i^*,j} \leftarrow \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}'_j - (\mathbf{s}_0^\top \mathbf{r}'_j + \boxed{\tilde{e}'_j}), \quad \text{for all } j \in [Q'].$$

- For all other pairs (i, j) , the components $y_{i,j}, y'_{i,j}$ are computed in the same way as in $H_{2,d,1}^{\text{Pre}}$.

Game $H_{2,d,3}^{\text{Pre}}$. The experiment is identical to $H_{2,d,2}^{\text{Pre}}$, except for how $y_{i,j}, y'_{i,j}$ are sampled. Specifically

- The challenger samples $\boxed{\delta_i \xleftarrow{\$} \mathbb{Z}_q}$ for each $i \in [Q]$, and $\boxed{\delta'_j \xleftarrow{\$} \mathbb{Z}_q}$ for each $j \in [Q']$.
- For $i = 1$ and $j \geq d$, set

$$y_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_1(j)}) + \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{r}_{\rho_1(j)} + \boxed{\delta_{\rho_1(j)}}.$$

- For $i = 1$ and each $j \in [Q']$, set

$$y'_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{r}'_j + \boxed{\delta'_j}.$$

- For $i = i^*$, set

$$y_{i^*,j} \leftarrow \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_{i^*}(j)}) + \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}_{\rho_{i^*}(j)} + \boxed{\delta_{\rho_{i^*}(j)}}, \quad \text{for all } j \in [N_{i^*}],$$

$$y'_{i^*,j} \leftarrow \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}'_j + \boxed{\delta'_j}, \quad \text{for all } j \in [Q'].$$

- For all other pairs (i, j) , the components $y_{i,j}, y'_{i,j}$ are computed in the same way as in $H_{2,d,2}^{\text{Pre}}$.

Game $H_{2,d,4}^{\text{Pre}}$. The experiment is identical to $H_{2,d,3}^{\text{Pre}}$ except it samples $\boxed{y_{1,d} \xleftarrow{\$} \mathbb{Z}_q}$.

Game $H_{2,d,5}^{\text{Pre}}$. The experiment is identical to $H_{2,d,4}^{\text{Pre}}$, except for how the values $y_{i,j}, y'_{i,j}$ are sampled. Specifically,

- The challenger samples $\tilde{e}_i \leftarrow D_{\mathbb{Z}, \chi_s}$ for each $i \in [Q]$, and $\tilde{e}'_j \leftarrow D_{\mathbb{Z}, \chi_s}$ for each $j \in [Q']$.

- For $i = 1$ and $j = d$, the challenger samples $y_{1,d} \xleftarrow{\$} \mathbb{Z}_q$.
- For $i = 1$ and $j > d$, set

$$y_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_1(j)}) + \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{r}_{\rho_1(j)} + \boxed{\mathbf{s}_0^\top \mathbf{r}_{\rho_1(j)} + \tilde{e}_{\rho_1(j)}}.$$

- For $i = 1$ and each $j \in [Q']$, set

$$y'_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{r}'_j + \boxed{\mathbf{s}_0^\top \mathbf{r}'_j + \tilde{e}'_j}.$$

- For $i = i^*$, set

$$y_{i^*,j} \leftarrow \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_{i^*}(j)}) + \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}_{\rho_{i^*}(j)} - \boxed{(\mathbf{s}_0^\top \mathbf{r}_{\rho_{i^*}(j)} + \tilde{e}_{\rho_{i^*}(j)})}, \quad \text{for all } j \in [N_{i^*}],$$

$$y'_{i^*,j} \leftarrow \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}'_j - \boxed{(\mathbf{s}_0^\top \mathbf{r}'_j + \tilde{e}'_j)}, \quad \text{for all } j \in [Q'].$$

- For all other pairs (i, j) , the components $y_{i,j}, y'_{i,j}$ are computed in the same way as in $H_{2,d,4}^{\text{Pre}}$.

Game $H_{2,d,6}^{\text{Pre}}$. This experiment is identical to $H_{2,d,5}^{\text{Pre}}$, except for how the values $y_{i,j}, y'_{i,j}$ are sampled.

- For $i = 1$ and $j > d$, set

$$y_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_1(j)}) + \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{r}_{\rho_1(j)} + \boxed{\mathbf{s}_0^\top \mathbf{r}_{\rho_1(j)}}.$$

- For $i = 1$ and each $j \in [Q']$, set

$$y'_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{r}'_j + \boxed{\mathbf{s}_0^\top \mathbf{r}'_j}.$$

- For $i = i^*$, set

$$y_{i^*,j} = \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_{i^*}(j)}) + \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}_{\rho_{i^*}(j)} + \boxed{\mathbf{s}_0^\top \mathbf{r}_{\rho_{i^*}(j)}}, \quad \text{for all } j \in [N_{i^*}],$$

$$y'_{i^*,j} = \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}'_j + \boxed{\mathbf{s}_0^\top \mathbf{r}'_j}, \quad \text{for all } j \in [Q'].$$

- For all other pairs (i, j) , the components $y_{i,j}, y'_{i,j}$ are computed in the same way as in $H_{2,d,5}^{\text{Pre}}$.

In the following, we present the proof that each adjacent pair of these intermediate hybrid games defined above is indistinguishable for all $d \in [N_1]$. By the hybrid argument, this will imply that $H_2^{\text{Pre}} \stackrel{c}{\approx} H_3^{\text{Pre}}$.

Claim 34. We have $H_{2,d,0}^{\text{Pre}} \equiv H_{2,d,1}^{\text{Pre}}$, for all $d \in [N_1]$.

Proof. The only difference between the two experiments lies in the way the vectors $\tilde{\mathbf{z}}_{2,1}$ and $\tilde{\mathbf{z}}_{2,i^*}$ are sampled. In $H_{2,d,0}^{\text{Pre}}$, they are sampled independently and uniformly from $\mathbb{Z}_q^{m'}$. In $H_{2,d,1}^{\text{Pre}}$, they are constructed as

$$\tilde{\mathbf{z}}_{2,1}^\top \leftarrow \tilde{\mathbf{z}}_{2,1} + \mathbf{s}_0, \tilde{\mathbf{z}}_{2,i^*}^\top \leftarrow \tilde{\mathbf{z}}_{2,i^*} - \mathbf{s}_0,$$

where $\tilde{\mathbf{z}}_{2,1}, \tilde{\mathbf{z}}_{2,i^*}, \mathbf{s}_0 \xleftarrow{\$} \mathbb{Z}_q^{m'}$. It follows that the joint distribution of the pair $(\tilde{\mathbf{z}}_{2,1}, \tilde{\mathbf{z}}_{2,i^*})$ in experiment $H_{2,d,1}^{\text{Pre}}$ is distributed identically to the pair sampled in $H_{2,d,0}^{\text{Pre}}$. This completes the proof. $\square$

Claim 35. Suppose that $\chi \geq \lambda^{\omega(1)} \cdot \sqrt{\lambda} \chi_s$. We have that $H_{2,d,1}^{\text{Pre}} \stackrel{s}{\approx} H_{2,d,2}^{\text{Pre}}$ for all $d \in [N_1]$.

Proof. The only difference between $H_{2,d,1}^{\text{Pre}}$ and $H_{2,d,2}^{\text{Pre}}$ lies in the modification of the error terms added to certain components $y_{i,j}$ and $y'_{i,j}$. We focus first on the error terms appearing in $\mathbf{z}_{3,i}^{(1)}$, as the reasoning for $\mathbf{z}_{3,i}^{(2)}$ is analogous. Let $z_{3,i,j}^{(1)}$ and $e_{3,i,j}^{(1)}$ denote the j -th entries of $\mathbf{z}_{3,i}^{(1)}$ and $\mathbf{e}_{3,i}^{(1)}$, respectively. Then

- In $H_{2,d,1}^{\text{Pre}}$, for the affected components, we have

$$z_{3,1,j}^{(1)} = \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_1(j)}) + \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{r}_{\rho_1(j)} + e_{3,1,j}^{(1)}, \quad \text{for } j > d$$

$$z_{3,i^*,j}^{(1)} = \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_{i^*}(j)}) + \tilde{\mathbf{z}}_{2,i^*}'^\top \mathbf{r}_{\rho_{i^*}(j)} + e_{3,i^*,j}^{(1)}, \quad \text{for all } j \in [N_{i^*}].$$

- In $H_{2,d,2}^{\text{Pre}}$, the corresponding components are computed as:

$$\begin{aligned} z_{3,1,j}^{(1)} &= \tilde{\mathbf{z}}_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_1(j)}) + \tilde{\mathbf{z}}_{2,1}^\top \mathbf{r}_{\rho_1(j)} + e_{3,1,j}^{(1)} + \tilde{e}_{\rho_1(j)}, \quad \text{for } j > d \\ z_{3,i^*,j}^{(1)} &= \tilde{\mathbf{z}}_{2,i^*}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_{i^*}(j)}) + \tilde{\mathbf{z}}_{2,i^*}^\top \mathbf{r}_{\rho_{i^*}(j)} + e_{3,i^*,j}^{(1)} - \tilde{e}_{\rho_{i^*}(j)} \quad \text{for all } j \in [N_{i^*}]. \end{aligned}$$

Since $\chi \geq \lambda^{\omega(1)} \cdot \sqrt{\lambda} \chi_s$, Lemma 4 ensures that $e_{3,1,j}^{(1)} + \tilde{e}_{\rho_1(j)} \stackrel{s}{\approx} e_{3,1,j}^{(1)}$, and similarly for $e_{3,i^*,j}^{(1)} - \tilde{e}_{\rho_{i^*}(j)} \stackrel{s}{\approx} e_{3,i^*,j}^{(1)}$.

For all other (i, j) , the distribution of $z_{3,i,j}^{(1)}$ remains unchanged between the two experiments. Hence we can conclude that the distribution of $\mathbf{z}_{3,i}^{(1)}$ is identical in both experiments for all $i \in [\ell]$. The same argument also applies to $\mathbf{z}_{3,i}^{(2)}$, as the additional error terms are smudged in an analogous manner. This completes the proof. $\square$

Claim 36. Suppose that $m' > 6n \log q$ and $\chi' = \Omega(\sqrt{n \log q})$. Suppose the $\text{LWE}_{n,m_1,q,\chi_s}$ assumption holds for some $m_1 = \text{poly}(Q_0)$. Then we have that $H_{2,d,2}^{\text{Pre}} \stackrel{c}{\approx} H_{2,d,3}^{\text{Pre}}$ for all $d \in [N_1]$.

Proof. Assume for contradiction that there exists an efficient distinguisher $\mathcal{D}$ that can distinguish between $H_{2,d,2}^{\text{Pre}}$ and $H_{2,d,3}^{\text{Pre}}$ with non-negligible advantage. We construct an algorithm $\mathcal{D}'$ for the $\text{FlipLWE}_{m',m_1,q,\chi',\chi_s}$ assumption. Algorithm $\mathcal{D}'$ proceeds as follows:

- 1) Algorithm $\mathcal{D}'$ begins by receiving a FlipLWE challenge $(\mathbf{D}, \delta)$, where $\mathbf{D} \in \mathbb{Z}_q^{m' \times (Q+Q')}$ and $\delta \in \mathbb{Z}_q^{Q+Q'}$. Then algorithm $\mathcal{D}'$ parses $\mathbf{D}$ and δ as:

$$\mathbf{D} = [\mathbf{D}_1 \mid \mathbf{D}'_1], \quad \delta^\top = [\delta_1^\top \mid \delta'_1{}^\top],$$

where $\mathbf{D}_1 \in \mathbb{Z}_q^{m' \times Q}$, $\mathbf{D}'_1 \in \mathbb{Z}_q^{m' \times Q'}$, and $\delta_1 \in \mathbb{Z}_q^Q$, $\delta'_1 \in \mathbb{Z}_q^{Q'}$. Let $\mathbf{d}_j, \mathbf{d}'_j$ denote the j -th column vectors of $\mathbf{D}_1, \mathbf{D}'_1$, respectively. Let δ_j, δ'_j denote the j -th entries of δ_1, δ'_1 , respectively.

- 2) Algorithm $\mathcal{D}'$ simulates $\mathcal{S}_{\mathcal{A},\mathbf{v}}$ as follows:

- It samples $\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2} \xleftarrow{\$} \{0, 1\}^\kappa$ as the randomness for algorithm $\mathcal{S}_{\mathcal{A},\mathbf{v}}$.
- Run algorithm $\mathcal{A}(1^\lambda; \mathbf{r}_{\text{pub}_1})$ with randomness $\mathbf{r}_{\text{pub}_1}$, and extract from its output: a set of Type I secret-key queries $\mathcal{Q} = \{(\text{gid}, A, \mathbf{v})\}$ and a partial set of Type II secret-key queries $\mathcal{Q}'_{\text{par}} = \{(\text{gid}', A')\}$.
- Run $\text{Samp}_{\mathbf{v}}(1^\lambda; \mathbf{r}_{\text{pub}_2})$ with randomness $\mathbf{r}_{\text{pub}_2}$. It outputs $\mathbf{v}'_1, \dots, \mathbf{v}'_{Q'}$.
- Finally, outputs

$$\begin{aligned} &1^\ell, \{1^{N_i+Q'}\}_{i \in [\ell]}; \\ &(\mathbf{d}_{\rho_1(1)}, \mathbf{v}_{\rho_1(1)}), \dots, (\mathbf{d}_{\rho_1(N_1)}, \mathbf{v}_{\rho_1(N_1)}), (\mathbf{d}'_1, \mathbf{v}'_1), \dots, (\mathbf{d}'_{Q'}, \mathbf{v}'_{Q'}); \\ &(\mathbf{d}_{\rho_2(1)}, \mathbf{v}_{\rho_2(1)}), \dots, (\mathbf{d}_{\rho_2(N_2)}, \mathbf{v}_{\rho_2(N_2)}), (\mathbf{d}'_1, \mathbf{v}'_1), \dots, (\mathbf{d}'_{Q'}, \mathbf{v}'_{Q'}); \\ &\dots; \\ &(\mathbf{d}_{\rho_\ell(1)}, \mathbf{v}_{\rho_\ell(1)}), \dots, (\mathbf{d}_{\rho_\ell(N_\ell)}, \mathbf{v}_{\rho_\ell(N_\ell)}), (\mathbf{d}'_1, \mathbf{v}'_1), \dots, (\mathbf{d}'_{Q'}, \mathbf{v}'_{Q'}). \end{aligned}$$

The only difference between the simulation and the original specification of $\mathcal{S}_{\mathcal{A},\mathbf{v}}$ lies in how the vectors $\mathbf{r}_i$ and $\mathbf{r}'_i$ are generated. In the simulation, the challenger replaces these vectors with $\mathbf{d}_i, \mathbf{d}'_i$ which are the columns of the FlipLWE matrix $\mathbf{D}$. However, since both $\mathbf{d}_i, \mathbf{d}'_i$ are sampled from the same distribution $D_{\mathbb{Z},\chi'}^{m'}$ as the original $\mathbf{r}_i, \mathbf{r}'_i$, this substitution does not affect the validity of the simulation.

- 3) Algorithm $\mathcal{D}'$ samples $\mathbf{r}_{\text{pri}} \xleftarrow{\$} \{0, 1\}^\kappa$ and computes $\mathbf{u} \leftarrow \mathcal{S}_{\mathcal{A},\mathbf{u}}(1^\lambda, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}); \mathbf{r}_{\text{pri}})$.
- 4) For each $i \in [\ell]$, it samples $\mathbf{A}_i, \mathbf{P}_i \xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{B}_i \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}$. It then sets

$$[\mathbf{B} \mid \mathbf{P}] \leftarrow \left[\begin{array}{c|c} \mathbf{B}_1 & \mathbf{P}_1 \\ \vdots & \vdots \\ \mathbf{B}_\ell & \mathbf{P}_\ell \end{array} \right] \in \mathbb{Z}_q^{n\ell \times (m'+m)}.$$

5) Algorithm $\mathcal{D}'$ samples $\tilde{\mathbf{z}}_{2,i} \xleftarrow{\$} \mathbb{Z}_q^{m'}$, $\tilde{\mathbf{z}}'_{2,i} \xleftarrow{\$} \mathbb{Z}_q^m$ for each $i \in [\ell]$, and constructs the following components:

- For $i = 1, j < d$, sample $y_{1,j}, y'_{1,j} \xleftarrow{\$} \mathbb{Z}_q$.
- For $i = 1, j \geq d$, compute

$$y_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_1(j)}) + \tilde{\mathbf{z}}_{2,1}^\top \mathbf{d}_{\rho_1(j)} + \delta_{\rho_1(j)}.$$

- For $i = 1, j \in [Q']$, compute

$$y'_{1,j} = \tilde{\mathbf{z}}_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,1}^\top \mathbf{d}'_j + \delta'_j.$$

- For $i = i^*$, compute

$$y_{i^*,j} = \tilde{\mathbf{z}}_{2,i^*}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_{i^*}(j)}) + \tilde{\mathbf{z}}_{2,i^*}^\top \mathbf{d}_{\rho_{i^*}(j)} + \delta_{\rho_{i^*}(j)}, \quad \text{for all } j \in [N_{i^*}],$$

$$y'_{i^*,j} = \tilde{\mathbf{z}}_{2,i^*}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i^*}^\top \mathbf{d}'_j + \delta'_j, \quad \text{for all } j \in [Q'].$$

- For $i \notin \{1, i^*\}$, compute $y_{i,j} \leftarrow \tilde{\mathbf{z}}_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{d}_{\rho_i(j)}$ and $y'_{i,j} \leftarrow \tilde{\mathbf{z}}_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{d}'_j$.

6) Algorithm $\mathcal{D}'$ constructs the components $\mathbf{z}_{1,i}, \mathbf{z}_2, \mathbf{z}_{3,i}$ as follows:

- $\mathbf{z}_{1,i}$: For each $i \in [\ell]$, sample $\mathbf{z}_{1,i} \xleftarrow{\$} \mathbb{Z}_q^m$.
- $\mathbf{z}_2$: Sample $\mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^{m'+m}$, and set

$$\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top + \mathbf{u}^\top \mathbf{G} \right] + \mathbf{e}_2^\top.$$

- $\mathbf{z}_{3,i}^\top = [\mathbf{z}_{3,i}^{(1)\top} \mid \mathbf{z}_{3,i}^{(2)\top}]$: For each $i \in [\ell]$, sample $\mathbf{e}_{3,i}^{(1)} \leftarrow D_{\mathbb{Z}, \chi}^{N_i}$, $\mathbf{e}_{3,i}^{(2)} \leftarrow D_{\mathbb{Z}, \chi}^{Q'}$, and set

$$\mathbf{z}_{3,i}^{(1)\top} \leftarrow [y_{i,1} \mid \cdots \mid y_{i,N_i}] + \mathbf{e}_{3,i}^{(1)\top},$$

$$\mathbf{z}_{3,i}^{(2)\top} \leftarrow [y'_{i,1} \mid \cdots \mid y'_{i,Q'}] + \mathbf{e}_{3,i}^{(2)\top}.$$

7) Finally, algorithm $\mathcal{D}'$ sends the challenge $\{1^\lambda, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}), (\mathbf{A}_i, \mathbf{z}_{1,i}^\top)_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top, \{\mathbf{z}_{3,i}^\top\}_{i \in [\ell]}\}$ to distinguisher $\mathcal{D}$ and outputs whatever $\mathcal{D}$ outputs.

By inspecting the construction of $\mathcal{D}'$, we can observe that the components

$$\{1^\lambda, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}_2}), (\mathbf{A}_i, \mathbf{z}_{1,i}^\top)_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}]\}$$

are identically distributed in both experiments $H_{2,d,2}^{\text{Pre}}$ and $H_{2,d,3}^{\text{Pre}}$. Therefore, it suffices to analyze the distributions of the components $\mathbf{z}_2$ and $\mathbf{z}_{3,i}$ in two experiments. We distinguish between the following two cases depending on the form of the flipped LWE challenge:

- If $\boldsymbol{\delta}^\top = \mathbf{s}_0^\top \mathbf{D} + \mathbf{e}^\top$ for some $\mathbf{s}_0 \xleftarrow{\$} \mathbb{Z}_q^{m'}$, $\mathbf{e} \leftarrow D_{\mathbb{Z}, \chi_s}^{Q+Q'}$, then by the construction procedure of $\mathbf{z}_2$ and $\mathbf{z}_{3,i}$, the simulated distribution is identical to that in experiment $H_{2,d,2}^{\text{Pre}}$.
- If $\boldsymbol{\delta} \xleftarrow{\$} \mathbb{Z}_q^{Q+Q'}$, then the resulting distributions of $\mathbf{z}_2, \mathbf{z}_{3,i}$ are exactly the same as in experiment $H_{2,d,3}^{\text{Pre}}$. Hence the simulation is exactly as in experiment $H_{2,d,3}^{\text{Pre}}$.

In conclusion, distinguisher $\mathcal{D}'$ constructed as above can distinguish flipped LWE sample from uniform samples with the same advantage as $\mathcal{D}$ distinguishes between $H_{2,d,2}^{\text{Pre}}$ and $H_{2,d,3}^{\text{Pre}}$. However, by corollary 8, under the assumption $\text{LWE}_{n,m_1,q,\chi_s}$ and given parameter constraints, the assumption $\text{FlipLWE}_{m',m_1,q,\chi',\chi_s}$ must hold. This leads to a contradiction, implying that $H_{2,d,2}^{\text{Pre}} \stackrel{c}{\approx} H_{2,d,3}^{\text{Pre}}$, which completes the proof. $\square$

Claim 37. We have that $H_{2,d,3}^{\text{Pre}} \equiv H_{2,d,4}^{\text{Pre}}$ for all $d \in [N_1]$.

Proof. The only difference between experiments $H_{2,d,3}^{\text{Pre}}$ and $H_{2,d,4}^{\text{Pre}}$ lies in the distribution of $y_{1,d}$. First, we argue that $\rho_1(d) \neq \rho_{i^*}(j)$ for all $j \in [N_{i^*}]$. This follows from the construction of i^* . Recall i^* was defined as the smallest index such that $\text{aid}_{i^*} \notin A_{\rho_1(d)}$. Since $\rho_{i^*}(j)$ indexes secret-key queries that contain aid_{i^*} , it

follows that $A_{\rho_{i^*}(j)} \neq A_{\rho_1(d)}$ for all $j \in [N_{i^*}]$. In particular, $\rho_{i^*}(j) \neq \rho_1(d)$ for all $j \in [N_{i^*}]$. This implies that the only component that involves $\delta_{\rho_1(d)}$ in experiment $H_{2,d,3}^{\text{Pre}}$ is $y_{1,d}$. Since $\delta_{\rho_1(d)}$ is sampled from the uniform distribution and is independent of other elements, the replacement of $y_{1,d}$ with a uniformly random value in $H_{2,d,4}^{\text{Pre}}$ does not affect the overall distribution of the experiment, completing the proof. $\square$

Claim 38. Suppose that $m' > 6n \log q$ and $\chi' = \Omega(\sqrt{n \log q})$. Suppose the $\text{LWE}_{n,m_1,q,\chi_s}$ assumption holds for some $m_1 = \text{poly}(Q_0)$. Then we have that $H_{2,d,4}^{\text{Pre}} \equiv H_{2,d,5}^{\text{Pre}}$ for all $d \in [N_1]$.

Proof. The proof follows essentially the same argument as that of Claim 36. $\square$

Claim 39. Suppose that $\chi \geq \lambda^{\omega(1)} \cdot \sqrt{\lambda} \chi_s$. We have that $H_{2,d,5}^{\text{Pre}} \stackrel{s}{\approx} H_{2,d,6}^{\text{Pre}}$ for all $d \in [N_1]$.

Proof. The proof follows essentially the same argument as that Claim 35. $\square$

Claim 40. For all $d \in [N_1]$, we have $H_{2,d-1,6}^{\text{Pre}} \equiv H_{2,d,0}^{\text{Pre}}$.

Proof. The proof follows essentially the same argument as that of Claim 34. $\square$

Proof of Lemma 33 (Continued). By Claims 34 through 40, we obtain that for all $d \in [N_1]$, $H_{2,d-1,0}^{\text{Pre}} \stackrel{c}{\approx} H_{2,d,0}^{\text{Pre}}$ for all $d \in [N_1]$. Applying a standard hybrid argument over the sequence of hybrids, $H_{2,1,0}^{\text{Pre}} \stackrel{c}{\approx} H_{2,N_1,0}^{\text{Pre}}$. By definition of these hybrid experiments above, $H_{2,1,0}^{\text{Pre}} \equiv H_2^{\text{Pre}}$ and $H_{2,N_1,0}^{\text{Pre}} \equiv H_3^{\text{Pre}}$, which together yields $H_2^{\text{Pre}} \stackrel{c}{\approx} H_3^{\text{Pre}}$, as claimed. $\square$

Lemma 41. Suppose that $\chi \geq \lambda^{\omega(1)} \cdot \sqrt{\lambda} \chi_s$ and $\text{Samp} = (\text{Samp}_v, \text{Samp}_u)$ produces pseudorandom noisy inner product with noise parameter χ_s . We have that $H_3^{\text{Pre}} \stackrel{c}{\approx} H_4^{\text{Pre}}$.

Proof. We prove the lemma by defining a sequence of hybrid experiments between H_3^{Pre} and H_4^{Pre} .

Game $H_{3,0}^{\text{Pre}}$. The experiment is identical to H_3^{Pre} , except for how it samples $\tilde{z}'_{2,1}$. Specifically, the challenger first samples $\tilde{z}'_{2,1} \xleftarrow{\$} \mathbb{Z}_q^m$, and sets $\tilde{z}'_{2,1} \leftarrow \tilde{z}'_{2,1} - \mathbf{u}^\top \mathbf{G}$. This modification affects the computation of the following components:

- $\mathbf{z}_2$: We have

$$\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}'^\top + \mathbf{u}^\top \mathbf{G} \right] = \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \tilde{\mathbf{z}}_{2,1}'^\top + \sum_{2 \leq i \leq \ell} \tilde{\mathbf{z}}_{2,i}'^\top \right].$$

- $y'_{1,j}$: We have

$$y'_{1,j} = \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,1}^\top \mathbf{r}'_j = \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,1}^\top \mathbf{r}'_j - \mathbf{u}^\top \mathbf{v}'_j$$

for each $j \in [Q']$.

Game $H_{3,1}^{\text{Pre}}$. The experiment is identical to $H_{3,0}^{\text{Pre}}$, except for the procedure how it samples $y'_{1,j}$. Specifically, the term $\mathbf{u}^\top \mathbf{v}'_j$ is perturbed by an additional noise term.

- For each $j \in [Q']$, the challenger first samples $e_j \xleftarrow{\$} D_{\mathbb{Z}, \chi_s}$, and sets

$$y'_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,1}^\top \mathbf{r}'_j - (\mathbf{u}^\top \mathbf{v}'_j + \boxed{e_j}).$$

Game $H_{3,2}^{\text{Pre}}$. The experiment is identical to $H_{3,1}^{\text{Pre}}$ except for the procedure how it samples $y'_{1,j}$.

- For each $j \in [Q']$, the challenger first samples $\omega_j \xleftarrow{\$} \mathbb{Z}_q$, and sets

$$y'_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}'^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,1}^\top \mathbf{r}'_j + \boxed{\omega_j}.$$

Game $H_{3,3}^{\text{Pre}}$. The experiment is identical to $H_{3,2}^{\text{Pre}}$ except it samples $y'_{1,j} \xleftarrow{\$} \mathbb{Z}_q$ for each $j \in [Q']$.

From the constructions above, we first observe that $H_{3,0}^{\text{Pre}} \equiv H_3^{\text{Pre}}$. The only difference between these two experiments lies in the way $\tilde{\mathbf{z}}'_{2,1}$ is sampled, which is set as $\tilde{\mathbf{z}}'_{2,1} - \mathbf{u}^\top \mathbf{G}$ in $H_{3,0}^{\text{Pre}}$. However, since $\tilde{\mathbf{z}}'_{2,1}$ is sampled uniformly from $\mathbb{Z}_q^m$, it follows that $\tilde{\mathbf{z}}'_{2,1}$ remains uniformly distributed over $\mathbb{Z}_q^m$ and independent of all other components. Therefore, the joint distribution of all variables in $H_{3,0}^{\text{Pre}}$ remains identical to that in H_3^{Pre} .

Similarly, we have that $H_{3,3}^{\text{Pre}} \equiv H_4^{\text{Pre}}$. This equivalence follows directly from the fact that the only symbolic change between the two experiments is the replacement of $\tilde{\mathbf{z}}'_{2,1}$ with $\tilde{\mathbf{z}}'_{2,1}$. Since both variables are sampled uniformly from $\mathbb{Z}_q^m$, this substitution does not affect the distribution of any component in the experiment. Since all values $y'_{1,j}$ are sampled uniformly at random in experiments $H_{3,3}^{\text{Pre}}$ and H_4^{Pre} , the overall distributions of the two experiments are identical.

In the following, we present the proof that each adjacent pair of these intermediate hybrids $H_{3,i}^{\text{Pre}}$ and $H_{3,i+1}^{\text{Pre}}$ are indistinguishable for $i = 0, 1, 2$. By applying a standard hybrid argument, these results yield that $H_3^{\text{Pre}} \approx H_4^{\text{Pre}}$, as desired.

Claim 42. Suppose that $\chi \geq \lambda^{\omega(1)} \cdot \chi_s$. Then we have that $H_{3,0}^{\text{Pre}} \approx H_{3,1}^{\text{Pre}}$.

Proof. The proof follows essentially the same noise smudging argument as that of Claim 35. $\square$

Claim 43. Suppose that the sampler $\text{Samp} = (\text{Samp}_v, \text{Samp}_u)$ produces pseudorandom noisy inner products with noise parameter χ_s . Then we have $H_{3,1}^{\text{Pre}} \approx H_{3,2}^{\text{Pre}}$.

Proof. Assume for contradiction that there exists an efficient distinguisher $\mathcal{D}$ that can distinguish $H_{3,1}^{\text{Pre}}$ and $H_{3,2}^{\text{Pre}}$ with non-negligible advantage. We use $\mathcal{D}$ to construct an algorithm $\mathcal{D}'$ that breaks the pseudorandomness of noisy inner products produced by the sampler Samp . Algorithm $\mathcal{D}'$ proceeds as follows:

- 1) Algorithm $\mathcal{D}'$ begins by receiving a challenge instance $(\mathbf{r}_{\text{pub}}, \{\beta_j\}_{j \in [Q']})$.
- 2) Algorithm $\mathcal{D}'$ simulates the sampler $\mathcal{S}_{\mathcal{A},v}$ as follows:
 - It samples $\mathbf{r}_{\text{pub}_1} \xleftarrow{\$} \{0, 1\}^\kappa$ as part of the randomness for algorithm $\mathcal{S}_{\mathcal{A},v}$.
 - It runs algorithm $\mathcal{A}(1^\lambda; \mathbf{r}_{\text{pub}_1})$ with randomness $\mathbf{r}_{\text{pub}_1}$, and extracts from its output: a set of Type I secret-key queries $\mathcal{Q} = \{(\text{gid}, A, \mathbf{v})\}$ and a partial set of Type II secret-key queries $\mathcal{Q}'_{\text{par}} = \{(\text{gid}', A')\}$.
 - For each $i \in [Q]$, sample $\mathbf{r}_i \leftarrow D_{\mathbb{Z}, \chi'}^{m'}$, and for each $j \in [Q']$, sample $\mathbf{r}'_j \leftarrow D_{\mathbb{Z}, \chi'}^{m'}$.
 - Run $\text{Samp}_v(1^\lambda; \mathbf{r}_{\text{pub}})$ with randomness $\mathbf{r}_{\text{pub}}$. It outputs $\mathbf{v}'_1, \dots, \mathbf{v}'_{Q'}$.
 - Finally, outputs

$$\begin{aligned}
 & 1^\ell, \{1^{N_i+Q'}\}_{i \in [\ell]}; \\
 & (\mathbf{r}_{\rho_1(1)}, \mathbf{v}_{\rho_1(1)}), \dots, (\mathbf{r}_{\rho_1(N_1)}, \mathbf{v}_{\rho_1(N_1)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\
 & (\mathbf{r}_{\rho_2(1)}, \mathbf{v}_{\rho_2(1)}), \dots, (\mathbf{r}_{\rho_2(N_2)}, \mathbf{v}_{\rho_2(N_2)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\
 & \dots; \\
 & (\mathbf{r}_{\rho_\ell(1)}, \mathbf{v}_{\rho_\ell(1)}), \dots, (\mathbf{r}_{\rho_\ell(N_\ell)}, \mathbf{v}_{\rho_\ell(N_\ell)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}).
 \end{aligned}$$

The only difference between this simulation and the original specification of $\mathcal{S}_{\mathcal{A},v}$ lies in how the randomness $\mathbf{r}_{\text{pub}}$ is generated. In the simulation, algorithm $\mathcal{D}'$ uses the challenge-provided $\mathbf{r}_{\text{pub}}$ instead of a freshly sampled $\mathbf{r}_{\text{pub}_2}$. However, since $\mathbf{r}_{\text{pub}}$ is uniformly distributed, this substitution does not alter the distribution of any component. Thus, the overall output remains consistent with that of the original specification of $\mathcal{S}_{\mathcal{A},v}$.

3) Algorithm $\mathcal{D}'$ samples $\mathbf{A}_i, \mathbf{P}_i \xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{B}_i \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}$ for all $i \in [\ell]$. It then sets

$$[\mathbf{B} \mid \mathbf{P}] \leftarrow \left[\begin{array}{c|c} \mathbf{B}_1 & \mathbf{P}_1 \\ \vdots & \vdots \\ \mathbf{B}_\ell & \mathbf{P}_\ell \end{array} \right] \in \mathbb{Z}_q^{n\ell \times (m'+m)}.$$

4) Then algorithm $\mathcal{D}'$ constructs the components as follows:

- $\mathbf{z}_{1,i}$: For each $i \in [\ell]$, sample $\mathbf{z}_{1,i} \xleftarrow{\$} \mathbb{Z}_q^m$.
- $\mathbf{z}_2$: For each $i \in [\ell]$, sample $\tilde{\mathbf{z}}_{2,i} \xleftarrow{\$} \mathbb{Z}_q^{m'}$. Then for each $2 \leq i \leq \ell$, sample $\tilde{\mathbf{z}}'_{2,i} \xleftarrow{\$} \mathbb{Z}_q^m$, and sample $\tilde{\mathbf{z}}'_{2,1} \xleftarrow{\$} \mathbb{Z}_q^m$. Set

$$\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \tilde{\mathbf{z}}'_{2,1}^\top + \sum_{2 \leq i \leq \ell} \tilde{\mathbf{z}}'_{2,i}^\top \right].$$

- $\mathbf{z}_{3,i}^{(1)}$: It is computed in the same way as in experiment $\mathbf{H}_{3,1}^{\text{Pre}}$.
- $\mathbf{z}_{3,i}^{(2)}$:
 - For $i = 1$, compute

$$y'_{1,j} \leftarrow \tilde{\mathbf{z}}_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}'_j + \beta_j$$

for each $j \in [Q']$.

- For $i > 1$, the sampling procedure for $y'_{i,j}$ is the same as in experiment $\mathbf{H}_{3,1}^{\text{Pre}}$ and $\mathbf{H}_{3,2}^{\text{Pre}}$.
- Finally, it samples $\mathbf{e}_{3,i}^{(2)} \leftarrow D_{\mathbb{Z}, \chi}^{Q'}$ for each $i \in [\ell]$, and sets

$$\mathbf{z}_{3,i}^{(2)\top} \leftarrow [y'_{i,1} \mid \cdots \mid y'_{i,Q'}] + \mathbf{e}_{3,i}^{(2)\top}.$$

5) The algorithm $\mathcal{D}'$ sends the challenge $\{1^\lambda, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}}), (\mathbf{A}_i, \mathbf{z}_{1,i}^\top)_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top, \{\mathbf{z}_{3,i}^\top\}_{i \in [\ell]}\}$ to the distinguisher $\mathcal{D}$ and outputs whatever $\mathcal{D}$ outputs.

From the definition of the algorithm $\mathcal{D}$, it is clear that the components

$$\{1^\lambda, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}}), (\mathbf{A}_i, \mathbf{z}_{1,i}^\top)_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top, \{\mathbf{z}_{3,i}^{(1)\top}\}_{i \in [\ell]}, \{\mathbf{z}_{3,i}^{(2)\top}\}_{2 \leq i \leq \ell}\}$$

are exactly distributed as in $\mathbf{H}_{3,1}^{\text{Pre}}$ and $\mathbf{H}_{3,2}^{\text{Pre}}$. It remains to consider the distribution of $\mathbf{z}_{3,1}^{(2)}$ in two experiments.

- If $\beta_j = \mathbf{u}^\top \mathbf{v}'_j + e_j$ for some $\mathbf{u} \leftarrow \mathcal{S}_{\mathbf{u}}(1^\lambda, (\mathbf{r}_{\text{pub}_1}, \mathbf{r}_{\text{pub}}); \mathbf{r}_{\text{pri}})$, $e_j \leftarrow D_{\mathbb{Z}, \chi_s}$, then by the construction procedure, each $y'_{1,j}$ is computed exactly as in $\mathbf{H}_{3,1}^{\text{Pre}}$, and thus the resulting distribution of $\mathbf{z}_{3,1}^{(2)}$ matches that in $\mathbf{H}_{3,1}^{\text{Pre}}$.
- If $\beta_j \xleftarrow{\$} \mathbb{Z}_q$, then the resulting distribution of $\mathbf{z}_{3,1}^{(2)}$ is identical to that in experiment $\mathbf{H}_{3,2}^{\text{Pre}}$.

Therefore, algorithm $\mathcal{D}'$ breaks the pseudorandomness of the noisy inner product produced by Samp with the same advantage as $\mathcal{D}$ distinguishes between $\mathbf{H}_{3,1}^{\text{Pre}}$ and $\mathbf{H}_{3,2}^{\text{Pre}}$. This leads to a contradiction with the assumption that Samp produces pseudorandom noisy inner products. Hence $\mathbf{H}_{3,1}^{\text{Pre}} \stackrel{c}{\approx} \mathbf{H}_{3,2}^{\text{Pre}}$, as claimed. $\square$

Claim 44. We have that $\mathbf{H}_{3,2}^{\text{Pre}} \equiv \mathbf{H}_{3,3}^{\text{Pre}}$.

Proof. The claim follows directly from the observation that in experiment $\mathbf{H}_{3,2}^{\text{Pre}}$, the only component depending on ω_j is $y'_{1,j}$. Since ω_j is sampled uniformly and independent of all other components, it follows that $y'_{1,j}$ is also uniform and independent of all other components. This matches precisely the distribution of $y'_{1,j}$ in $\mathbf{H}_{3,3}^{\text{Pre}}$. $\square$

Proof of Lemma 41 (Continued). By applying a sequence of hybrid arguments from Claim 42 to Claim 44, we obtain that $\mathbf{H}_{3,0}^{\text{Pre}} \stackrel{c}{\approx} \mathbf{H}_{3,3}^{\text{Pre}}$. Since we have already established that $\mathbf{H}_{3,0}^{\text{Pre}} \equiv \mathbf{H}_3^{\text{Pre}}$ and $\mathbf{H}_{3,3}^{\text{Pre}} \equiv \mathbf{H}_4^{\text{Pre}}$, which completes the argument that $\mathbf{H}_3^{\text{Pre}} \stackrel{c}{\approx} \mathbf{H}_4^{\text{Pre}}$. $\square$

Lemma 45. Suppose $m' > 6n \log q$ and $\chi' = \Omega(\sqrt{n \log q})$. Let χ_s be an error parameter such that $\chi \geq \lambda^{\omega(1)} \cdot \sqrt{\lambda} \chi_s$ and suppose that the $\text{LWE}_{n, m_1, q, \chi_s}$ assumption holds for $m_1 = \text{poly}(Q_0)$, where Q_0 is the upper bound on the total number of secret-key queries (including those in the partial set $\mathcal{Q}'_{\text{par}}$) submitted by the adversary. Then we have that $H_4^{\text{Pre}} \stackrel{c}{\approx} H_5^{\text{Pre}}$.

Proof. We prove this lemma by defining a sequence of intermediate hybrid experiments between H_4^{Pre} and H_5^{Pre} . For each $1 < d \leq \ell + 1$, we introduce the following sequence of games:

Game $H_{4,d,0}^{\text{Pre}}$. The experiment is identical to H_4^{Pre} , except for how it samples $\mathbf{z}_2$ and $y_{i,j}, y'_{i,j}$. Specifically, the challenger first samples $\mathbf{z}_2 \xleftarrow{\$} \mathbb{Z}_q^{m'+m}$, and then sets $y_{i,j}$ and $y'_{i,j}$ as follows:

- For each $i < d$, sample $y_{i,j} \xleftarrow{\$} \mathbb{Z}_q$ for each $j \in [N_i]$ and $y'_{i,j} \xleftarrow{\$} \mathbb{Z}_q$ for each $j \in [Q']$.
- For each $i \geq d$, set

$$\begin{aligned} y_{i,j} &\leftarrow \tilde{\mathbf{z}}_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}_{\rho_i(j)}, \quad \text{for all } j \in [N_i], \\ y'_{i,j} &\leftarrow \tilde{\mathbf{z}}_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}'_j, \quad \text{for all } j \in [Q']. \end{aligned}$$

Game $H_{4,d,1}^{\text{Pre}}$. The experiment is identical to $H_{4,d,0}^{\text{Pre}}$, except for how it samples $y_{d,j}, y'_{d,j}$. The challenger first samples $e_i \leftarrow D_{\mathbb{Z}, \chi_s}$ for each $i \in [Q]$ and $e'_j \leftarrow D_{\mathbb{Z}, \chi_s}$ for each $j \in [Q']$, then sets $y_{i,j}$ and $y'_{i,j}$ as follows:

- For each $i < d$, sample $y_{i,j} \xleftarrow{\$} \mathbb{Z}_q$ for each $j \in [N_i]$ and $y'_{i,j} \xleftarrow{\$} \mathbb{Z}_q$ for each $j \in [Q']$ as in $H_{4,d,0}^{\text{Pre}}$.
- For $i = d$, set

$$\begin{aligned} y_{d,j} &\leftarrow \tilde{\mathbf{z}}_{2,d}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_d(j)}) + (\tilde{\mathbf{z}}_{2,d}^\top \mathbf{r}_{\rho_d(j)} + \boxed{e_{\rho_d(j)}}), \quad \text{for all } j \in [N_d], \\ y'_{d,j} &\leftarrow \tilde{\mathbf{z}}_{2,d}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + (\tilde{\mathbf{z}}_{2,d}^\top \mathbf{r}'_j + \boxed{e'_j}), \quad \text{for all } j \in [Q']. \end{aligned}$$

- For each $i > d$, compute $y_{i,j}, y'_{i,j}$ in the same way as in $H_{4,d,0}^{\text{Pre}}$.

Game $H_{4,d,2}^{\text{Pre}}$. The experiment is identical to $H_{4,d,1}^{\text{Pre}}$ except that it replaces the error terms in $y_{d,j}, y'_{d,j}$ with uniformly random values. The challenger first samples $\delta_i \xleftarrow{\$} \mathbb{Z}_q$ for each $i \in [Q]$, and $\delta'_j \xleftarrow{\$} \mathbb{Z}_q$ for each $j \in [Q']$. Then it sets $y_{d,j}$ and $y'_{d,j}$ as follows:

$$\begin{aligned} y_{d,j} &\leftarrow \tilde{\mathbf{z}}_{2,d}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_d(j)}) + \boxed{\delta_{\rho_d(j)}}, \quad \text{for all } j \in [N_d], \\ y'_{d,j} &\leftarrow \tilde{\mathbf{z}}_{2,d}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \boxed{\delta'_j}, \quad \text{for all } j \in [Q']. \end{aligned}$$

Game $H_{4,d,3}^{\text{Pre}}$. The experiment is identical to $H_{4,d,2}^{\text{Pre}}$ except for the values $y_{d,j}, y'_{d,j}$ are sampled uniformly at random. Specifically,

- The challenger samples $y_{d,j} \xleftarrow{\$} \mathbb{Z}_q$ for each $j \in [N_d]$ and $y'_{d,j} \xleftarrow{\$} \mathbb{Z}_q$ for each $j \in [Q']$.

From the construction above, it follows that $H_{4,2,0}^{\text{Pre}} \equiv H_4^{\text{Pre}}$. Since the only component dependent on $\tilde{\mathbf{z}}_{2,1}$ and $\tilde{\mathbf{z}}'_{2,1}$ is $\mathbf{z}_2$, directly sampling $\mathbf{z}_2 \xleftarrow{\$} \mathbb{Z}_q^{m'+m}$ does not affect the overall distribution. Moreover, it is straightforward to verify that $H_{4,\ell+1,0}^{\text{Pre}} \equiv H_5^{\text{Pre}}$. In the following, we prove that each adjacent pair of hybrids is computationally indistinguishable for all $2 \leq d \leq \ell$.

Claim 46. Suppose that $\chi \geq \lambda^{\omega(1)} \cdot \sqrt{\lambda} \chi_s$. Then we have that $H_{4,d,0}^{\text{Pre}} \stackrel{s}{\approx} H_{4,d,1}^{\text{Pre}}$ for $2 \leq d \leq \ell$.

Proof. The proof follows essentially the same argument as that of Claim 35. $\square$

Claim 47. Suppose that $m' > 6n \log q$ and $\chi' = \Omega(\sqrt{n \log q})$, and suppose the $\text{LWE}_{n,m_1,q,\chi_s}$ assumption holds for some $m_1 = \text{poly}(Q_0)$. Then we have $H_{4,d,1}^{\text{Pre}} \stackrel{c}{\approx} H_{4,d,2}^{\text{Pre}}$ for $2 \leq d \leq \ell$.

Proof. The proof follows essentially the same argument as that of Claim 36. $\square$

Claim 48. For all $2 \leq d \leq \ell$, we have $H_{4,d,2}^{\text{Pre}} \equiv H_{4,d,3}^{\text{Pre}}$.

Proof. This follows directly from the fact that δ_j, δ'_j are sampled uniformly and independently over $\mathbb{Z}_q$. Thus, replacing them with fresh uniform values does not alter the distributions of $y_{d,j}$ and $y'_{d,j}$. $\square$

Claim 49. For all $2 \leq d \leq \ell$, we have $H_{4,d,3}^{\text{Pre}} \equiv H_{4,d+1,0}^{\text{Pre}}$.

Proof. This follows directly by the construction of the two experiments. $H_{4,d,3}^{\text{Pre}}$ ends with $y_{d,j}, y'_{d,j}$ sampled uniformly at random, which matches the initial sampling procedure of $H_{4,d+1,0}^{\text{Pre}}$ for index d . $\square$

Proof of Lemma 45 (Continued). By Claims 46 through 49, we obtain that $H_{4,d,0}^{\text{Pre}} \stackrel{c}{\approx} H_{4,d+1,0}^{\text{Pre}}$ for all $2 \leq d \leq \ell$. By applying a standard hybrid argument, we can conclude that $H_4^{\text{Pre}} \equiv H_{4,2,0}^{\text{Pre}} \stackrel{c}{\approx} H_{4,\ell+1,0}^{\text{Pre}} \equiv H_5^{\text{Pre}}$, completing the proof. $\square$

Proof of Claim 27 (Continued). By combining Lemmas 31, 32, 33, and 41, and applying a standard hybrid argument, we conclude that $H_0^{\text{Pre}} \stackrel{c}{\approx} H_5^{\text{Pre}}$, thereby completing the proof. $\square$

C. Parameters

Let λ be the security parameter.

- We set the lattice dimension $n = \lambda^{1/\epsilon}$ and the modulus $q = 2^{\tilde{O}(n^\epsilon)} = 2^{\tilde{O}(\lambda)}$ for some constant $\epsilon > 0$, where $\tilde{O}(\cdot)$ suppresses constant and logarithmic factors. Then $m = n \lceil \log q \rceil = \tilde{O}(\lambda^{1+1/\epsilon})$.
- We set $L = 2^\lambda$ to support ciphertext policies of arbitrary polynomial size, i.e., $\ell = \text{poly}(\lambda)$.
- For the static security (Theorem 25), we set $m' = \tilde{O}(\lambda^{1+1/\epsilon})$, $Q_0 = \text{poly}(\lambda)$ and $\chi' = \Theta(\lambda^{1+1/\epsilon})$. The construction relies on the $\text{LWE}_{n,m_1,q,\chi_s}$ assumption and the $\text{IND-evIPFE}_{n,m,m',q,\chi,\chi}$ assumption, where the noise parameter $\chi_s = \chi_s(\lambda)$ is set polynomial-bounded and χ satisfies the bound $\chi \geq \lambda^{\omega(1)} \cdot (\sqrt{\lambda}(m + \ell)\chi_s + \lambda m \chi' \chi_s)$. This requirement is satisfied by setting $\chi = 2^{\tilde{O}(n^\epsilon)}$.
- To ensure correctness (Theorem 24), we require $\chi > \sqrt{n \log q} \cdot \omega(\sqrt{\log n})$, which is also satisfied by setting $\chi = 2^{\tilde{O}(n^\epsilon)}$. In addition, we have the relationship $B_0 = \sqrt{\lambda} \chi m + \lambda \chi \chi' m' + \lambda \chi^2 m L$, which implies $B_0 = 2^{\tilde{O}(n^\epsilon)}$. Then we obtain $B_0/(\sqrt{n} \chi_s) = \text{superpoly}(\lambda)$, which aligns with the convention in the NIPFE scheme as in [AP20].

We summarize with the following instantiation

Corollary 1 ((B_0, χ_s) -MA-ABevIPFE for Subset Policies in the Random Oracle Model). *Let λ be a security parameter. Assuming the polynomial hardness of LWE and of the evIPFE (defined in Section IV), both holding under a sub-exponential modulus-to-noise ratio, there exists a statically secure (B_0, χ_s) -MA-ABevIPFE scheme for subset policies in the random oracle model, with $B_0/\chi_s = \text{superpoly}(\lambda)$.*

VII. MA-ABNIPFE SCHEME FROM IND-evIPFE ASSUMPTION (IN THE RANDOM ORACLE MODEL)

We observe that the MA-ABevIPFE construction presented in Construction 1 also naturally satisfies the syntax of a (B_0, B_1) -MA-ABNIPFE scheme over $\mathbb{Z}_q^n$ for subset policies. Accordingly, we reinterpret the scheme in Construction 1 as an MA-ABNIPFE scheme, and denote it by

$$\Pi_{\text{MA-ABNIPFE}} = (\text{GlobalSetup}, \text{AuthSetup}, \text{Keygen}, \text{Enc}, \text{Dec}),$$

where all algorithms are identical to those defined in Construction 1.

In the following, we present the approximate correctness and static security properties of this scheme.

A. Correctness

Theorem 50 (Correctness). *Let $\chi_0 = \sqrt{n \log q} \cdot \omega(\sqrt{\log n})$ be a polynomial such that Lemma 7 holds. Suppose that the lattice parameters n, q, χ are such that $\chi \geq \chi_0(n, q)$. Then the scheme $\Pi_{\text{MA-ABNIPFE}}$ in Construction 1 is correct as a (B_0, B_1) -MA-ABNIPFE scheme, where the parameter $B_0 = \sqrt{\lambda} \chi m + \lambda \chi \chi' m' + \lambda \chi^2 m \ell$.*

Proof. The proof is essentially the same as Theorem 24. $\square$

B. Security

Theorem 51 (Static Security). *Let $\chi_0(n, q) = \sqrt{n \log q} \cdot \omega(\sqrt{\log n})$ be a polynomial such that Lemma 7 holds. Let Q_0 be an upper bound on the number of secret-key queries submitted by the adversary $\mathcal{A}$. Suppose that the following conditions hold:*

- $m' > 6n \log q$.
- Let χ' be an error distribution parameter such that $\chi' = \Omega(\sqrt{n \log q})$.
- Let χ be an error distribution parameter such that $\chi \geq \lambda^{\omega(1)} \cdot \max\{B_1, \sqrt{\lambda} \chi_s, \chi_0\}$, where χ_s is an noise parameter such that $\text{LWE}_{n, m_1, q, \chi_s}$ assumption holds for some $m_1 = \text{poly}(m, m', Q_0)$.
- The assumption $\text{IND-evIPFE}_{n, m, m', q, \chi, \chi'}$ holds.

Then, Construction 1 $\Pi_{\text{MA-ABNIPFE}}$ is statically secure as a (B_0, B_1) -MA-ABNIPFE scheme.

Proof. We prove the static security of $\Pi_{\text{MA-ABNIPFE}}$ under the standard static security model by defining two experiments H_0 and H_1 , corresponding to challenge bits 0 and 1, respectively. The goal is to show that no efficient adversary can distinguish between these two games with non-negligible advantage. We begin by giving the definition of the structure of Game H_b for $b \in \{0, 1\}$.

Game H_b . This experiment corresponds to the real static security game in which the challenger encrypts the plaintext $\mathbf{u}_b$ exactly as specified in Construction 1, where $b \in \{0, 1\}$. The detailed structure of the experiment is as follows.

At the beginning of the experiment, the adversary $\mathcal{A}$ specifies the following parameters:

- A set of corrupt authorities $\mathcal{C} \subseteq \mathcal{AU}$ along with their public keys: $\text{pk}_{\text{aid}} = (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$ for each $\text{aid} \in \mathcal{C}$.
- A set of non-corrupt authorities $\mathcal{N} \subseteq \mathcal{AU}$, satisfying $\mathcal{N} \cap \mathcal{C} = \emptyset$.
- A pair of challenge messages $\mathbf{u}_0, \mathbf{u}_1 \in \mathbb{Z}_q^n$ along with a designated authority set $A^* \subseteq \mathcal{C} \cup \mathcal{N}$ satisfying $(A^* \cap \mathcal{C}) \subsetneq A^*$.
- A set of secret-key queries $\mathcal{Q} = \{(\text{gid}, A, \mathbf{v})\}$, where $A \subseteq \mathcal{N}$. For each $(\text{gid}, A, \mathbf{v}) \in \mathcal{Q}$, exactly one of the following conditions must hold:
 - **Type I secret-key queries:** $(A \cup \mathcal{C}) \cap A^* \subsetneq A^*$;
 - **Type II secret-key queries:** $(A \cup \mathcal{C}) \cap A^* = A^*$ and $\|(\mathbf{u}_0 - \mathbf{u}_1)^\top \mathbf{v}\| \leq B_1$.

To simulate the random oracle, the challenger initializes an empty table $T : \mathcal{GID} \times \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q^{m'}$. This table will be used to store and consistently respond to all queries made to the random oracle during the experiment. The challenger then processes the adversary's queries as follows.

- **Public keys for non-corrupt authorities:** For each non-corrupt authority $\text{aid} \in \mathcal{N}$, the challenger samples $(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, $\mathbf{B}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}$, and $\mathbf{P}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$. The public key associated with aid is then set as $\text{pk}_{\text{aid}} = (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$.
- **Secret-key queries:** For each secret-key query $(\text{gid}, A, \mathbf{v}) \in \mathcal{Q}$, the challenger first computes $\mathbf{r}_{\text{gid}, \mathbf{v}} \leftarrow H(\text{gid}, \mathbf{v})$, then samples $\mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}} \leftarrow \text{SamplePre}(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}, \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{B}_{\text{aid}} \mathbf{r}_{\text{gid}, \mathbf{v}}, \chi)$ for each $\text{aid} \in A$. The resulting secret key is set as $\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}} = \mathbf{k}_{\text{aid}, \text{gid}, \mathbf{v}}$.

- **Challenge ciphertext:** The challenger first samples $s_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^n$, $e_{1,\text{aid}} \leftarrow D_{\mathbb{Z},\chi}^m$ for each $\text{aid} \in A^*$. Then it samples $e_2 \leftarrow D_{\mathbb{Z},\chi'}^{m'}$, and $e_3 \leftarrow D_{\mathbb{Z},\chi}^m$. The challenge ciphertext is constructed as $\text{ct} = (\{c_{1,\text{aid}}^\top\}_{\text{aid} \in A^*}, c_2^\top, c_3^\top)$, where

$$c_{1,\text{aid}}^\top = s_{\text{aid}}^\top A_{\text{aid}} + e_{1,\text{aid}}^\top, c_2^\top = \sum_{\text{aid} \in A^*} s_{\text{aid}}^\top B_{\text{aid}} + e_2^\top, c_3^\top = \sum_{\text{aid} \in A^*} s_{\text{aid}}^\top P_{\text{aid}} + e_3^\top + u_b^\top G.$$

- **Random oracle queries:** Upon receiving a query $(\text{gid}, \mathbf{v}) \in \mathcal{GID} \times \mathbb{Z}_q^n$, the challenger first checks whether the input $(\text{gid}, \mathbf{v})$ has been queried before, either during previous random oracle queries by the adversary or during processing the secret-key queries. If it has, the challenger retrieves and returns the stored value $\mathbf{r}_{\text{gid},\mathbf{v}}$ from the table $\mathcal{T}$. If the input is new, the challenger samples $\mathbf{r}_{\text{gid},\mathbf{v}} \leftarrow D_{\mathbb{Z},\chi'}^{m'}$, stores the mapping $(\text{gid}, \mathbf{v}) \mapsto \mathbf{r}_{\text{gid},\mathbf{v}}$ to the table $\mathcal{T}$, and then responds with $\mathbf{r}_{\text{gid},\mathbf{v}}$.

At the end of the game, the adversary outputs a bit $b' \in \{0, 1\}$ as its guess for b , which is taken as the output of the experiment.

To establish the static security of Construction 1 as a (B_0, B_1) -MA-ABNIPFE scheme, it suffices to show that games H_0 and H_1 are computationally indistinguishable. Suppose, for contradiction, that there exists an *efficient* adversary $\mathcal{A}$ that can distinguish between H_0 and H_1 with non-negligible advantage. We will show that such an adversary can be used to break the IND-evIPFE assumption, thereby contradicting its assumed hardness.

We construct a sampling algorithm $\mathcal{S}_{\mathcal{A}}$ that uses $\mathcal{A}$ as a subroutine for the IND-evIPFE assumption. On input the global parameter $\text{gp} = (1^\lambda, q, 1^m, 1^{m'}, 1^\chi, 1^\chi)$, the sampling algorithm $\mathcal{S}_{\mathcal{A}}(\text{gp})$ proceeds as follows:

- 1) Let $\kappa = \kappa(\lambda)$ be an upper bound on the number of random bits used by the adversary $\mathcal{A}$. Sample $\mathbf{r} \xleftarrow{\$} \{0, 1\}^\kappa$ and run $\mathcal{A}(1^\lambda; \mathbf{r})$.
- 2) The adversary $\mathcal{A}$ outputs a set of corrupt authorities $\mathcal{C}$, a set of non-corrupt authorities $\mathcal{N}$, a set of secret-key queries $\mathcal{Q} = \{(\text{gid}, A, \mathbf{v})\}$, a pair of challenge messages $\mathbf{u}_0, \mathbf{u}_1 \in \mathbb{Z}_q^n$ and a set of authorities A^* associated with the challenge ciphertext. These outputs correspond to the queries submitted by the adversary in the static security game for MA-ABNIPFE.
- 3) Let $\ell = |A^* \cap \mathcal{N}|$ and denote the set $A^* \cap \mathcal{N} = \{\text{aid}_1^*, \dots, \text{aid}_\ell^*\}$.
- 4) Partition the set $\mathcal{Q} = \mathcal{Q}_I \cup \mathcal{Q}_{II}$, where $\mathcal{Q}_I, \mathcal{Q}_{II}$ contain all secret-key queries of Type I and Type II, respectively. Let $|\mathcal{Q}_I| = Q$, $|\mathcal{Q}_{II}| = Q'$ and write

$$\mathcal{Q}_I = \{(\text{gid}_1, A_1, \mathbf{v}_1), \dots, (\text{gid}_Q, A_Q, \mathbf{v}_Q)\}, \quad \mathcal{Q}_{II} = \{(\text{gid}'_1, A'_1, \mathbf{v}'_1), \dots, (\text{gid}'_{Q'}, A'_{Q'}, \mathbf{v}'_{Q'})\}.$$

- 5) For each $i \in [\ell]$, let $N_i \in [Q]$ denote the number of Type I secret-key queries in which the challenge authority aid_i^* appears. Suppose that authority aid_i^* is contained in the set $A_{j_1^{(i)}}, \dots, A_{j_{N_i}^{(i)}}$ for some indices $j_1^{(i)}, \dots, j_{N_i}^{(i)} \in [Q]$, listed in increasing order. Define the mapping $\rho_i : [N_i] \rightarrow [Q]$ by setting $\rho_i(k) = j_k^{(i)}$. That is, aid_i^* appears exactly in the set $A_{\rho_i(1)}, \dots, A_{\rho_i(N_i)}$ with the indices ordered increasingly.
- 6) For each $i \in [Q]$, sample $\mathbf{r}_i \leftarrow D_{\mathbb{Z},\chi'}^{m'}$. For each $j \in [Q']$, sample $\mathbf{r}'_j \leftarrow D_{\mathbb{Z},\chi'}^{m'}$.
- 7) Finally, $\mathcal{S}_{\mathcal{A}}$ outputs

$$\begin{aligned} & 1^\ell, \{1^{N_i+Q'}\}_{i \in [\ell]}; \\ & (\mathbf{r}_{\rho_1(1)}, \mathbf{v}_{\rho_1(1)}), \dots, (\mathbf{r}_{\rho_1(N_1)}, \mathbf{v}_{\rho_1(N_1)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ & (\mathbf{r}_{\rho_2(1)}, \mathbf{v}_{\rho_2(1)}), \dots, (\mathbf{r}_{\rho_2(N_2)}, \mathbf{v}_{\rho_2(N_2)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ & \dots \\ & (\mathbf{r}_{\rho_\ell(1)}, \mathbf{v}_{\rho_\ell(1)}), \dots, (\mathbf{r}_{\rho_\ell(N_\ell)}, \mathbf{v}_{\rho_\ell(N_\ell)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ & \mathbf{u}_0, \mathbf{u}_1. \end{aligned}$$

We remark that the sampling algorithm $\mathcal{S}_A$ defined above does not explicitly guarantee that each tuple $(\mathbf{r}_i, \mathbf{v}_i)$ or $(\mathbf{r}'_j, \mathbf{v}'_j)$ is non-zero, as required in Assumption 4. However, since each $\mathbf{r}_i, \mathbf{r}'_j$ component in the tuple is sampled from discrete Gaussian distribution $D_{\mathbb{Z}, \chi'}^{m'}$, the probability that any of them equals the all-zero vector is *negligible*. Specifically, by a standard tail bound argument, we can upper bound the probability that a sample equals the all-zero vector as follows:

$$\begin{aligned} \Pr[\mathbf{r} = \mathbf{0}_{m'} : \mathbf{r} \leftarrow D_{\mathbb{Z}, \chi'}^{m'}] &= \frac{1}{\sum_{\mathbf{x} \in \mathbb{Z}^{m'}} e^{-\pi \|\mathbf{x}\|^2 / \chi'^2}} = \left(\frac{1}{\sum_{x \in \mathbb{Z}} e^{-\pi |x|^2 / \chi'^2}} \right)^{m'} \\ &< \left(\frac{1}{1 + 2e^{-\pi / \chi'^2}} \right)^{m'} \leq (1 - ce^{-\pi / \chi'^2})^{m'} \leq \exp(-c \cdot m' e^{-\pi / \chi'^2}), \end{aligned}$$

for some constant $c > 0$. When χ' is taken to be polynomial in m' , the probability is $e^{-\Omega(m')}$, which is negligible in m' . Therefore, we may safely ignore the negligible probability that the sampling algorithm outputs an all-zero tuple $(\mathbf{r}_i, \mathbf{v}_i)$ or $(\mathbf{r}'_j, \mathbf{v}'_j)$.

We now claim that for all efficient distinguishers $\mathcal{D}$, the advantage $\text{Adv}_{\mathcal{S}_A, \mathcal{D}}^{\text{Pre}}(\lambda)$ (cf. Assumption 4) for sampling algorithm $\mathcal{S}_A$ is negligible in λ . More precisely,

Claim 52. *Let Q_0 be an upper bound on the number of secret-key queries submitted by the adversary $\mathcal{A}$. Suppose that the following conditions hold:*

- $m' > 6n \log q$.
- Let χ' be an error distribution such that $\chi' = \Omega(\sqrt{n \log q})$.
- Let χ be an error distribution parameter such that $\chi \geq \lambda^{\omega(1)} \cdot \max\{B_1, \sqrt{\lambda} \chi_s\}$, where χ_s is an error parameter such that $\text{LWE}_{n, m_1, q, \chi_s}$ assumption holds for some $m_1 = \text{poly}(m, m', Q_0)$.

Then, for every efficient distinguisher $\mathcal{D}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, we have $\text{Adv}_{\mathcal{S}_A, \mathcal{D}}^{\text{Pre}}(\lambda) = \text{negl}(\lambda)$, where $\text{Adv}_{\mathcal{S}_A, \mathcal{D}}^{\text{Pre}}$ denotes the advantage of the distinguisher $\mathcal{D}$ in the IND-evIPFE assumption described in Assumption 4.

Assuming Claim 27, we proceed with the proof of Theorem 51. For clarity, the proof of Claim 27 is postponed to the end of this section.

Proof of Theorem 51 (Continued). To complete the proof, we show that if there exists an *efficient* adversary $\mathcal{A}$ that can distinguish between H_0 and H_1 with non-negligible advantage, then we can construct an *efficient* algorithm $\mathcal{B}$ that breaks the IND-evIPFE assumption with respect to the sampling algorithm $\mathcal{S}_A$. The algorithm $\mathcal{B}$ proceeds as follows:

- 1) Algorithm $\mathcal{B}$ receives an IND-evIPFE challenge

$$(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{y}_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{y}_2^\top, \{\mathbf{K}_i\}_{i \in [\ell]}),$$

where $\mathbf{r}_{\text{pub}} \in \{0, 1\}^*$, $\mathbf{A}_i \in \mathbb{Z}_q^{n \times m}$, $\mathbf{y}_{1,i} \in \mathbb{Z}_q^m$, $\mathbf{K}_i \in \mathbb{Z}_q^{m \times (N_i + Q')}$ for each $i \in [\ell]$, $\mathbf{B} \in \mathbb{Z}_q^{n\ell \times m'}$, $\mathbf{P} \in \mathbb{Z}_q^{n\ell \times m}$, $\mathbf{y}_2 \in \mathbb{Z}_q^{m' + m}$. Recall that in the public-coin model, $\mathbf{r}_{\text{pub}}$ contains all the randomness used by sampling algorithm $\mathcal{S}_A$.

- 2) For each $i \in [\ell]$, algorithm $\mathcal{B}$ parses the matrix $\mathbf{K}_i$ as

$$\mathbf{K}_i = [\mathbf{K}_i^{(1)} \mid \mathbf{K}_i^{(2)}],$$

where $\mathbf{K}_i^{(1)} \in \mathbb{Z}_q^{m \times N_i}$ and $\mathbf{K}_i^{(2)} \in \mathbb{Z}_q^{m \times Q'}$. Let $\mathbf{k}_{i,j}, \mathbf{k}'_{i,j}$ denote the j -th column vectors of $\mathbf{K}_i^{(1)}$ and $\mathbf{K}_i^{(2)}$, respectively.

- 3) Algorithm $\mathcal{B}$ runs algorithm $\mathcal{A}$, using the appropriate portion of the public randomness $\mathbf{r}_{\text{pub}}$ —specifically, the same portion used internally by $\mathcal{S}_A$ to simulate $\mathcal{A}$. The adversary $\mathcal{A}$ outputs the following queries:

- A set of corrupt authorities $\mathcal{C} \subseteq \mathcal{AU}$, along with their public keys $\text{pk}_{\text{aid}} = (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$ for all $\text{aid} \in \mathcal{C}$.

- A set of non-corrupt authorities $\mathcal{N} \subseteq \mathcal{AU}$, satisfying $\mathcal{N} \cap \mathcal{C} = \emptyset$.
- A pair of challenge messages $\mathbf{u}_0, \mathbf{u}_1 \in \mathbb{Z}_q^n$, and a ciphertext authority set $A^* \subseteq \mathcal{C} \cup \mathcal{N}$ such that $(A^* \cap \mathcal{C}) \subsetneq A^*$.
- A set of secret-key queries $\mathcal{Q}$, where each query $(\text{gid}, A, \mathbf{v})$ is such that $A \subseteq \mathcal{N}$ and satisfies exactly one of the following two conditions:
 - Type I: $(A_i \cup \mathcal{C}) \cap A^* \subsetneq A^*$.
 - Type II: $(A_i \cup \mathcal{C}) \cap A^* = A^*$ and $\|(\mathbf{u}_0 - \mathbf{u}_1)^\top \mathbf{v}_i\| \leq B_1$.

4) Algorithm $\mathcal{B}$ runs sampling algorithm $\mathcal{S}_{\mathcal{A}}(\text{gp}; \mathbf{r}_{\text{pub}})$, which outputs

$$\begin{aligned} & 1^\ell, \{1^{N_i+Q'}\}_{i \in [\ell]}; \\ & (\mathbf{r}_{\rho_1(1)}, \mathbf{v}_{\rho_1(1)}), \dots, (\mathbf{r}_{\rho_1(N_1)}, \mathbf{v}_{\rho_1(N_1)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ & (\mathbf{r}_{\rho_2(1)}, \mathbf{v}_{\rho_2(1)}), \dots, (\mathbf{r}_{\rho_2(N_2)}, \mathbf{v}_{\rho_2(N_2)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ & \dots; \\ & (\mathbf{r}_{\rho_\ell(1)}, \mathbf{v}_{\rho_\ell(1)}), \dots, (\mathbf{r}_{\rho_\ell(N_\ell)}, \mathbf{v}_{\rho_\ell(N_\ell)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}). \end{aligned}$$

Since the randomness used to simulate $\mathcal{A}$ in Step 3) comes from a specific portion of $\mathbf{r}_{\text{pub}}$, the values produced here—namely, the vectors $\mathbf{v}_1, \dots, \mathbf{v}_Q, \mathbf{v}'_1, \dots, \mathbf{v}'_{Q'}$ —align exactly with those generated during the simulation of $\mathcal{A}$ in Step 3). This ensures that algorithm $\mathcal{B}$'s internal simulation of $\mathcal{A}$ is consistent with the actual input instance of the IND-evIPFE challenge.

5) From the construction of $\mathcal{S}_{\mathcal{A}}$, we have that $\ell = |A^* \cap \mathcal{N}|$, i.e., the number of non-corrupt authorities appearing in the challenge ciphertext. Let $A^* \cap \mathcal{N} = \{\text{aid}_1^*, \dots, \text{aid}_\ell^*\}$. Algorithm $\mathcal{B}$ first sets $\mathbf{A}_{\text{aid}_i^*} \leftarrow \mathbf{A}_i$ for each $i \in [\ell]$, and parses $[\mathbf{B} \mid \mathbf{P}]$ as

$$[\mathbf{B} \mid \mathbf{P}] = \left[\begin{array}{c|c} \mathbf{B}_{\text{aid}_1^*} & \mathbf{P}_{\text{aid}_1^*} \\ \vdots & \vdots \\ \mathbf{B}_{\text{aid}_\ell^*} & \mathbf{P}_{\text{aid}_\ell^*} \end{array} \right] \in \mathbb{Z}_q^{n\ell \times (m'+m)},$$

where $\mathbf{B}_{\text{aid}_i^*} \in \mathbb{Z}_q^{n \times m'}$ and $\mathbf{P}_{\text{aid}_i^*} \in \mathbb{Z}_q^{n \times m}$ for each $\text{aid}_i^* \in A^* \cap \mathcal{N}$.

6) Algorithm $\mathcal{B}$ partitions the set of secret-key queries $\mathcal{Q} = \mathcal{Q}_I \cup \mathcal{Q}_{II}$, where $\mathcal{Q}_I, \mathcal{Q}_{II}$ contain all secret-key queries of Type I and Type II, respectively. Let $|\mathcal{Q}_I| = Q$, $|\mathcal{Q}_{II}| = Q'$, and denote

$$\mathcal{Q}_I = \{(\text{gid}_1, A_1, \mathbf{v}_1), \dots, (\text{gid}_Q, A_Q, \mathbf{v}_Q)\}, \quad \mathcal{Q}_{II} = \{(\text{gid}'_1, A'_1, \mathbf{v}'_1), \dots, (\text{gid}'_{Q'}, A'_{Q'}, \mathbf{v}'_{Q'})\}.$$

For each $i \in [Q]$, it partitions $A_i = A_{i,\text{chal}} \cup \bar{A}_{i,\text{chal}}$, where $A_{i,\text{chal}}$ consists of the authorities in A_i that appear in the ciphertext, i.e. $A_{i,\text{chal}} = A_i \cap A^*$.

7) Algorithm $\mathcal{B}$ initializes an empty table $\mathbf{T} : \mathcal{GID} \times \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q^{m'}$. This table will be used to store and consistently respond to all queries made to the random oracle during the experiment.

8) The algorithm $\mathcal{B}$ responds to the queries as follows:

• **Public keys for non-corrupt authorities:**

- For each $\text{aid}_i^* \in A^* \cap \mathcal{N}$, algorithm $\mathcal{B}$ sets $\text{pk}_{\text{aid}_i^*} \leftarrow (\mathbf{A}_{\text{aid}_i^*}, \mathbf{B}_{\text{aid}_i^*}, \mathbf{P}_{\text{aid}_i^*})$.
- For authorities $\text{aid} \in \mathcal{N} \setminus A^*$, algorithm $\mathcal{B}$ samples $(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, $\mathbf{B}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}$ and $\mathbf{P}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, and sets the public key $\text{pk}_{\text{aid}} \leftarrow (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$.

• **Secret keys:** The algorithm $\mathcal{B}$ responds to each secret-key query depending on its type:

- **Type I:** For a Type I secret-key query $(\text{gid}_k, A_k, \mathbf{v}_k)$, recall that A_k is partitioned as $A_k = A_{k,\text{chal}} \cup \bar{A}_{k,\text{chal}}$, where $A_{k,\text{chal}} = A_k \cap A^*$.
 - * For each $\text{aid}_i^* \in A^* \cap \mathcal{N}$, recall that the number of secret-key queries involving aid_i^* is the parameter N_i . Let $\rho(\cdot)$ be the index mapping previously defined in the proof of Lemma 51. For each $j \in [N_i]$, set $\text{sk}_{\text{aid}_i^*, \text{gid}_{\rho_i(j)}, \mathbf{v}_{\rho_i(j)}} \leftarrow \mathbf{k}_{i,j}$. Then the algorithm $\mathcal{B}$ checks if the table $\mathbf{T}$ has ever recorded the image of $(\text{gid}_{\rho_i(j)}, \mathbf{v}_{\rho_i(j)})$, if not, store the mapping $(\text{gid}_{\rho_i(j)}, \mathbf{v}_{\rho_i(j)}) \mapsto \mathbf{r}_{\rho_i(j)}$ to the table.

- * For each $k \in [Q]$, if $A_{k,\text{chal}} = \emptyset$, then sample $\mathbf{r}_{\text{gid}_k, \mathbf{v}_k} \leftarrow D_{\mathbb{Z}, \chi'}^{m'}$, and add the mapping $(\text{gid}_k, \mathbf{v}_k) \mapsto \mathbf{r}_{\text{gid}_k, \mathbf{v}_k}$ to the table. At this point, the table contains the image of all pairs $(\text{gid}_k, \mathbf{v}_k)$ for each $k \in [Q]$.
- * For each $k \in [Q]$, for each $\text{aid} \in \bar{A}_{k,\text{chal}}$, compute

$$\text{sk}_{\text{aid}, \text{gid}_k, \mathbf{v}_k} \leftarrow \text{SamplePre}(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}, \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}_k) + \mathbf{B}_{\text{aid}} \mathbf{H}(\text{gid}_k, \mathbf{v}_k)),$$

where the value $\mathbf{H}(\text{gid}_k, \mathbf{v}_k)$ is retrieved from the table $\mathbb{T}$.

- **Type II:** For a Type II query $(\text{gid}'_j, A'_j, \mathbf{v}'_j)$, recall that $A^* \cap \mathcal{N} = \{\text{aid}_1^*, \dots, \text{aid}_\ell^*\} \subseteq A'_j$.
 - * For each $i \in [\ell], j \in [Q']$, set $\text{sk}_{\text{aid}_i^*, \text{gid}'_j, \mathbf{v}'_j} \leftarrow \mathbf{k}'_{i,j} \in \mathbb{Z}_q^m$. Next, algorithm $\mathcal{B}$ adds the mapping $(\text{gid}'_j, \mathbf{v}'_j) \mapsto \mathbf{r}'_j$ to the table $\mathbb{T}$.
 - * For each $\text{aid} \in A'_j \setminus A^*$, algorithm $\mathcal{B}$ computes

$$\text{sk}_{\text{aid}, \text{gid}'_j, \mathbf{v}'_j} \leftarrow \text{SamplePre}(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}, \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}'_j) + \mathbf{B}_{\text{aid}} \mathbf{r}'_j, \chi)$$

efficiently.

- **Challenge ciphertext:** Algorithm $\mathcal{B}$ parses $\mathbf{y}_2$ as $\mathbf{y}_2^\top = [\hat{\mathbf{y}}_2^\top \mid \hat{\mathbf{y}}_3^\top]$ where $\hat{\mathbf{y}}_2 \in \mathbb{Z}_q^{m'}$, $\hat{\mathbf{y}}_3 \in \mathbb{Z}_q^m$. It constructs the ciphertext as follows: For each $\text{aid} \in A^* \cap \mathcal{C}$, sample $\mathbf{s}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^n$ and $\mathbf{e}_{1,\text{aid}} \leftarrow D_{\mathbb{Z}, \chi}^m$, then set $\mathbf{c}_{1,\text{aid}}^\top \leftarrow \mathbf{s}_{\text{aid}}^\top \mathbf{A}_{\text{aid}} + \mathbf{e}_{1,\text{aid}}^\top \in \mathbb{Z}_q^m$. For $\text{aid}_i^* \in A^* \cap \mathcal{N}$, set $\mathbf{c}_{1,\text{aid}_i^*} \leftarrow \mathbf{y}_{1,i}$. Finally, algorithm $\mathcal{B}$ constructs the ciphertext

$$\text{ct} \leftarrow \left(\{\mathbf{c}_{1,\text{aid}}^\top\}_{\text{aid} \in A^*}, \sum_{\text{aid} \in A^* \cap \mathcal{C}} \mathbf{s}_{\text{aid}}^\top \mathbf{B}_{\text{aid}} + \hat{\mathbf{y}}_2^\top, \sum_{\text{aid} \in A^* \cap \mathcal{N}} \mathbf{s}_{\text{aid}}^\top \mathbf{P}_{\text{aid}} + \hat{\mathbf{y}}_3^\top \right).$$

- **Random oracle queries:** Upon receiving a query $(\text{gid}, \mathbf{v}) \in \mathcal{GID} \times \mathbb{Z}_q^n$, algorithm $\mathcal{B}$ checks whether the input $(\text{gid}, \mathbf{v})$ has been queried before—either during the adversary's direct random oracle queries or implicitly through processing the secret-key queries. If so, algorithm $\mathcal{B}$ retrieves and responds with the stored value $\mathbf{r}_{\text{gid}, \mathbf{v}}$ from the table $\mathbb{T}$. If the input is new, the challenger samples $\mathbf{r}_{\text{gid}, \mathbf{v}} \mapsto D_{\mathbb{Z}, \chi}^{m'}$, records the mapping $(\text{gid}, \mathbf{v}) \mapsto \mathbf{r}_{\text{gid}, \mathbf{v}}$ to the table $\mathbb{T}$, and then replies with $\mathbf{r}_{\text{gid}, \mathbf{v}}$.

9) Finally, algorithm $\mathcal{B}$ outputs whatever algorithm $\mathcal{A}$ outputs.

The distributions of the public keys for non-corrupt authorities are exactly the same as those in $\mathbf{H}_0$ and $\mathbf{H}_1$, as they are uniformly generated. We now examine the responses to the secret-key queries:

- For each $\text{aid}_i^* \in A^* \cap \mathcal{N}$, we have

$$\begin{aligned} \mathbf{k}_{i,j} &\leftarrow (\mathbf{A}_{\text{aid}_i^*})_\chi^{-1}(\mathbf{P}_{\text{aid}_i^*} \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \mathbf{B}_{\text{aid}_i^*} \mathbf{r}_{\rho_i(j)}), \quad \text{for all } j \in [N_i], \\ \mathbf{k}'_{i,j} &\leftarrow (\mathbf{A}_{\text{aid}_i^*})_\chi^{-1}(\mathbf{P}_{\text{aid}_i^*} \mathbf{G}^{-1}(\mathbf{v}'_j) + \mathbf{B}_{\text{aid}_i^*} \mathbf{r}'_j), \quad \text{for all } j \in [Q']. \end{aligned}$$

This perfectly matches the distribution of $\text{sk}_{\text{aid}_i^*, \text{gid}_{\rho_i(j)}, \mathbf{v}_{\rho_i(j)}}$ and $\text{sk}_{\text{aid}_i^*, \text{gid}'_j, \mathbf{v}'_j}$ in the actual game, respectively.

- For each $\text{aid} \in A_j \setminus A^*$ for some $j \in [Q]$, the secret key $\text{sk}_{\text{aid}, \text{gid}_j, \mathbf{v}_j}$ is generated using SamplePre , identical to the procedure in $\mathbf{H}_0$ and $\mathbf{H}_1$. The same argument also applies to $\text{aid} \in A'_j \setminus A^*$ for $j \in [Q']$.

Finally, we analyze the distribution of the challenge ciphertext. Observe that for each $i \in [\ell]$, $\mathbf{y}_{1,i}^\top = \mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top$ and $\mathbf{y}_2^\top = \mathbf{s}^\top [\mathbf{B} \mid \mathbf{P}] + \mathbf{e}_2^\top + [\mathbf{0}_{m'}^\top \mid \mathbf{u}_b^\top \mathbf{G}]$ for some $\mathbf{s}^\top = [\mathbf{s}_1^\top \mid \dots \mid \mathbf{s}_\ell^\top] \xleftarrow{\$} \mathbb{Z}_q^{n\ell}$, and we write $\mathbf{e}_2^\top$ as $[\hat{\mathbf{e}}_2^\top \mid \hat{\mathbf{e}}_3^\top]$ where $\hat{\mathbf{e}}_2 \in \mathbb{Z}_q^{m'}$, $\hat{\mathbf{e}}_3 \in \mathbb{Z}_q^m$. Then

$$\begin{aligned} \mathbf{c}_{1,\text{aid}_i^*}^\top &= \mathbf{z}_{1,i}^\top = \mathbf{s}_i^\top \mathbf{A}_{\text{aid}_i^*} + \mathbf{e}_{1,i}^\top, \quad \text{for each } \text{aid}_i^* \in A^* \cap \mathcal{N}, \\ \hat{\mathbf{y}}_2^\top &= \sum_{\text{aid}_i^* \in A^* \cap \mathcal{N}} \mathbf{s}_i^\top \mathbf{B}_{\text{aid}_i^*} + \hat{\mathbf{e}}_2^\top, \\ \hat{\mathbf{y}}_3^\top &= \sum_{\text{aid}_i^* \in A^* \cap \mathcal{N}} \mathbf{s}_i^\top \mathbf{P}_{\text{aid}_i^*} + \hat{\mathbf{e}}_3^\top + \mathbf{u}_b^\top \mathbf{G}. \end{aligned}$$

From the specification of algorithm $\mathcal{B}$, the resulting ciphertext is constructed as:

$$\begin{aligned} \mathbf{c}_{1,\text{aid}_i^*}^\top &= \mathbf{s}_i^\top \mathbf{A}_{\text{aid}_i^*} + \mathbf{e}_{1,i}^\top, & \text{for each } \text{aid}_i^* \in A^* \cap \mathcal{N}, \\ \mathbf{c}_{1,\text{aid}}^\top &= \mathbf{s}_{\text{aid}}^\top \mathbf{A}_{\text{aid}} + \mathbf{e}_{1,\text{aid}}^\top, & \text{for each } \text{aid} \in A^* \cap \mathcal{C}, \\ \mathbf{c}_2^\top &= \sum_{\text{aid} \in A^* \cap \mathcal{C}} \mathbf{s}_{\text{aid}}^\top \mathbf{B}_{\text{aid}} + \sum_{\text{aid}_i^* \in A^* \cap \mathcal{N}} \mathbf{s}_i^\top \mathbf{B}_{\text{aid}_i^*} + \hat{\mathbf{e}}_2^\top, \\ \mathbf{c}_3^\top &= \sum_{\text{aid} \in A^* \cap \mathcal{C}} \mathbf{s}_{\text{aid}}^\top \mathbf{P}_{\text{aid}} + \sum_{\text{aid}_i^* \in A^* \cap \mathcal{N}} \mathbf{s}_i^\top \mathbf{P}_{\text{aid}_i^*} + \hat{\mathbf{e}}_3^\top + \mathbf{u}_b^\top \mathbf{G}. \end{aligned}$$

Therefore, the full ciphertext generated by $\mathcal{B}$ is identically distributed to that in the experiment $\mathbf{H}_b$.

Therefore, the advantage $\text{Adv}_{\mathcal{S}_A, \mathcal{B}}^{\text{Post}}$ in the IND-evIPFE assumption with respect to the sampling algorithm $\mathcal{S}_A$ is non-negligible. By the IND-evIPFE assumption, this implies the existence of another efficient algorithm $\mathcal{B}'$, such that the advantage $\text{Adv}_{\mathcal{S}_A, \mathcal{B}'}^{\text{Pre}}$ is also non-negligible, thereby contradicting the indistinguishability guarantee asserted in Claim 52. Hence, under the IND-evIPFE assumption, no efficient adversary can distinguish between $\mathbf{H}_0$ and $\mathbf{H}_1$ with non-negligible advantage. This concludes the proof of Theorem 51. $\square$

We now provide the proof of Claim 52, which was used in the proof of Theorem 51.

Proof of Claim 52. We prove the claim via a sequence of hybrid games, gradually transitioning from the real IND-evIPFE experiment to a game independent of the challenge bit b . In the following, we define a series of hybrid experiments $\mathbf{H}_{0,b}^{\text{Pre}}, \mathbf{H}_{1,b}^{\text{Pre}}, \mathbf{H}_{2,b}^{\text{Pre}}, \mathbf{H}_{3,b}^{\text{Pre}}$ for $b \in \{0, 1\}$, where $\mathbf{H}_{0,b}^{\text{Pre}}$ corresponds to the original IND-evIPFE game, and $\mathbf{H}_{3,b}^{\text{Pre}}$ is constructed to be independent of b . By showing that each adjacent pair of hybrids is indistinguishable, we can conclude that $\mathbf{H}_{0,0}^{\text{Pre}} \stackrel{c}{\approx} \mathbf{H}_{0,1}^{\text{Pre}}$, thereby completing the proof.

Game $\mathbf{H}_{0,b}^{\text{Pre}}$. This experiment corresponds to the real game in the IND-evIPFE assumption. Specifically, the challenger proceeds as follows:

- 1) Let $\kappa = \kappa(\lambda)$ be an upper bound on the number of random bits used by the adversary $\mathcal{S}_A$. Sample $\mathbf{r}_{\text{pub}} \xleftarrow{\$} \{0, 1\}^\kappa$.
- 2) Run the sampling algorithm $\mathcal{S}_A(\text{gp}; \mathbf{r}_{\text{pub}})$ which outputs

$$\begin{aligned} &1^\ell, \{1^{N_i+Q'}\}_{i \in [\ell]}; \\ &(\mathbf{r}_{\rho_1(1)}, \mathbf{v}_{\rho_1(1)}), \dots, (\mathbf{r}_{\rho_1(N_1)}, \mathbf{v}_{\rho_1(N_1)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ &(\mathbf{r}_{\rho_2(1)}, \mathbf{v}_{\rho_2(1)}), \dots, (\mathbf{r}_{\rho_2(N_2)}, \mathbf{v}_{\rho_2(N_2)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ &\dots \\ &(\mathbf{r}_{\rho_\ell(1)}, \mathbf{v}_{\rho_\ell(1)}), \dots, (\mathbf{r}_{\rho_\ell(N_\ell)}, \mathbf{v}_{\rho_\ell(N_\ell)}), (\mathbf{r}'_1, \mathbf{v}'_1), \dots, (\mathbf{r}'_{Q'}, \mathbf{v}'_{Q'}); \\ &\mathbf{u}_0, \mathbf{u}_1. \end{aligned}$$

- 3) Sample $\mathbf{B} \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}$, $\mathbf{P} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, and parse the matrices

$$\mathbf{B} = \begin{pmatrix} \mathbf{B}_1 \\ \vdots \\ \mathbf{B}_\ell \end{pmatrix}, \quad \mathbf{P} = \begin{pmatrix} \mathbf{P}_1 \\ \vdots \\ \mathbf{P}_\ell \end{pmatrix},$$

where $\mathbf{B}_i \in \mathbb{Z}_q^{n \times m'}$, $\mathbf{P}_i \in \mathbb{Z}_q^{n \times m}$ for each $i \in [\ell]$.

- 4) For each $i \in [\ell]$, define:

$$\mathbf{Q}_i^{(1)} \leftarrow [\mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(1)}) + \mathbf{B}_i \mathbf{r}_{\rho_i(1)} \mid \dots \mid \mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(N_i)}) + \mathbf{B}_i \mathbf{r}_{\rho_i(N_i)}] \in \mathbb{Z}_q^{n \times N_i}.$$

and similarly define

$$\mathbf{Q}_i^{(2)} = [\mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}'_1) + \mathbf{B}_i \mathbf{r}'_1 \mid \dots \mid \mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}'_{Q'}) + \mathbf{B}_i \mathbf{r}'_{Q'}] \in \mathbb{Z}_q^{n \times Q'}.$$

Set $\mathbf{Q}_i = [\mathbf{Q}_i^{(1)} \mid \mathbf{Q}_i^{(2)}] \in \mathbb{Z}_q^{n \times (N_i + Q')}$. The construction of each matrix $\mathbf{Q}_i$ matches that specified in Assumption 4.

- 5) Then the challenger samples $\mathbf{s}_1, \dots, \mathbf{s}_\ell \xleftarrow{\$} \mathbb{Z}_q^n$ and sets $\mathbf{s}^\top = [\mathbf{s}_1^\top \mid \dots \mid \mathbf{s}_\ell^\top] \in \mathbb{Z}_q^{n\ell}$. For each $i \in [\ell]$, it samples $\mathbf{e}_{1,i} \leftarrow D_{\mathbb{Z}, \chi}^m, \mathbf{e}_{3,i} \leftarrow D_{\mathbb{Z}, \chi}^{N_i + Q'}$. Then it samples $\mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^{m' + m}$.
- 6) Next, the challenger samples $(\mathbf{A}_1, \text{td}_1), \dots, (\mathbf{A}_\ell, \text{td}_\ell) \leftarrow \text{TrapGen}(1^n, 1^m, q)$. It computes the following values :
 - $\mathbf{z}_{1,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top \in \mathbb{Z}_q^m$ for each $i \in [\ell]$.
 - $\mathbf{z}_2^\top \leftarrow \mathbf{s}^\top [\mathbf{B} \mid \mathbf{P}] + \mathbf{e}_2^\top + [\mathbf{0}_{m'}^\top \mid \mathbf{u}_b^\top \mathbf{G}] = \left[\sum_{i \in [\ell]} \mathbf{s}_i^\top \mathbf{B}_i \mid \sum_{i \in [\ell]} \mathbf{s}_i^\top \mathbf{P}_i + \mathbf{u}_b^\top \mathbf{G} \right] + \mathbf{e}_2^\top \in \mathbb{Z}_q^{m' + m}$.
 - $\mathbf{z}_{3,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{Q}_i + \mathbf{e}_{3,i}^\top \in \mathbb{Z}_q^{N_i + Q'}$ for each $i \in [\ell]$.

The component $\mathbf{z}_{3,i}$ and $\mathbf{e}_{3,i}$ are parsed as

$$\mathbf{z}_{3,i}^\top = [\mathbf{z}_{3,i}^{(1)\top} \mid \mathbf{z}_{3,i}^{(2)\top}] \in \mathbb{Z}_q^{N_i + Q'}, \quad \mathbf{e}_{3,i}^\top = [\mathbf{e}_{3,i}^{(1)\top} \mid \mathbf{e}_{3,i}^{(2)\top}] \in \mathbb{Z}_q^{N_i + Q'},$$

where $\mathbf{z}_{3,i}^{(1)}, \mathbf{e}_{3,i}^{(1)} \in \mathbb{Z}_q^{N_i}$ and $\mathbf{z}_{3,i}^{(2)}, \mathbf{e}_{3,i}^{(2)} \in \mathbb{Z}_q^{Q'}$. For clarity, define $\mathbf{t}_{i,j} = \mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \mathbf{B}_i \mathbf{r}_{\rho_i(j)} \in \mathbb{Z}_q^n$ and $y_{i,j} = \mathbf{s}_i^\top \mathbf{t}_{i,j} \in \mathbb{Z}_q$ for each $i \in [\ell]$ and $j \in [N_i]$. Similarly define $\mathbf{t}'_{i,j} = \mathbf{P}_i \mathbf{G}^{-1}(\mathbf{v}'_j) + \mathbf{B}_i \mathbf{r}'_j \in \mathbb{Z}_q^n$ and $y'_{i,j} = \mathbf{s}_i^\top \mathbf{t}'_{i,j} \in \mathbb{Z}_q$ for each $i \in [\ell]$ and $j \in [Q']$.

In summary, we obtain

$$\begin{aligned} \mathbf{z}_{3,i}^{(1)\top} &= \mathbf{s}_i^\top \mathbf{Q}_i^{(1)} + \mathbf{e}_{3,i}^{(1)\top} = [\mathbf{s}_i^\top \mathbf{t}_{i,1} \mid \dots \mid \mathbf{s}_i^\top \mathbf{t}_{i,N_i}] + \mathbf{e}_{3,i}^{(1)\top} = [y_{i,1} \mid \dots \mid y_{i,N_i}] + \mathbf{e}_{3,i}^{(1)\top} \in \mathbb{Z}_q^{N_i}, \\ \mathbf{z}_{3,i}^{(2)\top} &= \mathbf{s}_i^\top \mathbf{Q}_i^{(2)} + \mathbf{e}_{3,i}^{(2)\top} = [\mathbf{s}_i^\top \mathbf{t}'_{i,1} \mid \dots \mid \mathbf{s}_i^\top \mathbf{t}'_{i,Q'}] + \mathbf{e}_{3,i}^{(2)\top} = [y'_{i,1} \mid \dots \mid y'_{i,Q'}] + \mathbf{e}_{3,i}^{(2)\top} \in \mathbb{Z}_q^{Q'}. \end{aligned}$$

- 7) The challenger sends the tuple $(1^\lambda, \mathbf{r}_{\text{pub}}, \{(\mathbf{A}_i, \mathbf{z}_{1,i}^\top)\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{z}_2^\top, \{\mathbf{z}_{3,i}^\top\}_{i \in [\ell]})$ to the distinguisher $\mathcal{D}$.
- 8) The distinguisher $\mathcal{D}$ outputs a bit $\hat{b} \in \{0, 1\}$, which is taken as the output of the experiment.

Game $\text{H}_{1,b}^{\text{Pre}}$. The experiment is identical to $\text{H}_{0,b}^{\text{Pre}}$, except for how the values $\mathbf{z}_{1,i}, \mathbf{z}_2, \mathbf{z}_{3,i}$ for each $i \in [\ell]$ are generated. The changes are as follows:

- $\mathbf{z}_{1,i}$: For each $i \in [\ell]$, sample $\tilde{\mathbf{e}}_{1,i} \leftarrow D_{\mathbb{Z}, \chi_s}^m$, and compute $\mathbf{z}_{1,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{A}_i + \tilde{\mathbf{e}}_{1,i}^\top + \mathbf{e}_{1,i}^\top$.
- $\mathbf{z}_2$: For each $i \in [\ell]$, sample $\tilde{\mathbf{e}}_{2,i} \leftarrow D_{\mathbb{Z}, \chi_s}^{m'}$ and $\tilde{\mathbf{e}}'_{2,i} \leftarrow D_{\mathbb{Z}, \chi_s}^m$, then compute $\tilde{\mathbf{z}}_{2,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{B}_i + \tilde{\mathbf{e}}_{2,i}^\top$ and $\tilde{\mathbf{z}}'_{2,i} \leftarrow \mathbf{s}_i^\top \mathbf{P}_i + \tilde{\mathbf{e}}'_{2,i}$. Finally set

$$\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{\mathbf{z}}'_{2,i} + \mathbf{u}_b^\top \mathbf{G} \right] + \mathbf{e}_2^\top = \left[\sum_{i \in [\ell]} (\mathbf{s}_i^\top \mathbf{B}_i + \tilde{\mathbf{e}}_{2,i}^\top) \mid \sum_{i \in [\ell]} (\mathbf{s}_i^\top \mathbf{P}_i + \tilde{\mathbf{e}}'_{2,i}) + \mathbf{u}_b^\top \mathbf{G} \right] + \mathbf{e}_2^\top.$$

- $\mathbf{z}_{3,i}^{(1)}$: For each $i \in [\ell]$ and $j \in [N_i]$, compute

$$\begin{aligned} y_{i,j} &\leftarrow (\mathbf{s}_i^\top \mathbf{P}_i + \tilde{\mathbf{e}}'_{2,i}) \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + (\mathbf{s}_i^\top \mathbf{B}_i + \tilde{\mathbf{e}}_{2,i}^\top) \mathbf{r}_{\rho_i(j)} \\ &= \tilde{\mathbf{z}}'_{2,i} \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}_{\rho_i(j)}, \end{aligned}$$

and set

$$\mathbf{z}_{3,i}^{(1)\top} \leftarrow [y_{i,1} \mid \dots \mid y_{i,N_i}] + \mathbf{e}_{3,i}^{(1)\top} \in \mathbb{Z}_q^{N_i}.$$

- $\mathbf{z}_{3,i}^{(2)}$: For each $i \in [\ell]$ and $j \in [Q']$, compute

$$\begin{aligned} y'_{i,j} &\leftarrow (\mathbf{s}_i^\top \mathbf{P}_i + \tilde{\mathbf{e}}'_{2,i}) \mathbf{G}^{-1}(\mathbf{v}'_j) + (\mathbf{s}_i^\top \mathbf{B}_i + \tilde{\mathbf{e}}_{2,i}^\top) \mathbf{r}'_j \\ &= \tilde{\mathbf{z}}'_{2,i} \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}'_j, \end{aligned}$$

and set

$$\mathbf{z}_{3,i}^{(2)\top} \leftarrow [y'_{i,1} \mid \cdots \mid y'_{i,Q'}] + \mathbf{e}_{3,i}^{(2)\top} \in \mathbb{Z}_q^{Q'}.$$

Game $H_{2,b}^{\text{Pre}}$. The experiment is identical to $H_{1,b}^{\text{Pre}}$, except for how the values $\mathbf{z}_{1,i}, \mathbf{z}_2, \mathbf{z}_{3,i}$ for each $i \in [\ell]$ are generated.

- $\mathbf{z}_{1,i}$: For each $i \in [\ell]$, sample $\boxed{\mathbf{z}_{1,i} \xleftarrow{\$} \mathbb{Z}_q^m}$.
- $\mathbf{z}_2$: For each $i \in [\ell]$, sample $\boxed{\tilde{\mathbf{z}}_{2,i} \xleftarrow{\$} \mathbb{Z}_q^{m'}}$ and $\boxed{\tilde{\mathbf{z}}'_{2,i} \xleftarrow{\$} \mathbb{Z}_q^m}$. Then set

$$\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{\mathbf{z}}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{\mathbf{z}}'_{2,i}^\top + \mathbf{u}_b^\top \mathbf{G} \right] + \mathbf{e}_2^\top.$$

- $\mathbf{z}_{3,i}^{(1)}$: For each $i \in [\ell]$ and $j \in [N_i]$, compute

$$y_{i,j} \leftarrow \tilde{\mathbf{z}}_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}_{\rho_i(j)}) + \tilde{\mathbf{z}}'_{2,i}^\top \mathbf{r}_{\rho_i(j)},$$

then set

$$\mathbf{z}_{3,i}^{(1)\top} \leftarrow [y_{i,1} \mid \cdots \mid y_{i,N_i}] + \mathbf{e}_{3,i}^{(1)\top} \in \mathbb{Z}_q^{N_i}.$$

- $\mathbf{z}_{3,i}^{(2)}$: For each $i \in [\ell]$ and $j \in [Q']$, compute

$$y'_{i,j} \leftarrow \tilde{\mathbf{z}}'_{2,i}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{\mathbf{z}}_{2,i}^\top \mathbf{r}'_j,$$

then set

$$\mathbf{z}_{3,i}^{(2)\top} \leftarrow [y'_{i,1} \mid \cdots \mid y'_{i,Q'}] + \mathbf{e}_{3,i}^{(2)\top} \in \mathbb{Z}_q^{Q'}.$$

Game $H_{3,b}^{\text{Pre}}$. The experiment is identical to $H_{2,b}^{\text{Pre}}$, except it samples $\boxed{\mathbf{z}_{3,1}^{(1)} \xleftarrow{\$} \mathbb{Z}_q^{N_1}}$.

Lemma 53. Suppose that $\chi \geq \lambda^{\omega(1)} \cdot (\sqrt{\lambda}(m + \ell)\chi_s + \lambda m' \chi' \chi_s)$. Then we have $H_{0,b}^{\text{Pre}} \stackrel{s}{\approx} H_{1,b}^{\text{Pre}}$.

Proof. The claim follows directly from a standard noise smudging argument, as in the proof of Lemma 31. $\square$

Lemma 54. Suppose that the assumption $\text{LWE}_{n,m_1,q,\chi_s}$ holds for $m_1 = \text{poly}(m, m')$. We have $H_{1,b}^{\text{Pre}} \stackrel{c}{\approx} H_{2,b}^{\text{Pre}}$ for $b \in \{0, 1\}$.

Proof. The claim follows directly from a standard application of the LWE indistinguishability argument, as in the proof of Lemma 32. $\square$

Lemma 55. Let $m' > 6n \log q$ and $\chi' = \Omega(\sqrt{n \log q})$. Suppose that χ_s is an error parameter such that $\chi \geq \lambda^{\omega(1)} \cdot \sqrt{\lambda} \chi_s$, and that the $\text{LWE}_{n,m_1,q,\chi_s}$ assumption holds for $m_1 = \text{poly}(Q_0)$, where Q_0 is the upper bound of the number of secret-key queries submitted by the adversary. We have that $H_{2,b}^{\text{Pre}} \stackrel{c}{\approx} H_{3,b}^{\text{Pre}}$.

Proof. This claim follows essentially the same argument as in the proof of Lemma 33. Specifically, the only syntactic difference between the security model of the MA-ABNIPFE scheme and that of the MA-ABeIPFE scheme lies in whether the Type II secret-key queries are determined solely by the adversary or jointly with the challenger. However, in the proof of $H_{2,b}^{\text{Pre}} \stackrel{c}{\approx} H_{3,b}^{\text{Pre}}$ (and likewise $H_2^{\text{Pre}} \stackrel{c}{\approx} H_3^{\text{Pre}}$ in Lemma 33), the distribution of the components involving Type II secret-key queries remains unchanged throughout the transition. Therefore, the indistinguishability follows directly from the argument in Lemma 33. $\square$

Lemma 56. Suppose that $\chi > \lambda^{\omega(1)} \cdot B_1$. Then we have $H_{3,0}^{\text{Pre}} \stackrel{s}{\approx} H_{3,1}^{\text{Pre}}$.

Proof. We prove this by defining an intermediate hybrid experiment $H_{3,b,\text{mid}}^{\text{Pre}}$ between $H_{3,0}^{\text{Pre}}$ and $H_{3,1}^{\text{Pre}}$.

Game $H_{3,b,\text{mid}}^{\text{Pre}}$. The game is identical to $H_{3,b}^{\text{Pre}}$, except for how the challenger samples $\tilde{z}'_{2,1}$. Specifically:

- z_2 : The challenger first samples $\tilde{z}'_{2,1} \xleftarrow{s} \mathbb{Z}_q^m$, and sets $\tilde{z}_{2,1}^\top \leftarrow \tilde{z}'_{2,1} - \mathbf{u}_b^\top \mathbf{G}$. Then it computes

$$\mathbf{z}_2^\top \leftarrow \left[\sum_{i \in [\ell]} \tilde{z}_{2,i}^\top \mid \sum_{i \in [\ell]} \tilde{z}'_{2,i}^\top + \mathbf{u}_b^\top \mathbf{G} \right] + \mathbf{e}_2^\top = \left[\sum_{i \in [\ell]} \tilde{z}_{2,i}^\top \mid \tilde{z}'_{2,1} + \sum_{2 \leq i \leq \ell} \tilde{z}'_{2,i}^\top \right] + \mathbf{e}_2^\top.$$

- $z_{3,1}^{(2)}$: In particular, we have the affected component

$$y'_{1,j} = \tilde{z}_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{z}_{2,1}^\top \mathbf{r}'_j = \tilde{z}'_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{z}_{2,1}^\top \mathbf{r}'_j - \mathbf{u}_b^\top \mathbf{v}'_j.$$

Claim 57. For $b \in \{0, 1\}$, the two experiments $H_{3,b}^{\text{Pre}}$ and $H_{3,b,\text{mid}}^{\text{Pre}}$ are identical, i.e., $H_{3,b}^{\text{Pre}} \equiv H_{3,b,\text{mid}}^{\text{Pre}}$.

Proof. The only difference between Game $H_{3,b}^{\text{Pre}}$ and Game $H_{3,b,\text{mid}}^{\text{Pre}}$ lies in how the challenger samples $\tilde{z}'_{2,1}$. In Game $H_{3,b,\text{mid}}^{\text{Pre}}$, $\tilde{z}'_{2,1}$ is generated by first sampling an independent uniform vector $\tilde{z}'_{2,1} \leftarrow \mathbb{Z}_q^m$ and then shifted by $-\mathbf{u}_b^\top \mathbf{G}$, the resulting $\tilde{z}_{2,1}$ is uniform and independent of other components, which matches exactly the distribution of $\tilde{z}_{2,1}$ in $H_{3,b}^{\text{Pre}}$. $\square$

Claim 58. Suppose that $\chi \geq \lambda^{\omega(1)} \cdot B_1$. Then we have $H_{3,0,\text{mid}}^{\text{Pre}} \stackrel{s}{\approx} H_{3,1,\text{mid}}^{\text{Pre}}$.

Proof. Since in experiment $H_{3,b,\text{mid}}^{\text{Pre}}$, the components $z_{1,i}, z_2, z_{3,i}$ ($i \neq 1$) are sampled independently of the choice of the challenge bit b , it suffices to consider the distribution of $z_{3,1}$ in the two experiments. In $H_{3,0,\text{mid}}^{\text{Pre}}$, $\mathbf{z}_{3,1}^{(2)\top} = [y'_{1,1} \mid \cdots \mid y'_{1,Q'}] + \mathbf{e}_{3,1}^{(2)\top}$. Specifically, for each $j \in [Q']$, the j -th entry of $\mathbf{z}_{3,1}^{(2)}$ is given by

$$\begin{aligned} y'_{1,j} + e_{3,1,j}^{(2)} &= \tilde{z}_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{z}_{2,1}^\top \mathbf{r}'_j + e_{3,1,j}^{(2)} \\ &= (\tilde{z}'_{2,1} - \mathbf{u}_0^\top \mathbf{G}) \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{z}'_{2,1}^\top \mathbf{r}'_j + e_{3,1,j}^{(2)} \\ &= \tilde{z}'_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{z}'_{2,1}^\top \mathbf{r}'_j + e_{3,1,j}^{(2)} - \mathbf{u}_0^\top \mathbf{v}'_j \\ &= \tilde{z}'_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{z}'_{2,1}^\top \mathbf{r}'_j + e_{3,1,j}^{(2)} - (\mathbf{u}_1^\top + (\mathbf{u}_0 - \mathbf{u}_1)^\top) \mathbf{v}'_j \\ &\stackrel{s}{\approx} \tilde{z}'_{2,1}^\top \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{z}'_{2,1}^\top \mathbf{r}'_j + e_{3,1,j}^{(2)} - \mathbf{u}_1^\top \mathbf{v}'_j \\ &= (\tilde{z}'_{2,1} - \mathbf{u}_1^\top \mathbf{G}) \mathbf{G}^{-1}(\mathbf{v}'_j) + \tilde{z}'_{2,1}^\top \mathbf{r}'_j + e_{3,1,j}^{(2)}. \end{aligned} \tag{9}$$

The approximate identity holds due to the following reason: for each Type II secret-key query $(\text{gid}'_j, A'_j, \mathbf{v}'_j)$, we have $\|(\mathbf{u}_0 - \mathbf{u}_1)^\top \mathbf{v}'_j\| \leq B_1$ by the bounded inner product condition. Since $\chi \geq \lambda^{\omega(1)} \cdot B_1$, Lemma 5 implies that the noise term statistically hides the difference, i.e.,

$$(\mathbf{u}_0 - \mathbf{u}_1)^\top \mathbf{v}'_j + e_{3,1,j}^{(2)} \stackrel{s}{\approx} e_{3,1,j}^{(2)}.$$

Moreover, the expression (9) matches exactly the distribution of $y'_{1,j} + e_{3,1,j}^{(2)}$ in experiment $H_{3,1,\text{mid}}^{\text{Pre}}$. Since the distribution of $\mathbf{z}_{3,1}^{(1)}$ remains unchanged across the two experiments, the distributions of $\mathbf{z}_{3,1}$ are statistically indistinguishable between $H_{3,0,\text{mid}}^{\text{Pre}}$ and $H_{3,1,\text{mid}}^{\text{Pre}}$. This completes the proof. $\square$

Proof of Lemma 56 (Continued). By Claims 57 and 58, and by applying a standard hybrid argument, we conclude that $H_{3,0}^{\text{Pre}} \stackrel{c}{\approx} H_{3,1}^{\text{Pre}}$. $\square$

Proof of Claim 52 (Continued). Recall that our goal in Claim 52 is to show $H_{0,0}^{\text{Pre}} \stackrel{c}{\approx} H_{0,1}^{\text{Pre}}$. By Lemmas 53~56 and standard hybrid argument, we conclude that no efficient distinguisher can distinguish between experiments $H_{0,0}^{\text{Pre}}$ and $H_{0,1}^{\text{Pre}}$ with non-negligible advantage. Specifically,

$$\begin{aligned} &(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{s}^\top [\mathbf{B} \mid \mathbf{P}] + \mathbf{e}_2^\top + [\mathbf{0} \mid \mathbf{u}_0^\top \mathbf{G}], \{\mathbf{s}_i^\top \mathbf{Q}_i + \mathbf{e}_3^\top\}_{i \in [\ell]}) \\ &\stackrel{c}{\approx} (1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top\}_{i \in [\ell]}, [\mathbf{B} \mid \mathbf{P}], \mathbf{s}^\top [\mathbf{B} \mid \mathbf{P}] + \mathbf{e}_2^\top + [\mathbf{0} \mid \mathbf{u}_1^\top \mathbf{G}], \{\mathbf{s}_i^\top \mathbf{Q}_i + \mathbf{e}_3^\top\}_{i \in [\ell]}). \end{aligned}$$

$\square$

C. Parameters

Let λ be the security parameter.

- We set the lattice dimension $n = \lambda^{1/\epsilon}$ and the modulus $q = 2^{\tilde{O}(n^\epsilon)} = 2^{\tilde{O}(\lambda)}$ for some constant $\epsilon > 0$, where $\tilde{O}(\cdot)$ suppresses constant and logarithmic factors. Then $m = n \lceil \log q \rceil = \tilde{O}(\lambda^{1+1/\epsilon})$.
- We set $L = 2^\lambda$ to support ciphertext policies of arbitrary polynomial size, i.e., $\ell = \text{poly}(\lambda)$.
- For static security (Theorem 51), we set $m' = \tilde{O}(\lambda^{1+1/\epsilon})$, $Q_0 = \text{poly}(\lambda)$ and $\chi' = \Theta(\lambda^{1+1/\epsilon})$. We rely on the $\text{LWE}_{n,m_1,q,\chi_s}$ assumption and the $\text{IND-evIPFE}_{n,m,m',q,\chi,\chi}$ assumption, where the noise parameter $\chi_s = \chi_s(\lambda)$ is set polynomial-bounded and χ satisfies the bound $\chi \geq \lambda^{\omega(1)} \cdot \max\{B_1, \sqrt{\lambda\chi_s}\}$. By setting $B_1 = \text{poly}(\lambda)$, we can choose $\chi = 2^{\tilde{O}(n^\epsilon)}$.
- To ensure correctness (Theorem 50), we require $\chi > \sqrt{n \log q} \cdot \omega(\sqrt{\log n})$, which is also satisfied by setting $\chi = 2^{\tilde{O}(n^\epsilon)}$. In addition, the correctness bound requires $B_0 = \sqrt{\lambda\chi}m + \lambda\chi\chi'm' + \lambda\chi^2mL$, which implies $B_0 = 2^{\tilde{O}(n^\epsilon)}$.

We summarize with the following instantiation:

Corollary 2 ((B_0, B_1)-MA-ABNIPFE for subset policies in the random oracle model). *Let λ be a security parameter. Assuming polynomial hardness of LWE and the IND-evIPFE, both holding under a sub-exponential modulus-to-noise ratio, there exists a statically secure (B_0, B_1)-MA-ABNIPFE scheme for subset policies in the random oracle model with $B_0/B_1 = \text{superpoly}(\lambda)$.*

VIII. NOISELESS MA-ABIPFE SCHEME FROM IND-evIPFE ASSUMPTION (IN THE RANDOM ORACLE MODEL)

In this section, we present a construction of a *noiseless* MA-ABIPFE scheme for subset policies in the random oracle model. This construction is obtained by applying the modulus-switching technique to Construction 1.

Construction 2 (MA-ABIPFE for subset policies in the random oracle model). *Let λ be the security parameter. Let n, p and q be lattice parameters, and define $m = n \lceil \log q \rceil$. Let m', χ , and χ' be additional lattice parameters. Let $\mathcal{AU} = \{0, 1\}^\lambda$ be the universe of authority identifiers, and $\mathcal{GID} = \{0, 1\}^\lambda$ be the universe of global user identifiers. Let $H : \mathcal{GID} \times \mathbb{Z}_p^n \rightarrow \mathbb{Z}_q^{m'}$ be a hash function, modeled as a random oracle which outputs samples drawn from the discrete Gaussian distribution $D_{\mathbb{Z}_q^{m'}, \chi'}$, as in Construction 1.*

A multi-authority attribute-based (noiseless) inner-product functional encryption scheme (MA-ABIPFE) over $\mathbb{Z}_p^n$ for subset policies consists of a tuple of efficient algorithms

$$\Pi_{\text{MA-ABIPFE}} = (\text{GlobalSetup}, \text{AuthSetup}, \text{Keygen}, \text{Enc}, \text{Dec}).$$

The algorithms proceed as follows:

- $\text{GlobalSetup}(1^\lambda) \rightarrow \text{gp}$: The global setup algorithm takes as input the security parameter λ , and outputs the global parameters $\text{gp} = (\lambda, n, m, m', p, q, \chi, \chi', H)$.
- $\text{AuthSetup}(\text{gp}, \text{aid}) \rightarrow (\text{pk}_{\text{aid}}, \text{msk}_{\text{aid}})$: The authority setup algorithm takes as input the global parameters gp and an authority identifier $\text{aid} \in \mathcal{AU}$. It samples $(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, $\mathbf{B}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m'}$, $\mathbf{P}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$. It outputs a public key $\text{pk}_{\text{aid}} = (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$ and a master secret key $\text{msk}_{\text{aid}} = \text{td}_{\text{aid}}$.
- $\text{Keygen}(\text{gp}, \text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}, \text{gid}, \mathbf{v}) \rightarrow \text{sk}_{\text{aid}, \text{gid}, \mathbf{v}}$: The key generation algorithm takes as input the global parameters gp , the public key $\text{pk}_{\text{aid}} = (\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})$, the authority's master secret key $\text{msk}_{\text{aid}} = \text{td}_{\text{aid}}$, the user identifier $\text{gid} \in \mathcal{GID}$, the key vector $\mathbf{v} \in \mathbb{Z}_p^n$ (naturally lifted to $\mathbb{Z}_q^n$ when required). It first computes $\mathbf{r} \leftarrow H(\text{gid}, \mathbf{v})$ and then uses the trapdoor td_{aid} for $\mathbf{A}_{\text{aid}}$ to sample $\mathbf{k} \leftarrow \text{SamplePre}(\mathbf{A}_{\text{aid}}, \text{td}_{\text{aid}}, \mathbf{P}_{\text{aid}} \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{B}_{\text{aid}} \mathbf{r}, \chi)$. It outputs a secret key $\text{sk}_{\text{aid}, \text{gid}, \mathbf{v}} = \mathbf{k}$.
- $\text{Enc}(\text{gp}, \{\text{pk}_{\text{aid}}\}_{\text{aid} \in A}, \mathbf{u}) \rightarrow \text{ct}$: The encryption algorithm takes as input the global parameters gp , a set of public keys $\{\text{pk}_{\text{aid}}\}_{\text{aid} \in A} = \{(\mathbf{A}_{\text{aid}}, \mathbf{B}_{\text{aid}}, \mathbf{P}_{\text{aid}})\}_{\text{aid} \in A}$ associated with authorities $A \subseteq \mathcal{AU}$, and

a plaintext vector $\mathbf{u} \in \mathbb{Z}_p^n$. For each $\text{aid} \in A$, it samples $\mathbf{s}_{\text{aid}} \xleftarrow{\$} \mathbb{Z}_q^n$, $\mathbf{e}_{1,\text{aid}} \leftarrow D_{\mathbb{Z},\chi}^m$. It also samples $\mathbf{e}_2 \leftarrow D_{\mathbb{Z},\chi}^{m'}$, and $\mathbf{e}_3 \leftarrow D_{\mathbb{Z},\chi}^m$. It outputs the ciphertext $\text{ct} \in \mathbb{Z}_q^m \times \mathbb{Z}_q^{m'} \times \mathbb{Z}_q^m$, where

$$\text{ct} = \left(\left\{ \mathbf{s}_{\text{aid}}^\top \mathbf{A}_{\text{aid}} + \mathbf{e}_{1,\text{aid}}^\top \right\}_{\text{aid} \in A}, \sum_{\text{aid} \in A} \mathbf{s}_{\text{aid}}^\top \mathbf{B}_{\text{aid}} + \mathbf{e}_2^\top, \sum_{\text{aid} \in A} \mathbf{s}_{\text{aid}}^\top \mathbf{P}_{\text{aid}} + \mathbf{e}_3^\top + \lceil \mathbf{u}^\top \rceil_{p \rightarrow q} \mathbf{G} \right).$$

- $\text{Dec}(\text{gp}, \{\text{sk}_{\text{aid},\text{gid},\mathbf{v}}\}_{\text{aid} \in A}, \text{gid}, \mathbf{v}, \text{ct}) \rightarrow \Gamma$: The decryption algorithm takes as input the global parameters gp , a collection of secret keys $\text{sk}_{\text{aid},\text{gid},\mathbf{v}} = \mathbf{k}_{\text{aid},\text{gid},\mathbf{v}}$ associated with authorities $\text{aid} \in A$, a user identifier $\text{gid} \in \mathcal{GID}$, a key vector $\mathbf{v} \in \mathbb{Z}_p^n$, and a ciphertext $\text{ct} = (\{\mathbf{c}_{1,\text{aid}}\}_{\text{aid} \in A}, \mathbf{c}_2^\top, \mathbf{c}_3^\top)$. It computes $\mathbf{r} \leftarrow \text{H}(\text{gid}, \mathbf{v})$ and outputs

$$\Gamma = \lceil \mathbf{c}_3^\top \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{c}_2^\top \mathbf{r} - \sum_{\text{aid} \in A} \mathbf{c}_{1,\text{aid}}^\top \mathbf{k}_{\text{aid},\text{gid},\mathbf{v}} \rceil_{q \rightarrow p}.$$

A. Correctness

Theorem 59 (Correctness). *Let $\chi_0 = \sqrt{n \log q} \cdot \omega(\sqrt{\log n})$ be a polynomial such that Lemma 7 holds. Suppose that the lattice parameters $n, m, m', p, q, \chi, \chi'$ satisfy the following conditions:*

- $\chi \geq \chi_0(n, q)$.
- $q > np^2 + 2pB_0$, where $B_0 = \sqrt{\lambda}\chi m + \lambda\chi\chi'm' + \lambda\chi^2 m\ell$.

Then the scheme $\Pi_{\text{MA-ABIPFE}}$ in Construction 2 is correct as a noiseless MA-ABIPFE scheme.

Proof. The proof idea follows the same structure as that of Theorem 24. Take any plaintext vector $\mathbf{u} \in \mathbb{Z}_p^n$, any key vector $\mathbf{v} \in \mathbb{Z}_p^n$, an arbitrary set of authorities $\{\text{aid}\}_{\text{aid} \in A}$, and an arbitrary user identifier $\text{gid} \in \mathcal{GID}$. First, sample the global parameters $\text{gp} \leftarrow \text{GlobalSetup}(1^\lambda)$, and generate the authority keys $(\text{pk}_{\text{aid}}, \text{msk}_{\text{aid}}) \leftarrow \text{AuthSetup}(\text{gp}, \text{aid})$ for each $\text{aid} \in A$. Then, generate the secret keys $\text{sk}_{\text{aid},\text{gid},\mathbf{v}} = \mathbf{k}_{\text{aid},\text{gid},\mathbf{v}} \leftarrow \text{Keygen}(\text{gp}, \text{msk}_{\text{aid}}, \text{gid}, \mathbf{v})$ for each $\text{aid} \in A$, and obtain the resulting ciphertext $\text{ct} \leftarrow \text{Enc}(\text{gp}, \{\text{pk}_{\text{aid}}\}_{\text{aid} \in A}, \mathbf{u})$.

The decryption algorithm outputs:

$$\Gamma = \left\lceil \lceil \mathbf{u}^\top \rceil_{p \rightarrow q} \cdot \mathbf{v} + \mathbf{e}_3^\top \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{e}_2^\top \mathbf{r} - \sum_{\text{aid} \in A} \mathbf{e}_{1,\text{aid}}^\top \mathbf{k}_{\text{aid},\text{gid},\mathbf{v}} \right\rceil_{q \rightarrow p}.$$

Suppose that $\lceil \mathbf{u}^\top \rceil_{p \rightarrow q} = \frac{q}{p} \mathbf{u}^\top + \delta_{\mathbf{u}}^\top$ with $\|\delta_{\mathbf{u}}\| \leq 1/2$. Substituting into the expression for Γ , we obtain

$$\begin{aligned} \Gamma &= \left\lceil \frac{q}{p} \mathbf{u}^\top \mathbf{v} + \delta_{\mathbf{u}}^\top \mathbf{v} + \mathbf{e}_3^\top \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{e}_2^\top \mathbf{r} - \sum_{\text{aid} \in A} \mathbf{e}_{1,\text{aid}}^\top \mathbf{k}_{\text{aid},\text{gid},\mathbf{v}} \right\rceil_{q \rightarrow p} \\ &= \frac{p}{q} \left(\frac{q}{p} \mathbf{u}^\top \mathbf{v} + \delta_{\mathbf{u}}^\top \mathbf{v} + \mathbf{e}_3^\top \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{e}_2^\top \mathbf{r} - \sum_{\text{aid} \in A} \mathbf{e}_{1,\text{aid}}^\top \mathbf{k}_{\text{aid},\text{gid},\mathbf{v}} \right) + \delta' \\ &= \mathbf{u}^\top \mathbf{v} + \frac{p}{q} \left(\delta_{\mathbf{u}}^\top \mathbf{v} + \mathbf{e}_3^\top \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{e}_2^\top \mathbf{r} - \sum_{\text{aid} \in A} \mathbf{e}_{1,\text{aid}}^\top \mathbf{k}_{\text{aid},\text{gid},\mathbf{v}} \right) + \delta' \in \mathbb{Z}_p, \end{aligned}$$

for some $|\delta'| \leq 1/2$. Since both Γ and $\mathbf{u}^\top \mathbf{v}$ lie in $\mathbb{Z}_p$, we have $\Gamma = \mathbf{u}^\top \mathbf{v}$ if and only if

$$|e'| = \left| \frac{p}{q} \left(\delta_{\mathbf{u}}^\top \mathbf{v} + \mathbf{e}_3^\top \mathbf{G}^{-1}(\mathbf{v}) + \mathbf{e}_2^\top \mathbf{r} - \sum_{\text{aid} \in A} \mathbf{e}_{1,\text{aid}}^\top \mathbf{k}_{\text{aid},\text{gid},\mathbf{v}} \right) + \delta' \right| < 1.$$

According to the noise bounds analysis in Theorem 24, we can get the upper bound

$$|e'| \leq \frac{1}{2q} np^2 + \frac{p}{q} B_0 + \frac{1}{2},$$

where $B_0 = \sqrt{\lambda}\chi m + \lambda\chi\chi'm' + \lambda\chi^2mL$. Thus, when $q > np^2 + 2pB_0$, it holds that the total rounding error $|e'| < 1$, which ensures that $\Gamma = \mathbf{u}^\top \mathbf{v}$. Therefore, $\Pi_{\text{MA-ABIPFE}}$ is correct as a noiseless MA-ABIPFE scheme. $\square$

B. Static Security

The proof follows essentially the same structure and reduction strategy as that of Theorem 51, and is therefore omitted.

Theorem 60 (Static Security). *Let $\chi_0(n, q) = \sqrt{n \log q} \cdot \omega(\sqrt{\log n})$ be a polynomial such that Lemma 7 holds. Let Q_0 be an upper bound on the number of secret-key queries submitted by the adversary $\mathcal{A}$. Suppose that the following conditions hold:*

- $m' > 6n \log q$.
- Let χ' be an error distribution such that $\chi' = \Omega(\sqrt{n \log q})$.
- Let χ be an error distribution parameter such that $\chi \geq \max\{\chi_0, \lambda^{\omega(1)} \cdot \sqrt{\lambda}\chi_s\}$, where χ_s is an error parameter such that $\text{LWE}_{n, m_1, q, \chi_s}$ assumption holds for some $m_1 = \text{poly}(m, m', Q_0)$.
- The assumption $\text{IND-evIPFE}_{n, m, m', q, \chi, \chi}$ holds.

Construction 2 is statically secure as a noiseless MA-ABIPFE scheme.

C. Parameters

Let λ be the security parameter.

- We set the lattice dimension $n = \lambda^{1/\epsilon}$ and the modulus $q = 2^{\tilde{O}(n^\epsilon)} = 2^{\tilde{O}(\lambda)}$ for some constant $\epsilon > 0$, where $\tilde{O}(\cdot)$ suppresses constant and logarithmic factors. Then $m = n \lceil \log q \rceil = \tilde{O}(\lambda^{1+1/\epsilon})$.
- We set $L = 2^\lambda$ to support ciphertext policies of arbitrary polynomial size, i.e., $\ell = \text{poly}(\lambda)$.
- For static security (Theorem 60), we set $m' = \tilde{O}(\lambda^{1+1/\epsilon})$, $Q_0 = \text{poly}(\lambda)$ and $\chi' = \Omega(\lambda^{1+1/\epsilon})$. The construction relies on the $\text{LWE}_{n, m_1, q, \chi_s}$ assumption and the $\text{IND-evIPFE}_{n, m, m', q, \chi, \chi}$ assumption, where the noise parameter $\chi_s = \chi_s(\lambda)$ is polynomial-bounded and χ must satisfy the bound $\chi \geq \lambda^{\omega(1)} \cdot \sqrt{\lambda}\chi_s$. We can set $\chi = 2^{\tilde{O}(n^\epsilon)}$.
- To ensure correctness (Theorem 59), we require $\chi > \sqrt{n \log q} \cdot \omega(\sqrt{\log n})$, which can also be satisfied by setting $\chi = 2^{\tilde{O}(n^\epsilon)}$. In addition, the correctness bound requires $q > np^2 + 2pB_0$ where $B_0 = \sqrt{\lambda}\chi m + \lambda\chi\chi'm' + \lambda\chi^2m\ell = 2^{\tilde{O}(n^\epsilon)}$, and $p = 2^{\tilde{O}(n^\epsilon)}$ is chosen accordingly to meet this bound.

We summarize with the following instantiation:

Corollary 3 (Noiseless MA-ABIPFE for Subset Policies in the Random Oracle Model). *Let λ be a security parameter. Assuming polynomial hardness of LWE and the IND-evIPFE, both holding under a sub-exponential modulus-to-noise ratio, there exists a statically-secure noiseless MA-ABIPFE scheme for subset policies in the random oracle model.*

REFERENCES

- [ABDP15] Michel Abdalla, Florian Bourse, Angelo De Caro, and David Pointcheval. Simple functional encryption schemes for inner products. In *Public-Key Cryptography - PKC 2015*, pages 733–751, 2015.
- [ABDP16] Michel Abdalla, Florian Bourse, Angelo De Caro, and David Pointcheval. Better security for functional encryption for inner product evaluations. Cryptology ePrint Archive, Paper 2016/011, 2016.
- [ABKW19] Michel Abdalla, Fabrice Benhamouda, Markulf Kohlweiss, and Hendrik Waldner. Decentralizing inner-product functional encryption. In *Public-Key Cryptography - PKC 2019*, pages 128–157, 2019.
- [ABM⁺20] Michel Abdalla, Florian Bourse, Hugo Marival, David Pointcheval, Azam Soleimani, and Hendrik Waldner. Multi-client inner-product functional encryption in the random-oracle

- model. In *Security and Cryptography for Networks - 12th International Conference*, pages 525–545, 2020.
- [ACF⁺18] Michel Abdalla, Dario Catalano, Dario Fiore, Romain Gay, and Bogdan Ursu. Multi-input functional encryption for inner products: Function-hiding realizations and constructions without pairings. In *Advances in Cryptology - CRYPTO 2018*, pages 597–627, 2018.
- [ACGU20] Michel Abdalla, Dario Catalano, Romain Gay, and Bogdan Ursu. Inner-product functional encryption with fine-grained access control. In *Advances in Cryptology - ASIACRYPT 2020*, pages 467–497, 2020.
- [Agr19] Shweta Agrawal. Indistinguishability obfuscation without multilinear maps: New methods for bootstrapping and instantiation. In *Advances in Cryptology - EUROCRYPT 2019*, pages 191–225, 2019.
- [AGRW17] Michel Abdalla, Romain Gay, Mariana Raykova, and Hoeteck Wee. Multi-input inner-product functional encryption from pairings. In *Advances in Cryptology - EUROCRYPT 2017*, pages 601–626, 2017.
- [AGT21] Shweta Agrawal, Rishab Goyal, and Junichi Tomida. Multi-party functional encryption. In *Theory of Cryptography - 19th International Conference, TCC 2021*, pages 224–255, 2021.
- [ALS16] Shweta Agrawal, Benoît Libert, and Damien Stehlé. Fully secure functional encryption for inner products, from standard assumptions. In *Advances in Cryptology - CRYPTO 2016*, pages 333–362, 2016.
- [AP20] Shweta Agrawal and Alice Pellet-Mary. Indistinguishability obfuscation without maps: Attacks and fixes for noisy linear FE. In *Advances in Cryptology - EUROCRYPT 2020*, pages 110–140, 2020.
- [Ban93] Wojciech Banaszczyk. New bounds in some transference theorems in the geometry of numbers. *Math. Annalen*, 296(4):625–635, 1993.
- [BBL17] Fabrice Benhamouda, Florian Bourse, and Helger Lipmaa. CCA-secure inner-product functional encryption from projective hash functions. In *Public-Key Cryptography - PKC 2017*, pages 36–66, 2017.
- [BDE⁺18] Jonathan Bootle, Claire Delaplace, Thomas Espitau, Pierre-Alain Fouque, and Mehdi Tibouchi. LWE without modular reduction and improved side-channel attacks against BLISS. In *Advances in Cryptology - ASIACRYPT 2018*, pages 494–524, 2018.
- [Bei96] Amos Beimel. *Secure schemes for secret sharing and key distribution*. PhD thesis, Technion - Israel Institute of Technology, Israel, 1996.
- [BLMR13] Dan Boneh, Kevin Lewi, Hart Montgomery, and Ananth Raghunathan. Key homomorphic PRFs and their applications. In *Annual Cryptology Conference*, pages 410–428, 2013.
- [BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *CCS '93, Proceedings of the 1st ACM Conference on Computer and Communications Security*, pages 62–73, 1993.
- [BSW11] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In *Theory of Cryptography - 8th Theory of Cryptography Conference, TCC*, pages 253–273, 2011.
- [BSW13] Karyn Benson, Hovav Shacham, and Brent Waters. The k-BDH assumption family: Bilinear map cryptography from progressively weaker assumptions. In *Topics in Cryptology - CT-RSA 2013*, pages 310–325, 2013.
- [BTVW17] Zvika Brakerski, Rotem Tsabary, Vinod Vaikuntanathan, and Hoeteck Wee. Private constrained PRFs (and more) from LWE. In *Theory of Cryptography - 15th International Conference, TCC 2017*, pages 264–302, 2017.
- [BÜW24] Chris Brzuska, Akin Ünal, and Ivy K. Y. Woo. Evasive LWE assumptions: Definitions, classes, and counterexamples. In *Advances in Cryptology - ASIACRYPT 2024*, pages 418–449, 2024.
- [CHL⁺15] Jung Hee Cheon, Kyoohyung Han, Changmin Lee, Hansol Ryu, and Damien Stehlé. Cryptanalysis of the multilinear map over the integers. In *Advances in Cryptology-EUROCRYPT*

- 2015, pages 3–12, 2015.
- [CSG⁺18] J      Chotard, Edouard Dufour Sans, Romain Gay, Duong Hieu Phan, and David Pointcheval. Decentralized multi-client functional encryption for inner product. In *Advances in Cryptology - ASIACRYPT 2018*, pages 703–732, 2018.
 - [CVW18] Yilei Chen, Vinod Vaikuntanathan, and Hoeteck Wee. GGH15 beyond permutation branching programs: proofs, attacks, and candidates. In *Advances in Cryptology-CRYPTO 2018*, pages 577–607, 2018.
 - [DDM16] Pratish Datta, Ratna Dutta, and Sourav Mukhopadhyay. Functional encryption for inner product with full function privacy. In *Public-Key Cryptography - PKC 2016*, pages 164–195, 2016.
 - [DKW21a] Pratish Datta, Ilan Komargodski, and Brent Waters. Decentralized multi-authority ABE for DNFs from LWE. In *Advances in Cryptology - EUROCRYPT*, pages 177–209, 2021.
 - [DKW21b] Pratish Datta, Ilan Komargodski, and Brent Waters. Decentralized multi-authority ABE for NC   from computational-BDH. Cryptology ePrint Archive, Paper 2021/1325, 2021.
 - [DP23] Pratish Datta and Tapas Pal. Decentralized multi-authority attribute-based inner-product FE: large universe and unbounded. In *Public-Key Cryptography - PKC 2023*, pages 587–621, 2023.
 - [Gay20] Romain Gay. A new paradigm for public-key functional encryption for degree-2 polynomials. In *Public-Key Cryptography - PKC 2020*, pages 95–120, 2020.
 - [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *Proc. 41st Annu. ACM Symp. Theory Comput.*, pages 197–206, 2008.
 - [HJL21] Sam Hopkins, Aayush Jain, and Huijia Lin. Counterexamples to new circular security assumptions underlying iO. In *Advances in Cryptology-CRYPTO 2021*, pages 673–700. Springer, 2021.
 - [HLL24] Yao-Ching Hsieh, Huijia Lin, and Ji Luo. A general framework for lattice-based ABE using evasive inner-product functional encryption. In *Advances in Cryptology - EUROCRYPT 2024*, pages 433–464, 2024.
 - [JLLS23] Aayush Jain, Huijia Lin, Paul Lou, and Amit Sahai. Polynomial-time cryptanalysis of the subspace flooding assumption for post-quantum iO. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 205–235, 2023.
 - [JLS19] Aayush Jain, Huijia Lin, and Amit Sahai. Simplifying constructions and assumptions for iO. *IACR Cryptol. ePrint Arch*, page 1252, 2019.
 - [LT19] Beno  t Libert and Radu Titiu. Multi-client functional encryption for linear functions in the standard model from LWE. In *Advances in Cryptology - ASIACRYPT 2019*, pages 520–551, 2019.
 - [LW11] Allison B. Lewko and Brent Waters. Decentralizing attribute-based encryption. In *Advances in Cryptology - EUROCRYPT 2011*, pages 568–588, 2011.
 - [MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *Advances in Cryptology - EUROCRYPT 2012*, pages 700–718, 2012.
 - [MR07] Daniele Micciancio and Oded Regev. Worst-case to average-case reductions based on Gaussian measures. *SIAM J. Comput.*, 37(1):267–302, 2007.
 - [Reg09] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6):1–40, 2009.
 - [RW15] Yannis Rouselakis and Brent Waters. Efficient statically-secure large-universe multi-authority attribute-based encryption. In *Financial Cryptography and Data Security*, pages 315–332, 2015.
 - [Tom19] Junichi Tomida. Tightly secure inner product functional encryption: Multi-input and function-hiding constructions. In *Advances in Cryptology - ASIACRYPT 2019*, pages 459–488, 2019.
 - [Wee22] Hoeteck Wee. Optimal broadcast encryption and CP-ABE from evasive lattice assumptions.

In *Advances in Cryptology - EUROCRYPT 2022*, pages 217–241, 2022.

[WFL19] Zhedong Wang, Xiong Fan, and Feng-Hao Liu. FE for inner products and its application to decentralized ABE. In *IACR international workshop on public key cryptography*, pages 97–127, 2019.

[WWW22] Brent Waters, Hoeteck Wee, and David J. Wu. Multi-authority ABE from lattices without random oracles. In *Theory of Cryptography - 20th International Conference, TCC 2022*, pages 651–679, 2022.

APPENDIX A HARDNESS OF EVASIVE IPFE

A. Evasive LWE from [WWW22]

We recall the evasive LWE assumption proposed in [WWW22].

Theorem 61 (Evasive LWE [WWW22]). *Let $\lambda \in \mathbb{N}$ be a security parameter, and let n, q, m, K, χ, χ' be lattice parameters specified by λ . Denote $\text{gp} = (1^\lambda, q, 1^n, 1^m, 1^K, 1^\chi, 1^{\chi'})$. Let Samp be a sampling algorithm, which takes as input the global parameter gp , and outputs a matrix $\mathbf{C} \in \mathbb{Z}_q^{n \times K}$, a set of matrices $\mathbf{Q}_1 \in \mathbb{Z}_q^{n \times k_1}, \dots, \mathbf{Q}_\ell \in \mathbb{Z}_q^{n \times k_\ell}$, and auxiliary information $\text{aux} \in \{0, 1\}^*$.*

For two adversaries $\mathcal{A}_0$ and $\mathcal{A}_1$, we define their advantage functions as follows:

$$\begin{aligned} \text{Adv}_{\text{Samp}, \mathcal{A}_0}^{\text{Pre}}(\lambda) &:= \left| \Pr [\mathcal{A}_0(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{z}_{1,i}^\top\}_{i \in [\ell]}, \mathbf{C}, \mathbf{z}_2^\top, \{\mathbf{z}_{3,i}^\top\}_{i \in [\ell]}) = 1] \right. \\ &\quad \left. - \Pr [\mathcal{A}_0(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \boldsymbol{\delta}_{1,i}^\top\}_{i \in [\ell]}, \mathbf{C}, \boldsymbol{\delta}_2^\top, \{\boldsymbol{\delta}_{3,i}^\top\}_{i \in [\ell]}) = 1] \right|; \\ \text{Adv}_{\text{Samp}, \mathcal{A}_1}^{\text{Post}}(\lambda) &:= \left| \Pr [\mathcal{A}_1(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{z}_{1,i}^\top\}_{i \in [\ell]}, \mathbf{C}, \mathbf{z}_2^\top, \{\mathbf{K}_i\}_{i \in [\ell]}) = 1] \right. \\ &\quad \left. - \Pr [\mathcal{A}_1(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \boldsymbol{\delta}_{1,i}^\top\}_{i \in [\ell]}, \mathbf{C}, \boldsymbol{\delta}_2^\top, \{\mathbf{K}_i\}_{i \in [\ell]}) = 1] \right|; \end{aligned}$$

where the parameters are sampled as follows:

- $(\mathbf{B}, \mathbf{Q}_1, \dots, \mathbf{Q}_\ell, \text{aux}) \leftarrow \text{Samp}(\text{gp})$,
- $(\mathbf{A}_1, \text{td}_1), \dots, (\mathbf{A}_\ell, \text{td}_\ell) \leftarrow \text{TrapGen}(1^n, 1^m, q)$,
- $\mathbf{s}_1, \dots, \mathbf{s}_\ell \xleftarrow{\$} \mathbb{Z}_q^n, \mathbf{s}^\top \leftarrow [\mathbf{s}_1^\top \mid \dots \mid \mathbf{s}_\ell^\top] \in \mathbb{Z}_q^{n\ell}$,
- $\mathbf{e}_{1,i} \leftarrow D_{\mathbb{Z}, \chi}^m, \mathbf{e}_{3,i} \leftarrow D_{\mathbb{Z}, \chi}^{k_i}$ for each $i \in [\ell], \mathbf{e}_2 \leftarrow D_{\mathbb{Z}, \chi}^K$,
- $\boldsymbol{\delta}_{1,i} \xleftarrow{\$} \mathbb{Z}_q^m, \boldsymbol{\delta}_{3,i} \xleftarrow{\$} \mathbb{Z}_q^{k_i}$ for each $i \in [\ell], \boldsymbol{\delta}_2 \xleftarrow{\$} \mathbb{Z}_q^K$,
- $\mathbf{z}_{1,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{A}_i + \mathbf{e}_{1,i}^\top, \mathbf{z}_{3,i}^\top \leftarrow \mathbf{s}_i^\top \mathbf{Q}_i + \mathbf{e}_{3,i}^\top$ for each $i \in [\ell], \mathbf{z}_2^\top \leftarrow \mathbf{s}^\top \mathbf{C} + \mathbf{e}_2^\top$,
- $\mathbf{K}_i \leftarrow \text{SamplePre}(\mathbf{A}_i, \text{td}_i, \mathbf{Q}_i, \chi')$ for each $i \in [\ell]$.

We say that the $\text{evLWE}_{n,m,K,q,\chi,\chi'}$ assumption holds, if for all efficient samplers $\mathcal{S}$, the following implication holds: If there exists an efficient adversary $\mathcal{A}_1$ with a non-negligible advantage function $\text{Adv}_{\text{Samp}, \mathcal{A}_1}^{\text{Post}}(\lambda)$, then there exists another efficient adversary $\mathcal{A}_0$ with a non-negligible advantage function $\text{Adv}_{\text{Samp}, \mathcal{A}_0}^{\text{Pre}}(\lambda)$.

B. Reduction from evLWE ([WWW22]) to evIPFE

Theorem 62. *Suppose that the assumption $\text{evLWE}_{n,m,K,q,\chi,\chi'}$ holds, where $K = m + m'$, then the assumption $\text{evIPFE}_{n,m,m',q,\chi,\chi'}$ holds.*

Proof. Suppose, for contradiction, that $\text{evIPFE}_{n,m,m',q,\chi,\chi'}$ does not hold with respect to some sampler $\mathcal{S} = (\mathcal{S}_v, \mathcal{S}_u)$. First, we begin by constructing a sampler Samp for the $\text{evLWE}_{n,m,K,q,\chi,\chi'}$ assumption. On input the global parameter $\text{gp} = (1^\lambda, q, 1^n, 1^m, 1^K, 1^\chi, 1^{\chi'})$, the sampler Samp proceeds as follows.

- 1) It runs $\mathcal{S}_v(\text{gp}')$ where $\text{gp}' = (1^\lambda, q, 1^n, 1^m, 1^{m'}, 1^\chi, 1^{\chi'})$, which outputs

$$\begin{aligned} &1^\ell, \{1^{k_i}\}_{i \in [\ell]}; \\ &(\mathbf{r}_{1,1}, \mathbf{v}_{1,1}), \dots, (\mathbf{r}_{1,k_1}, \mathbf{v}_{1,k_1}); \\ &\dots \\ &(\mathbf{r}_{\ell,1}, \mathbf{v}_{\ell,1}), \dots, (\mathbf{r}_{\ell,k_\ell}, \mathbf{v}_{\ell,k_\ell}). \end{aligned}$$

2) It samples $\mathbf{C} \xleftarrow{\$} \mathbb{Z}_q^{n\ell \times K}$, and parses it as

$$\mathbf{C} = [\mathbf{B} \mid \mathbf{P}] = \left[\begin{array}{c|c} \mathbf{B}_1 & \mathbf{P}_1 \\ \vdots & \vdots \\ \mathbf{B}_\ell & \mathbf{P}_\ell \end{array} \right],$$

where $\mathbf{B} \in \mathbb{Z}_q^{n\ell \times m'}$, $\mathbf{P} \in \mathbb{Z}_q^{n\ell \times m}$ and $\mathbf{B}_i \in \mathbb{Z}_q^{n \times m'}$, $\mathbf{P}_i \in \mathbb{Z}_q^{n \times m}$ for each $i \in [\ell]$.

3) It constructs each $\mathbf{Q}_i$ as

$$\mathbf{Q}_i \leftarrow [\mathbf{B}_i \mid \mathbf{P}_i] \left[\begin{array}{c|c|c} \mathbf{r}_{i,1} & \cdots & \mathbf{r}_{i,k_i} \\ \mathbf{G}^{-1}(\mathbf{v}_{i,1}) & \cdots & \mathbf{G}^{-1}(\mathbf{v}_{i,k_i}) \end{array} \right] \in \mathbb{Z}_q^{n \times k_i}$$

4) It then outputs $(\mathbf{B}, \mathbf{Q}_1, \dots, \mathbf{Q}_\ell, \text{aux})$, where aux consists of the randomness used by $\mathcal{S}_v$.

We claim that the assumption $\text{evLWE}_{n,m,m',q,\chi,\chi'}$ does not hold with respect to the sampler Samp. Suppose that there exists an adversary $\mathcal{A}_1$ such that breaks the evIPFE assumption. We then construct an adversary $\mathcal{A}'_1$ that breaks the evLWE assumption, using $\mathcal{A}_1$ as a subroutine. The adversary $\mathcal{A}'_1$ proceeds as follows:

- 1) Adversary $\mathcal{A}'_1$ begins by receiving a evIPFE challenge $(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{y}_{1,i}^\top\}_{i \in [\ell]}, \mathbf{C}, \mathbf{y}_2^\top, \{\mathbf{K}_i\}_{i \in [\ell]})$, where $\mathbf{A}_i \in \mathbb{Z}_q^{n \times m}$, $\mathbf{z}_{1,i} \in \mathbb{Z}_q^m$, $\mathbf{K}_i \in \mathbb{Z}_q^{m \times k_i}$ for each $i \in [\ell]$, and $\mathbf{C} \in \mathbb{Z}_q^{n\ell \times K}$, $\mathbf{z}_2 \in \mathbb{Z}_q^{n \times K}$.
- 2) It samples $\mathbf{r}_{\text{pri}} \xleftarrow{\$} \{0, 1\}^\kappa$, where κ denotes the upper bounds of the random bits used by $\mathcal{S}_u$, runs $\mathbf{u} \leftarrow \mathcal{S}_u(\text{gp}, \mathbf{r}_{\text{pub}}; \mathbf{r}_{\text{pri}})$, and sends the tuple

$$(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_1, \mathbf{y}_{1,i}^\top\}_{i \in [\ell]}, \mathbf{C}, \mathbf{y}_2^\top + [\mathbf{0}_{m'} \mid \mathbf{u}^\top \mathbf{G}], \{\mathbf{K}_i\}_{i \in [\ell]})$$

to $\mathcal{A}_1$.

3) Finally, adversary $\mathcal{A}'_1$ outputs whatever $\mathcal{A}_1$ outputs.

By the definition of $\mathcal{S}$ and Samp above, adversary $\mathcal{A}'_1$ simulates perfectly the challenger of the evLWE distinguishing game. The tuple sent to $\mathcal{A}_1$ has the same distribution as the two possible types of evIPFE challenge instances, depending on which one is received.

It remains to show that the parameters provided by $\mathcal{A}'_1$ satisfies the precondition of the evLWE assumption, i.e.,

$$(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{z}_{1,i}^\top\}_{i \in [\ell]}, \mathbf{C}, \mathbf{z}_2^\top, \{\mathbf{z}_{3,i}^\top\}_{i \in [\ell]}) \stackrel{c}{\approx} (1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \boldsymbol{\delta}_{1,i}^\top\}_{i \in [\ell]}, \mathbf{C}, \boldsymbol{\delta}_2^\top, \{\boldsymbol{\delta}_{3,i}^\top\}_{i \in [\ell]}),$$

then the corresponding evIPFE precondition naturally holds, i.e.,

$$(1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \mathbf{z}_{1,i}^\top\}_{i \in [\ell]}, \mathbf{C}, \mathbf{z}_2^\top + \mathbf{u}^\top \mathbf{G}, \{\mathbf{z}_{3,i}^\top\}_{i \in [\ell]}) \stackrel{c}{\approx} (1^\lambda, \mathbf{r}_{\text{pub}}, \{\mathbf{A}_i, \boldsymbol{\delta}_{1,i}^\top\}_{i \in [\ell]}, \mathbf{C}, \boldsymbol{\delta}_2^\top, \{\boldsymbol{\delta}_{3,i}^\top\}_{i \in [\ell]}),$$

where the parameters $\mathbf{z}_{1,i}, \mathbf{z}_2, \mathbf{z}_{3,i}, \boldsymbol{\delta}_{1,i}, \boldsymbol{\delta}_2, \boldsymbol{\delta}_{3,i}$ are sampled as in Theorem 61. This equivalence of distributions follows directly from the parameter construction defined in the reduction procedure. $\square$